



COURSE SLIDES · WSQ

AI Vibe Coding for iOS Mobile Apps Development

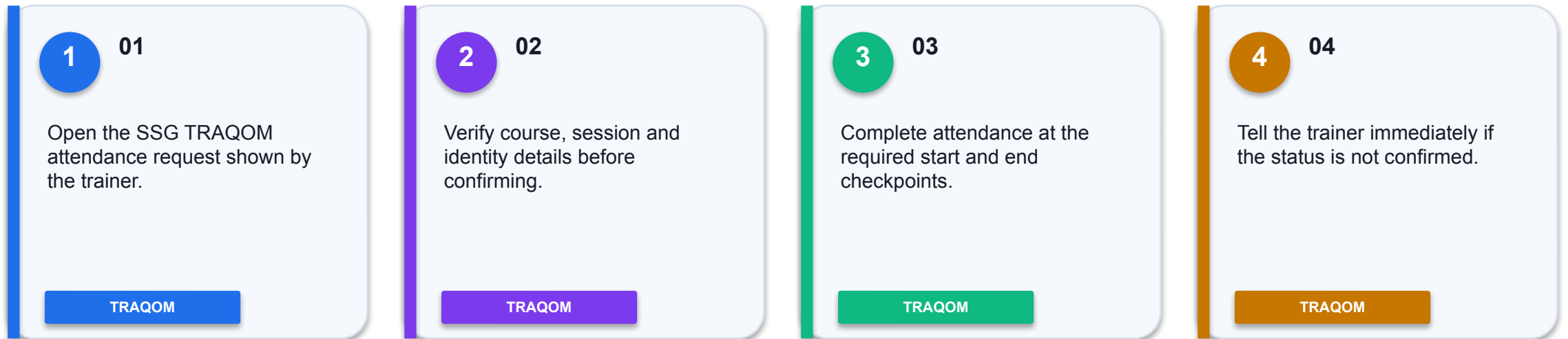
Portable C++ core · Objective-C++ adapter · SwiftUI shell

TGS-2023037545 · Software Design 3 · ICT-DES-3005-1.1
Tertiary Infotech Academy Pte Ltd · UEN 201200696W
Trainer: Dr Alfred Ang

v1.2 · 12 September 2026



Digital Attendance · TRAQOM



DESIGN JUDGEMENT

TRAQOM records SSG digital attendance. It is not a course survey.

About the Trainer



General Trainer

Template profile

NAME

TITLE / DESIGNATION

QUALIFICATIONS

AREAS OF EXPERTISE

TRAINING EXPERIENCE

CONTACT

About the Trainer



Dr Alfred Ang

Lead Trainer · AI and Software
Engineering

ROLE

Founder and principal trainer

CERTIFICATION

Adult educator and technology specialist

DELIVERS

AI, mobile, cloud and software design programmes

INDUSTRY FOCUS

Applied architecture and product delivery

TRAINING APPROACH

Mechanism-led, evidence-backed practice

CONTACT

enquiry@tertiaryinfotech.com

Learner Introduction

1

Experience

Share your Swift, C++ and iOS development background.

LEARNER

2

Outcome

Name one iOS capability you want to design and validate.

LEARNER

3

Evidence

Describe how you currently decide that software works.

LEARNER

4

Boundary

Identify one area where AI assistance needs review.

LEARNER

DESIGN JUDGEMENT

Use the introductions to calibrate examples without changing the regulated learning outcomes.

Working Agreements

1 Safe

Use mock data; never paste credentials or personal information.

GOVERNANCE

2 Scoped

Protect unrelated files and approved requirements.

GOVERNANCE

3 Observable

Run checks and retain evidence for material claims.

GOVERNANCE

4 Accountable

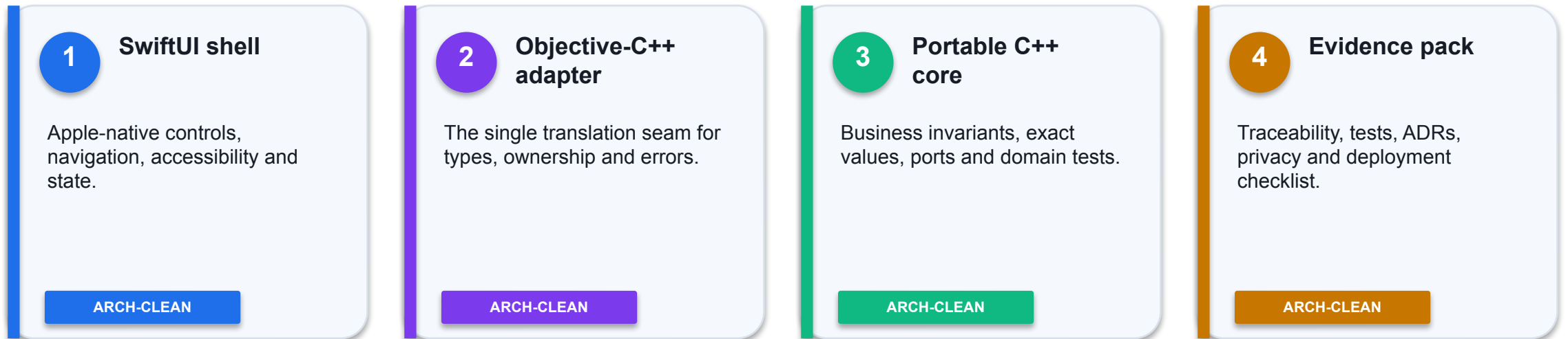
Human review owns consequential design and release decisions.

GOVERNANCE

DESIGN JUDGEMENT

Ask early when a requirement, permission or acceptance criterion is unclear.

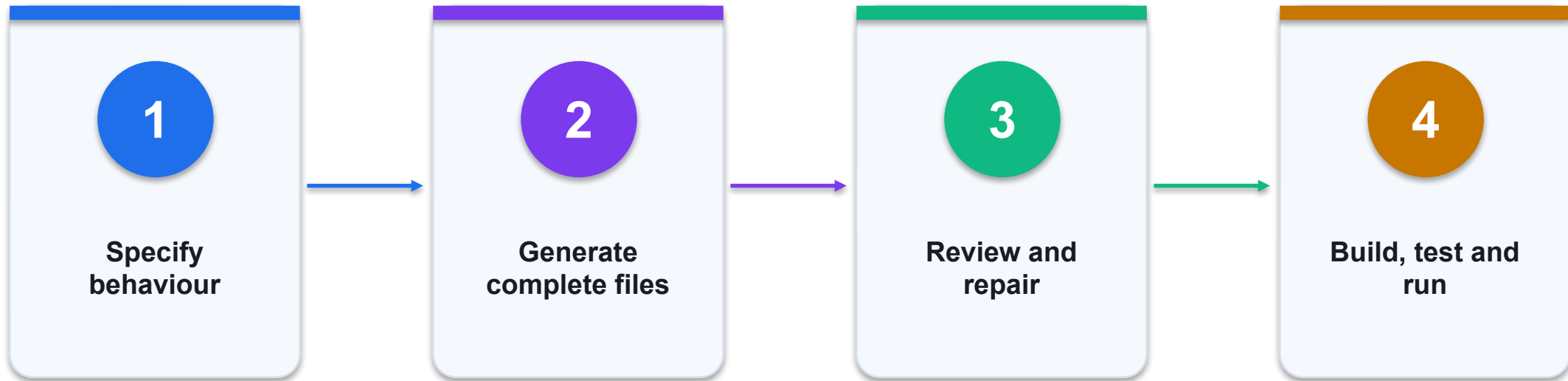
Course Architecture



DESIGN JUDGEMENT

Swift supplements the regulated C++ scope; it does not replace it.

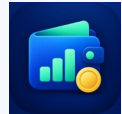
You Specify · AI Generates · Xcode Proves



MECHANISM

Swift appears as review vocabulary for generated iOS code. Learners own the requirement, protected interfaces, diagnostic and acceptance evidence; Xcode and the Simulator decide whether the proposal works.

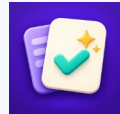
Three Runnable Apps · One Evidence-Driven Journey



BudgetBuddy

Regulated portable C++ policy, Objective-C++ seam, Observation state and accessible SwiftUI.

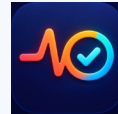
THREE-APP-JOURNEY



FocusCards

Modern SwiftUI flashcard interaction inspired by the legacy course project, generated and tested as a complete app.

THREE-APP-JOURNEY



HabitPulse

Stable-identity habit actions that make red → repair → green practice visible in the Simulator.

THREE-APP-JOURNEY



Release gate

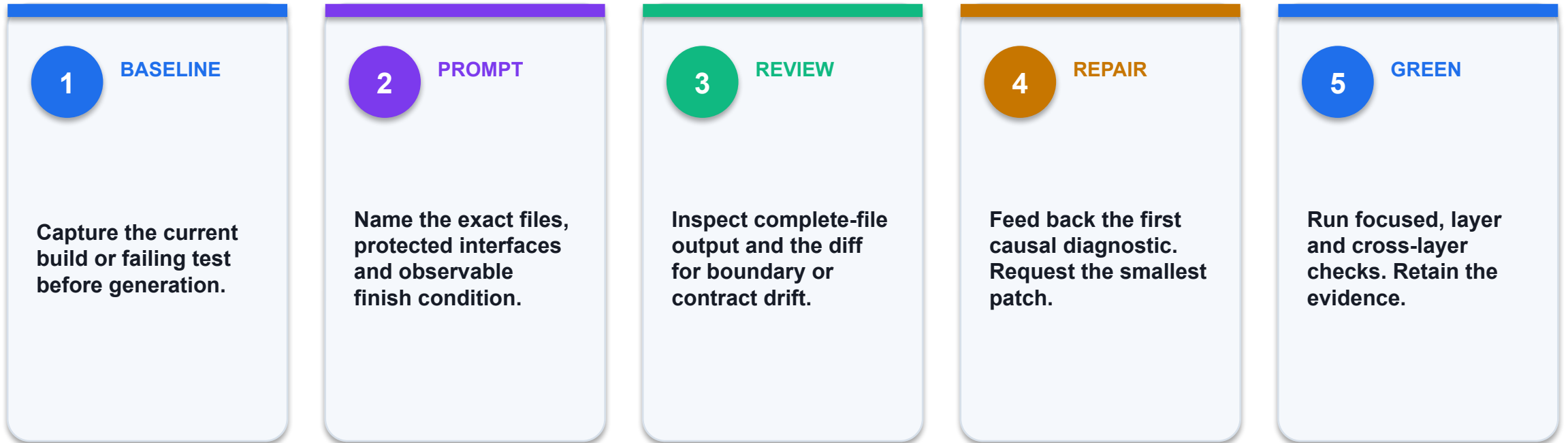
Every app regenerates from project.yml, builds, tests and launches before any credentialled App Store gate.

THREE-APP-JOURNEY

DESIGN JUDGEMENT

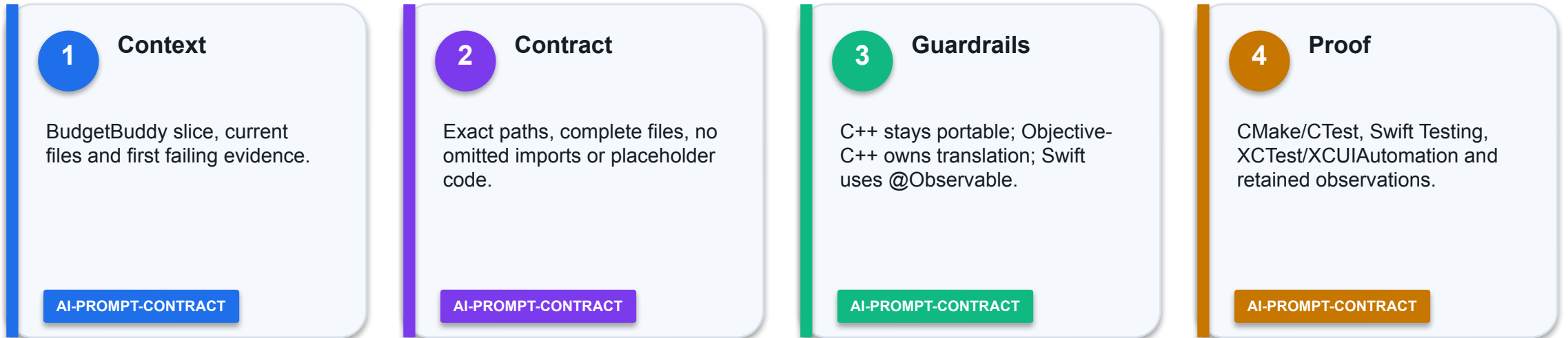
Legacy UIKit examples contribute app concepts only. The learner package uses current SwiftUI, Observation, Swift Testing and XCTest UI automation.

Vibe Coding Loop · From Context to Evidence



Prompt output contract: FILE: relative/path, then complete code. No ellipses, TODOs, placeholders or unverified run claims.

Prompt Anatomy · A Bounded Code-Generation Contract



DESIGN JUDGEMENT

A useful prompt constrains the change and defines evidence. The assistant proposes code; the learner verifies behaviour and owns acceptance.

Course Outline

1 Topic 1

Requirements and the Portable Component Boundary

COURSE-MAP

2 Topic 2

Vibe-Generated Portable Logic and Quality Gates

COURSE-MAP

3 Topic 3

AI-Generated SwiftUI App Shell and Controls

COURSE-MAP

4 Topic 4

iOS Interoperability, Persistence and Validation

COURSE-MAP

DESIGN JUDGEMENT

Topic 5 completes the evidence and deployment thread; ten Activities progressively build BudgetBuddy.

Lesson Plan · Day 1

1 09:30–11:00 Requirements, component boundaries and Activity 1. SCHEDULE	2 11:00–13:00 Portable C++ design and Activity 2. SCHEDULE	3 14:00–16:00 C++ methods, tools and AI-assisted TDD. SCHEDULE	4 16:00–18:30 Activities 3–4, reflection and evidence review. SCHEDULE
---	--	--	--

DESIGN JUDGEMENT

Detailed timings, methods and slide references are in the Lesson Plan.

Lesson Plan · Day 2

1 09:30–11:15 SwiftUI controls and Objective-C++ interoperability. SCHEDULE	2 11:15–13:00 Activities 5–6 and state validation. SCHEDULE	3 14:00–16:30 Activities 7–10, documentation and release evidence. SCHEDULE	4 16:30–18:30 Written Assessment 1 h · Practical Performance 1 h. SCHEDULE
---	---	---	--

DESIGN JUDGEMENT

Both assessments retain the approved one-hour duration.

Regulated Learning Outcomes

1 LO1

Determine basic software components using C++ methodologies to meet functional specifications.

LO1-LO4

2 LO2

Apply C++ methodologies and tools for software creation.

LO1-LO4

3 LO3

Select essential C++ controls and features to meet software design requirements.

LO1-LO4

4 LO4

Examine the interoperability and functionality of C++ software components.

LO1-LO4

DESIGN JUDGEMENT

LO5: Generate C++ design documentation aligned with user specifications.

Knowledge and Ability Pairing

1

K1 / A1

Design requirements for simple, basic software components. Apply through the corresponding Activity evidence.

K/A-MAP

2

K2 / A2

Basic software design tools and techniques. Apply through the corresponding Activity evidence.

K/A-MAP

3

K3 / A3

Types of controls, elements and features in software. Apply through the corresponding Activity evidence.

K/A-MAP

4

K4 / A4

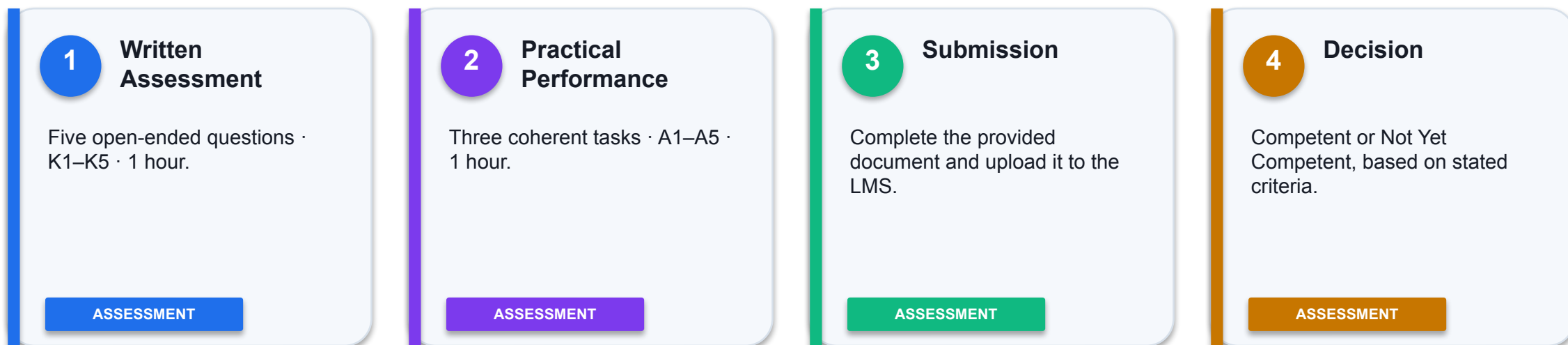
Indicators of software functionality and interoperability. Apply through the corresponding Activity evidence.

K/A-MAP

DESIGN JUDGEMENT

K5 / A5: Documentation of design details. Produce detailed design documentation mapped to user specifications.

Briefing for Assessment



DESIGN JUDGEMENT

Candidate papers contain instructions and grading on page 2; questions and scenario begin on page 3.

Assessment and Funding

1 Attendance

Meet the required attendance threshold and complete TRAQOM.

ASSESSMENT

2 Evidence

Submit both assessment instruments through the LMS.

ASSESSMENT

3 Authenticity

Complete individual work and explain your implementation.

ASSESSMENT

4 Support

Ask the assessor to clarify instructions, not supply answers.

ASSESSMENT

DESIGN JUDGEMENT

Assessment answer keys are trainer-only and never linked from learner channels.

Access the Course Activities

1 Clone

Use Git when you want version history and can run a terminal command.

[ACTIVITY-ACCESS](#)

2 Download ZIP

Use Code → Download ZIP when Git is not available on the learner device.

[ACTIVITY-ACCESS](#)

3 Verify identity

Confirm the repository name and course code TGS-2023037545 before using files.

[ACTIVITY-ACCESS](#)

4 Use the guides

Open activities/README.md, then enter the matching numbered Activity folder.

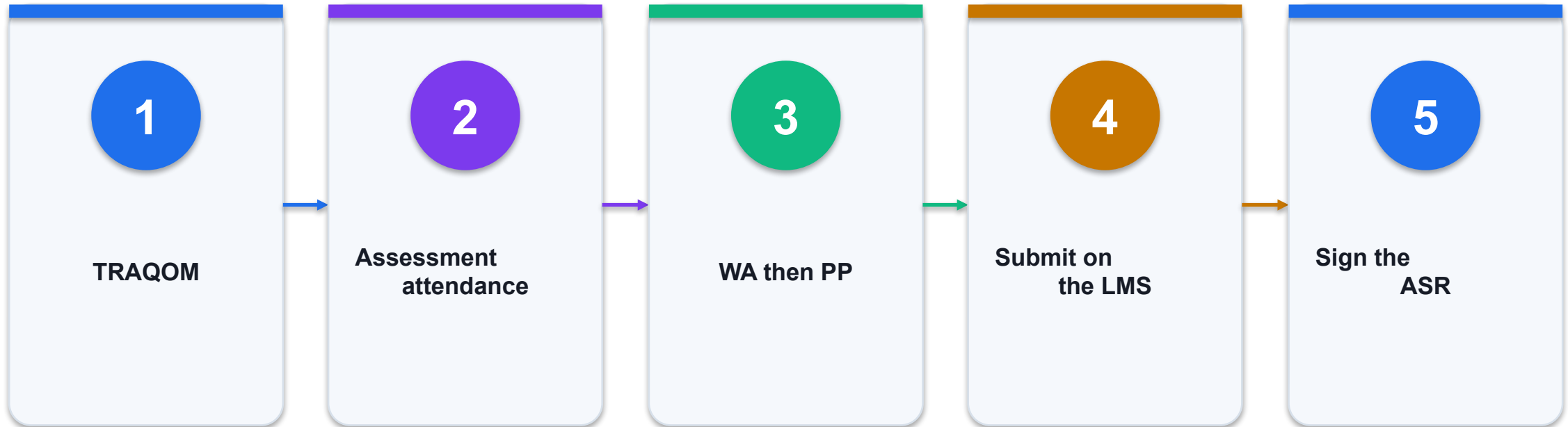
[ACTIVITY-ACCESS](#)

DESIGN JUDGEMENT

Clone: `git clone https://github.com/tertiarycourses/TGS-2023037545-AI-Vibe-Coding-for-iOS-Mobile-Apps-Development.git`

<https://github.com/tertiarycourses/TGS-2023037545-AI-Vibe-Coding-for-iOS-Mobile-Apps-Development>

Assessment Flow



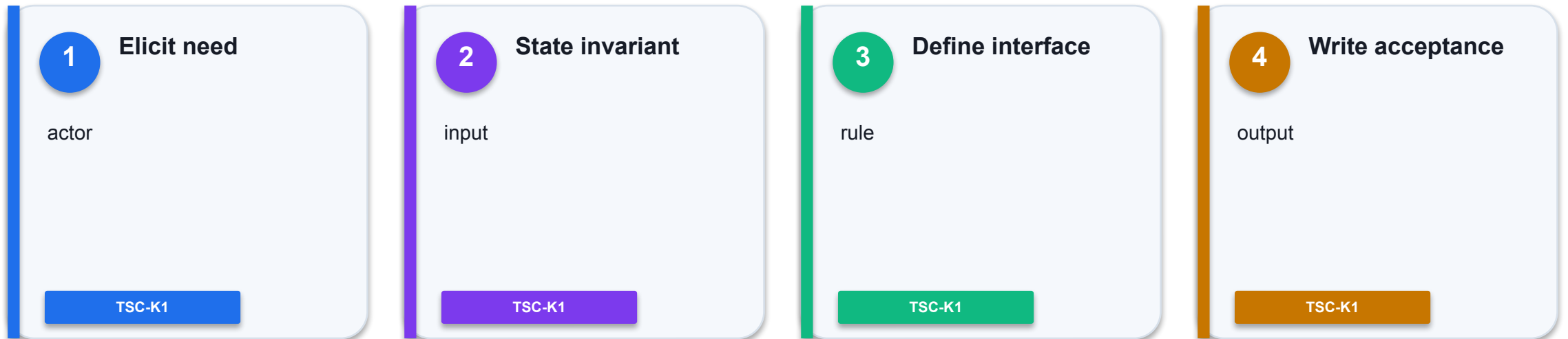
ASR = Assessment Summary Record · complete the sign-off only after both instruments are submitted.

TOPIC 1 · LO1 · K1 · A1

Requirements and the Portable Component Boundary

LO1: Determine basic software components using C++ methodologies to meet functional specifications.

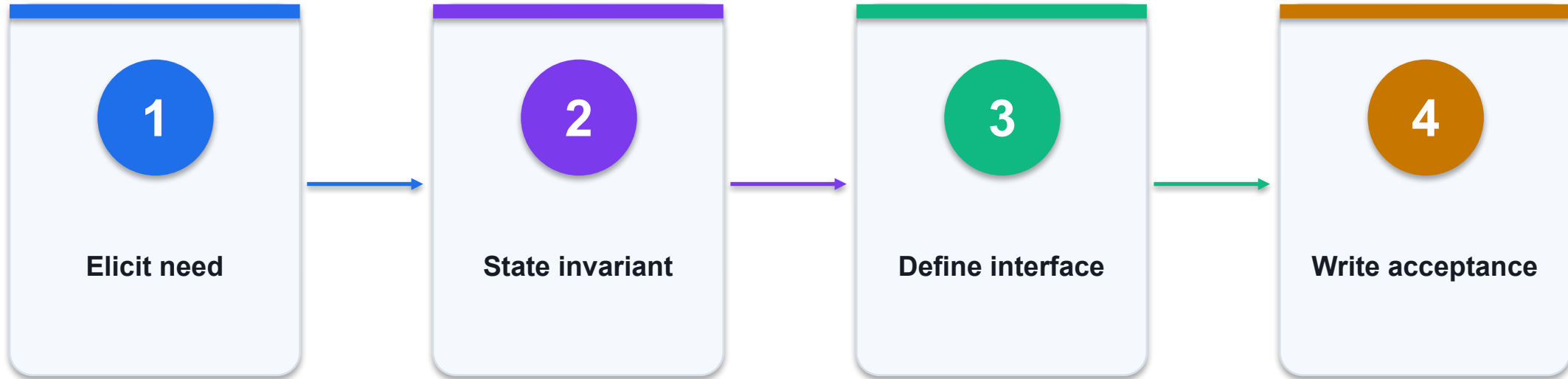
Functional requirements become contracts



DESIGN JUDGEMENT

Translate each user story into an invariant and testable acceptance criterion.

Functional requirements become contracts · Mechanism



MECHANISM

Failure mode: A prompt says build an app but never defines observable success. **Control:** Translate each user story into an invariant and testable acceptance criterion.

Functional requirements become contracts · Technical Anchor

```
struct Requirement { std::string id; bool testable; };
```

1

actor

2

input

3

rule

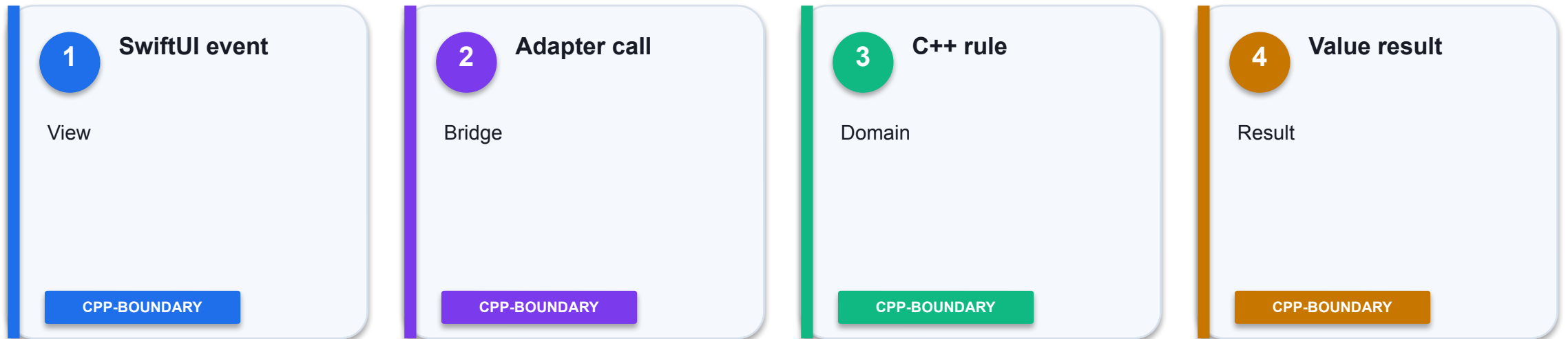
4

output

EVIDENCE METRIC

Accepted behaviours / specified behaviours

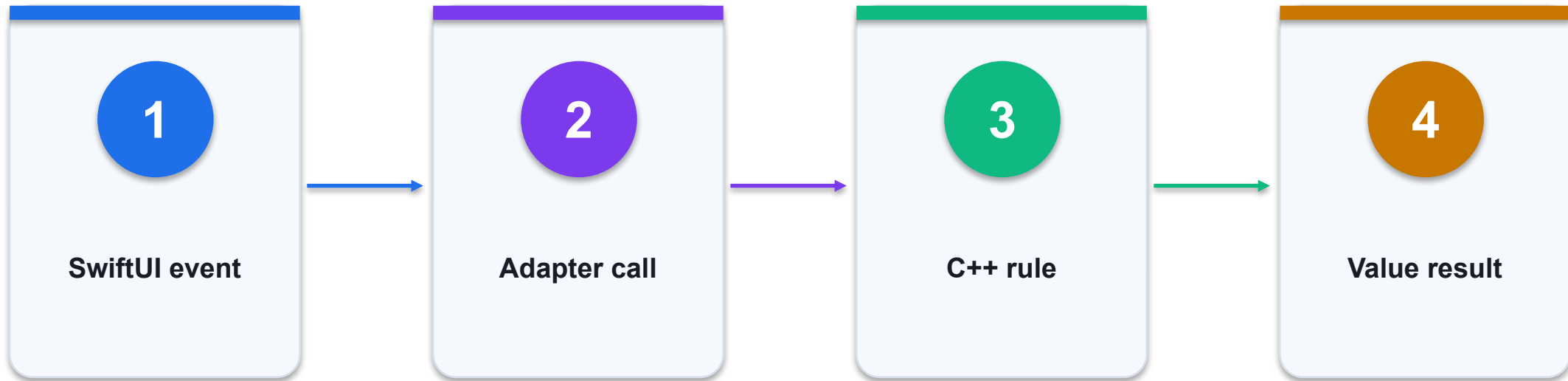
Separate business rules from the iOS shell



DESIGN JUDGEMENT

Keep portable policy in C++; keep presentation and Apple frameworks in Swift.

Separate business rules from the iOS shell · Mechanism



MECHANISM

Failure mode: Rules leak into View bodies and cannot be reused or isolated. **Control:** Keep portable policy in C++; keep presentation and Apple frameworks in Swift.

Separate business rules from the iOS shell - Technical Anchor

```
class BudgetPolicy { public: Decision evaluate(const  
Input&) const; };
```

1

View

2

Bridge

3

Domain

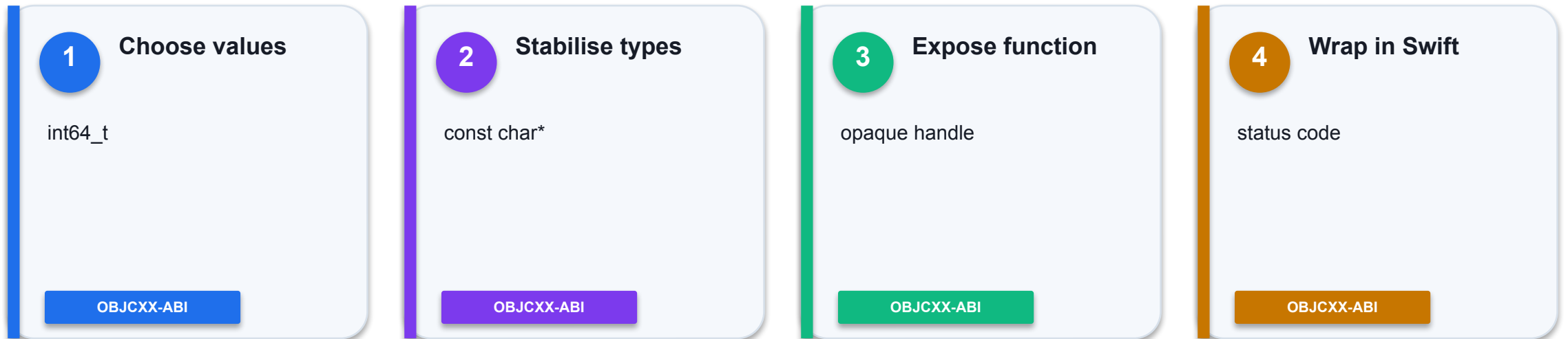
4

Result

EVIDENCE METRIC

Domain tests runnable without Simulator

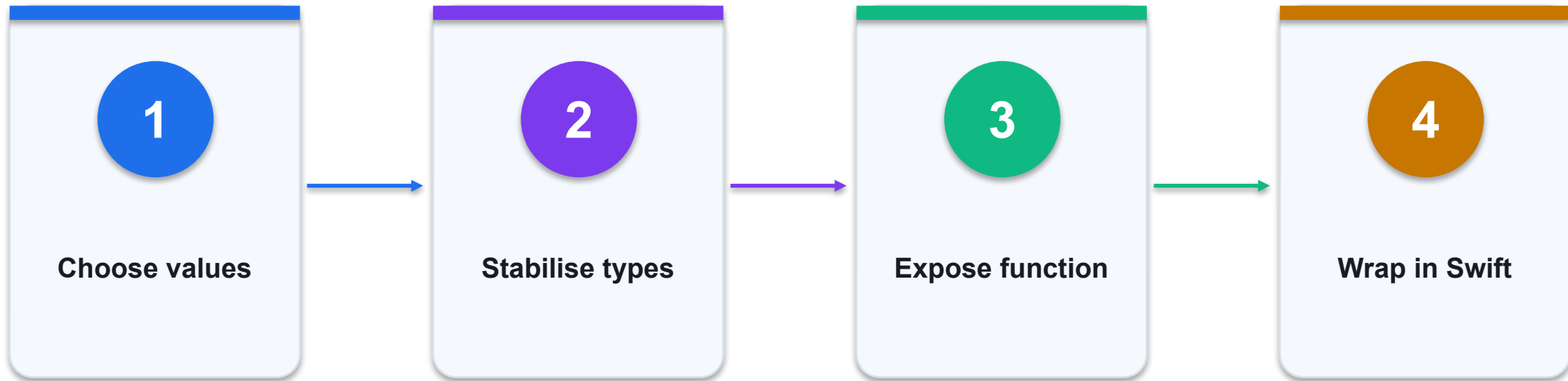
Define a narrow C ABI seam



DESIGN JUDGEMENT

Expose C-compatible data at the boundary; hide implementation behind the adapter.

Define a narrow C ABI seam · Mechanism



MECHANISM

Failure mode: C++ templates or STL containers cross directly into Swift. **Control:** Expose C-compatible data at the boundary; hide implementation behind the adapter.

Define a narrow C ABI seam · Technical Anchor

```
extern "C" BudgetResult budget_evaluate(BudgetInput  
input);
```

1

int64_t

2

const char*

3

opaque handle

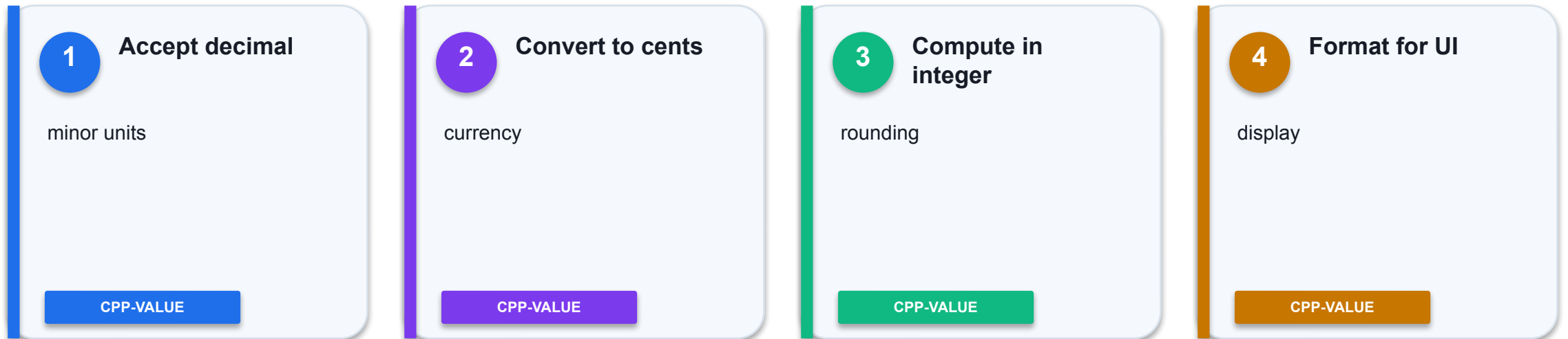
4

status code

EVIDENCE METRIC

Boundary surface measured in stable operations

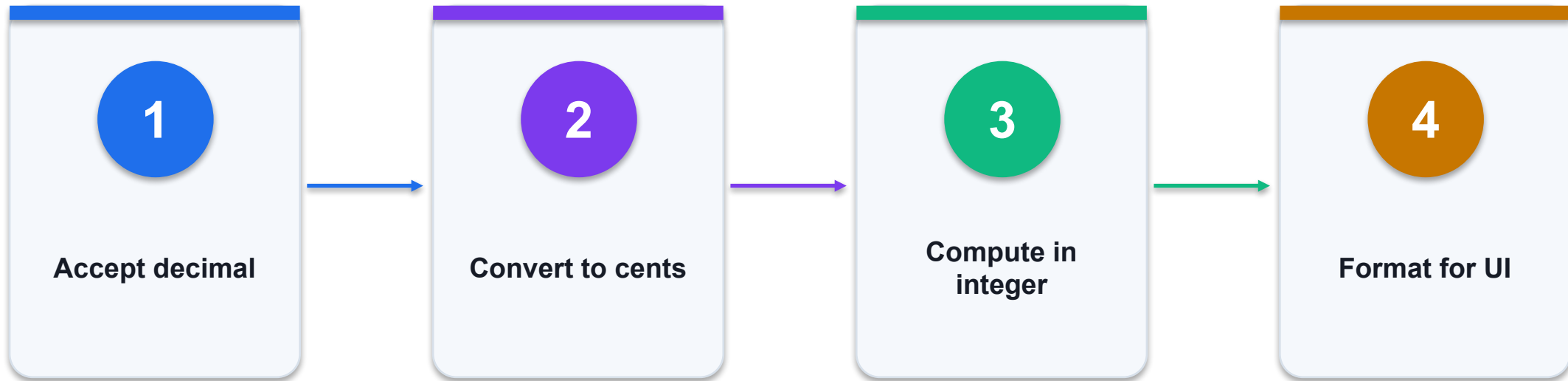
Model money without floating-point drift



DESIGN JUDGEMENT

Represent monetary amounts as signed 64-bit minor units in the domain.

Model money without floating-point drift · Mechanism



MECHANISM

Failure mode: Binary floating point creates a one-cent mismatch. **Control:** Represent monetary amounts as signed 64-bit minor units in the domain.

Model money without floating-point drift · Technical Anchor

```
struct Money { std::int64_t cents; };
```

1

minor units

2

currency

3

rounding

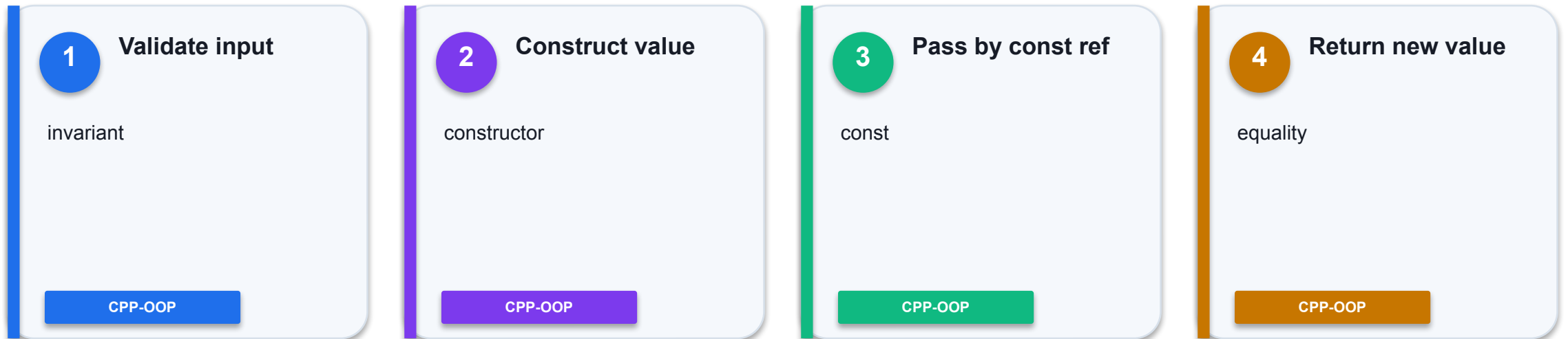
4

display

EVIDENCE METRIC

Exact equality for ledger totals

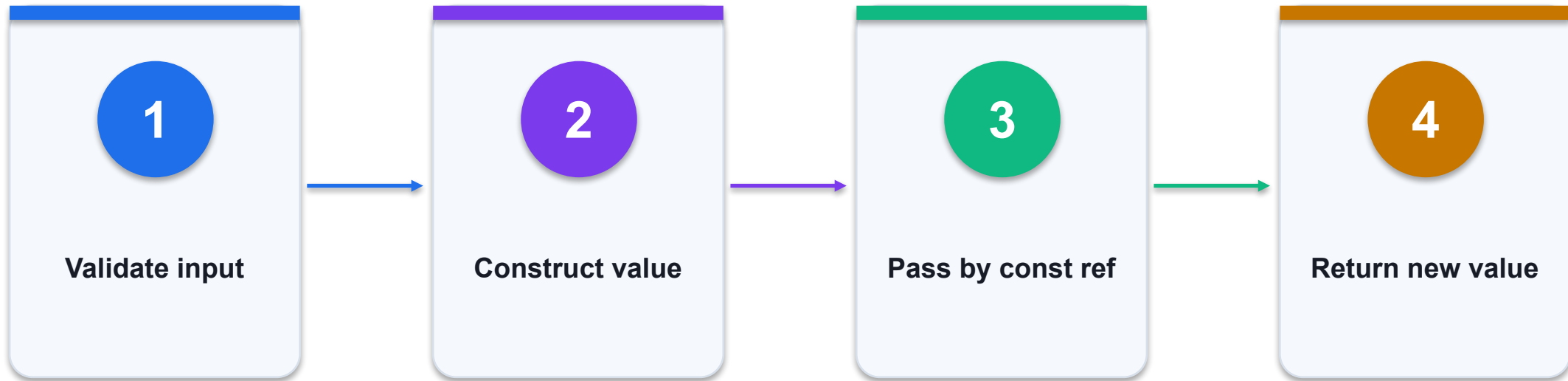
Design immutable value objects



DESIGN JUDGEMENT

Make invariants constructor-level and expose values through read-only accessors.

Design immutable value objects · Mechanism



MECHANISM

Failure mode: Partially initialised objects travel through the system. **Control:** Make invariants constructor-level and expose values through read-only accessors.

Design immutable value objects · Technical Anchor

```
class Category { std::string value_; public: explicit  
Category(std::string); };
```

1

invariant

2

constructor

3

const

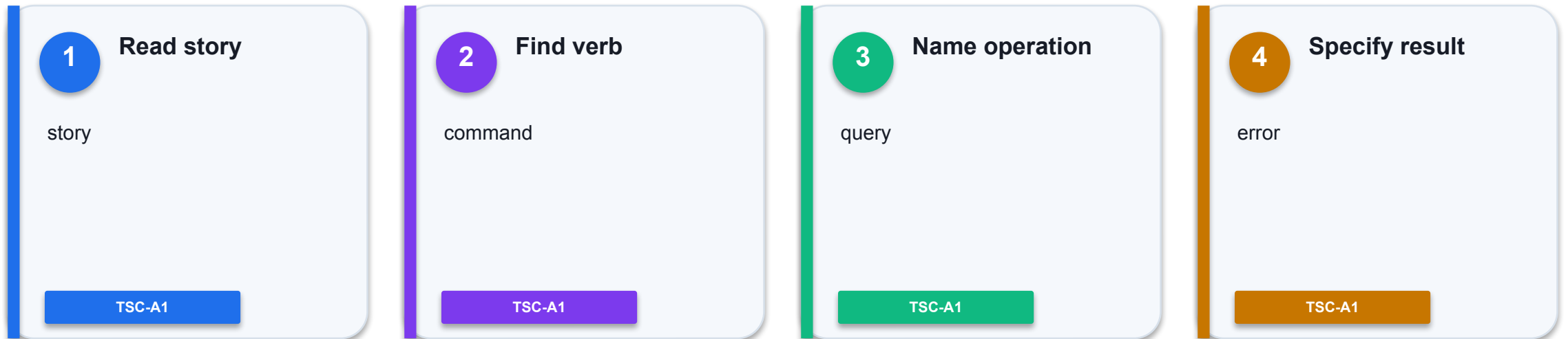
4

equality

EVIDENCE METRIC

Invalid states rejected at construction

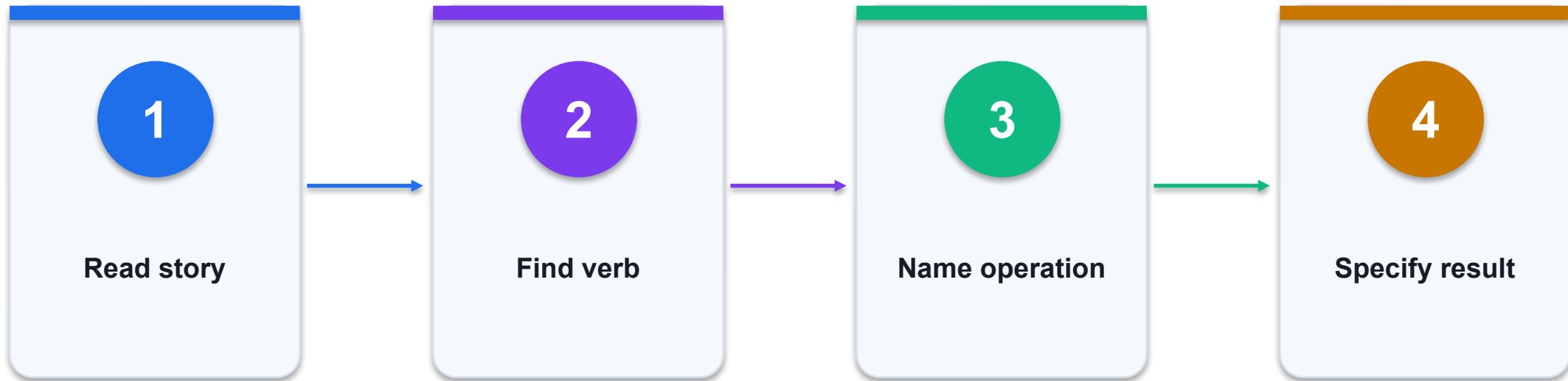
Map a user story to an interface



DESIGN JUDGEMENT

Name operations after domain decisions, then let SwiftUI adapt them.

Map a user story to an interface · Mechanism



MECHANISM

Failure mode: The API mirrors screen widgets instead of business intent. **Control:** Name operations after domain decisions, then let SwiftUI adapt them.

Map a user story to an interface · Technical Anchor

```
virtual AddResult addExpense(const ExpenseDraft&) = 0;
```

1

story

2

command

3

query

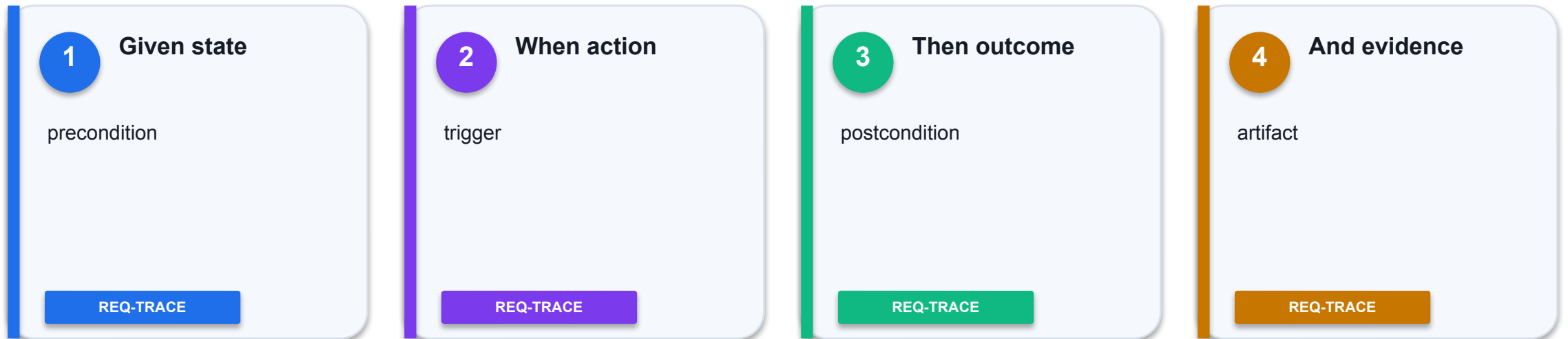
4

error

EVIDENCE METRIC

Traceable story-to-operation mapping

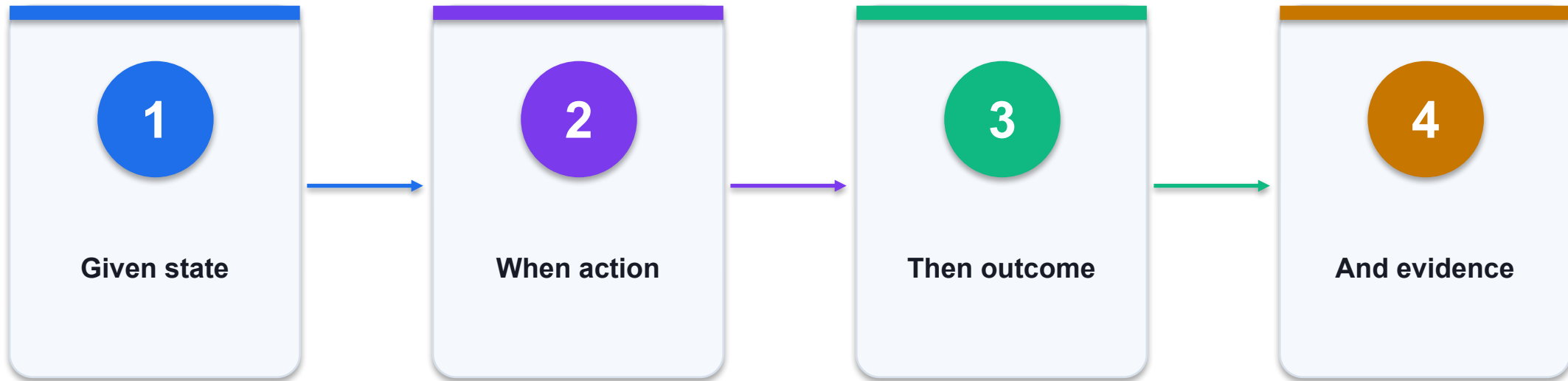
Use acceptance criteria as design pressure



DESIGN JUDGEMENT

Write externally observable outcomes before selecting controls or tools.

Use acceptance criteria as design pressure · Mechanism



MECHANISM

Failure mode: Acceptance criteria describe implementation steps rather than outcomes. **Control:** Write externally observable outcomes before selecting controls or tools.

Use acceptance criteria as design pressure · Technical Anchor

```
// Given valid draft, when added, then total increases  
by exact cents.
```

1

precondition

2

trigger

3

postcondition

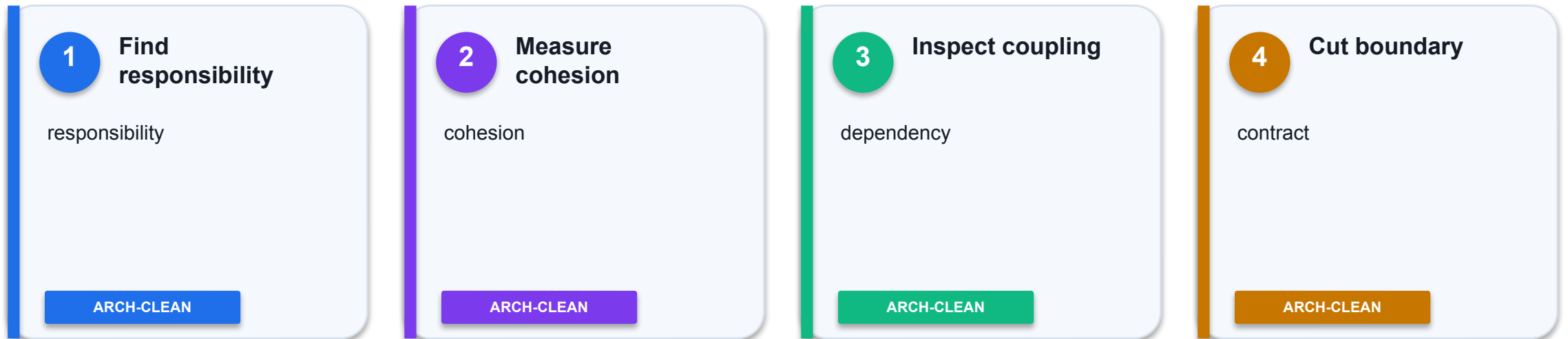
4

artifact

EVIDENCE METRIC

Criteria with deterministic evidence

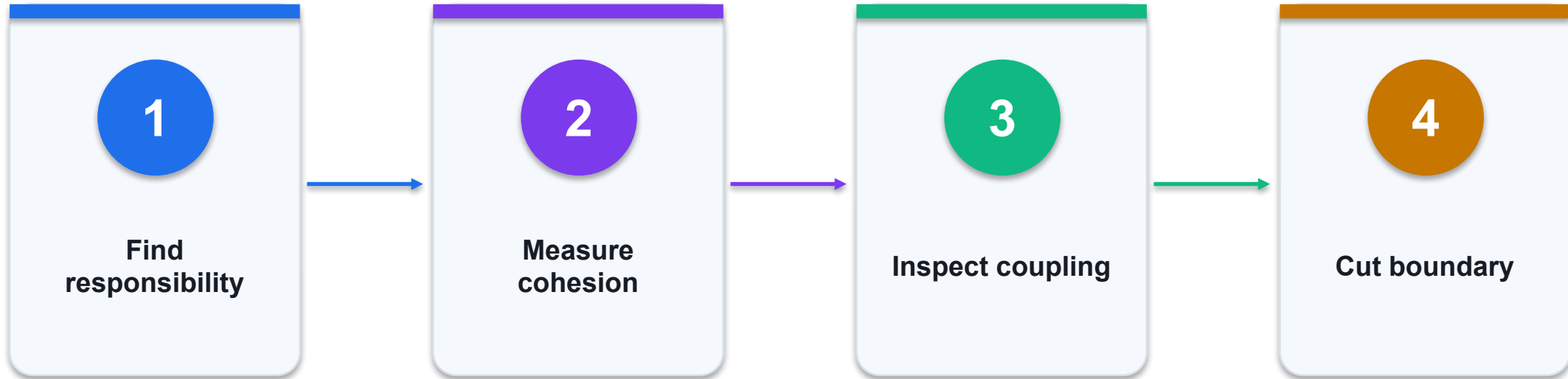
Choose component granularity



DESIGN JUDGEMENT

Partition by responsibility and dependency direction, not by file-count targets.

Choose component granularity · Mechanism



MECHANISM

Failure mode: A single Manager object owns UI, persistence, validation and policy. **Control:** Partition by responsibility and dependency direction, not by file-count targets.

Choose component granularity · Technical Anchor

```
ExpenseService -> ExpenseRepository; ViewModel ->
ExpenseService;
```

1

responsibility

2

cohesion

3

dependency

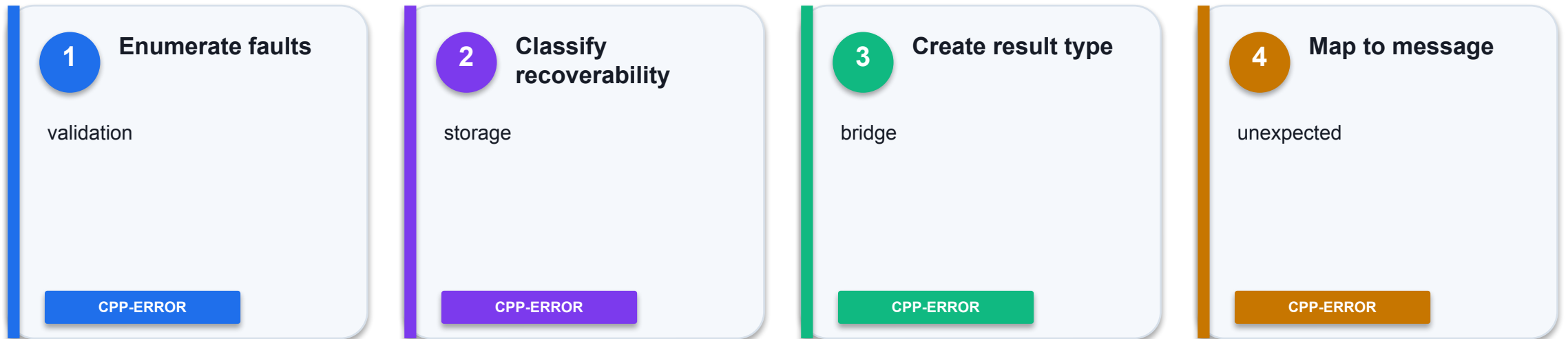
4

contract

EVIDENCE METRIC

Reasons-to-change per component

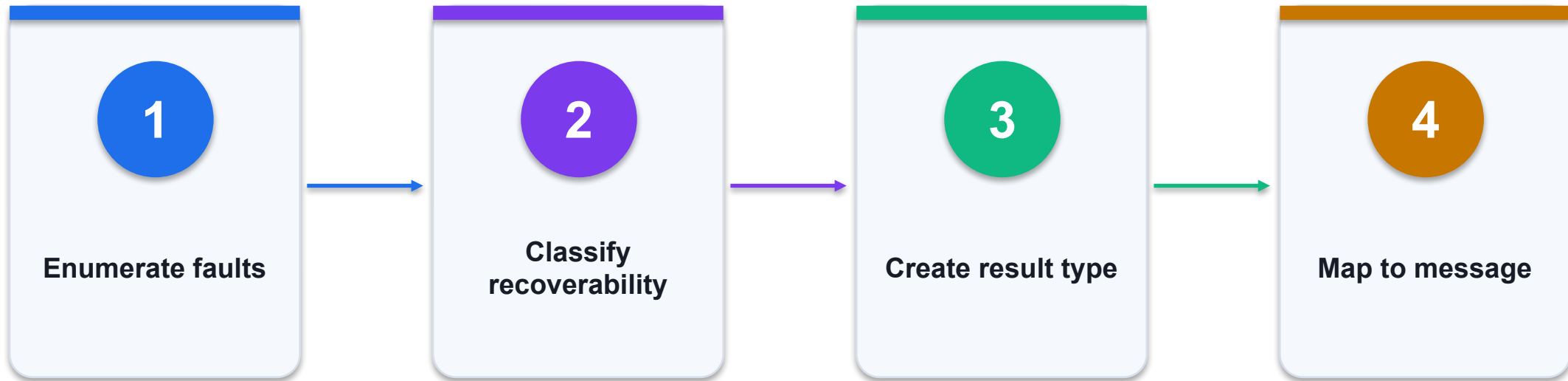
Express failure in the contract



DESIGN JUDGEMENT

Use typed errors in C++; translate them once at the Objective-C++ edge.

Express failure in the contract · Mechanism



MECHANISM

Failure mode: A Boolean failure discards the cause and repair path. **Control:** Use typed errors in C++; translate them once at the Objective-C++ edge.

Express failure in the contract · Technical Anchor

```
using AddResult = std::variant<ExpenseId, DomainError>;
```

1

validation

2

storage

3

bridge

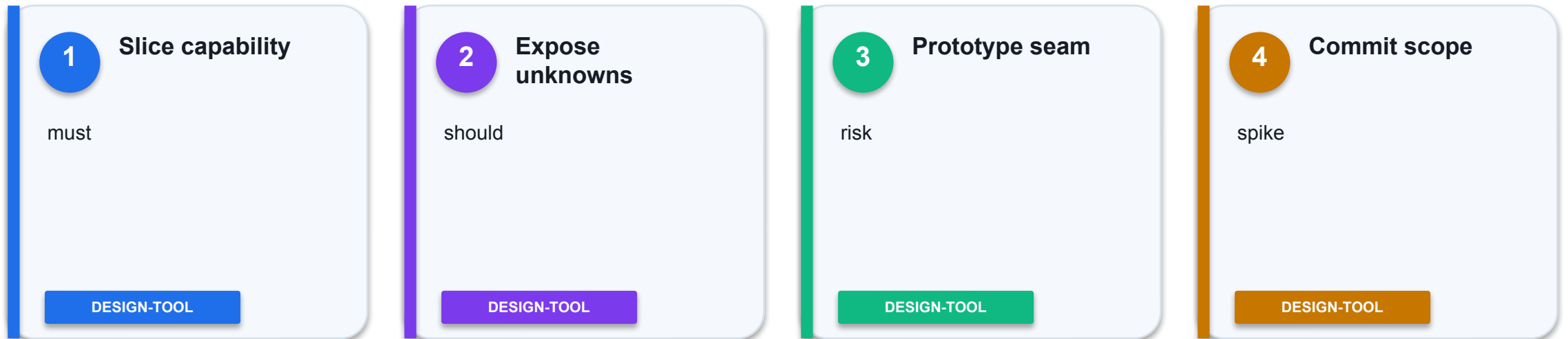
4

unexpected

EVIDENCE METRIC

Failure variants handled explicitly

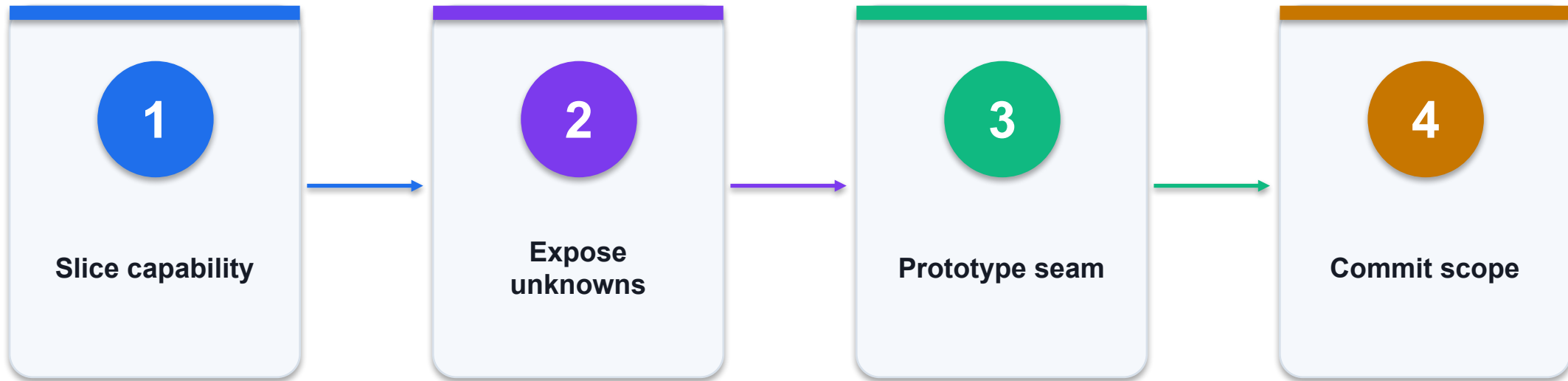
Estimate before implementation



DESIGN JUDGEMENT

Time-box an Objective-C++ seam spike before committing the architecture.

Estimate before implementation · Mechanism



MECHANISM

Failure mode: The hardest bridge uncertainty appears late in the capstone. **Control:** Time-box an Objective-C++ seam spike before committing the architecture.

Estimate before implementation · Technical Anchor

```
// Spike: Swift -> ObjC++ -> C++ -> immutable result
```

1

must

2

should

3

risk

4

spike

EVIDENCE METRIC

Unknowns retired before full build

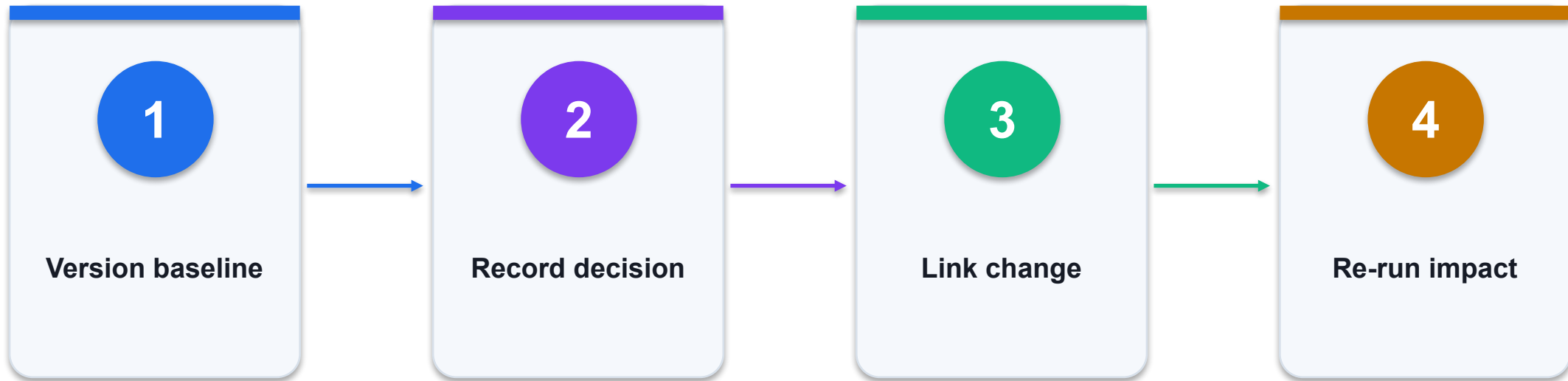
Requirements baseline and change control



DESIGN JUDGEMENT

Treat AI output as a proposal against a versioned baseline.

Requirements baseline and change control · Mechanism



MECHANISM

Failure mode: A model-generated change silently alters an approved requirement. **Control:** Treat AI output as a proposal against a versioned baseline.

Requirements baseline and change control - Technical Anchor

```
struct Change { std::string requirementId; std::string  
rationale; };
```

1

requirement id

2

version

3

rationale

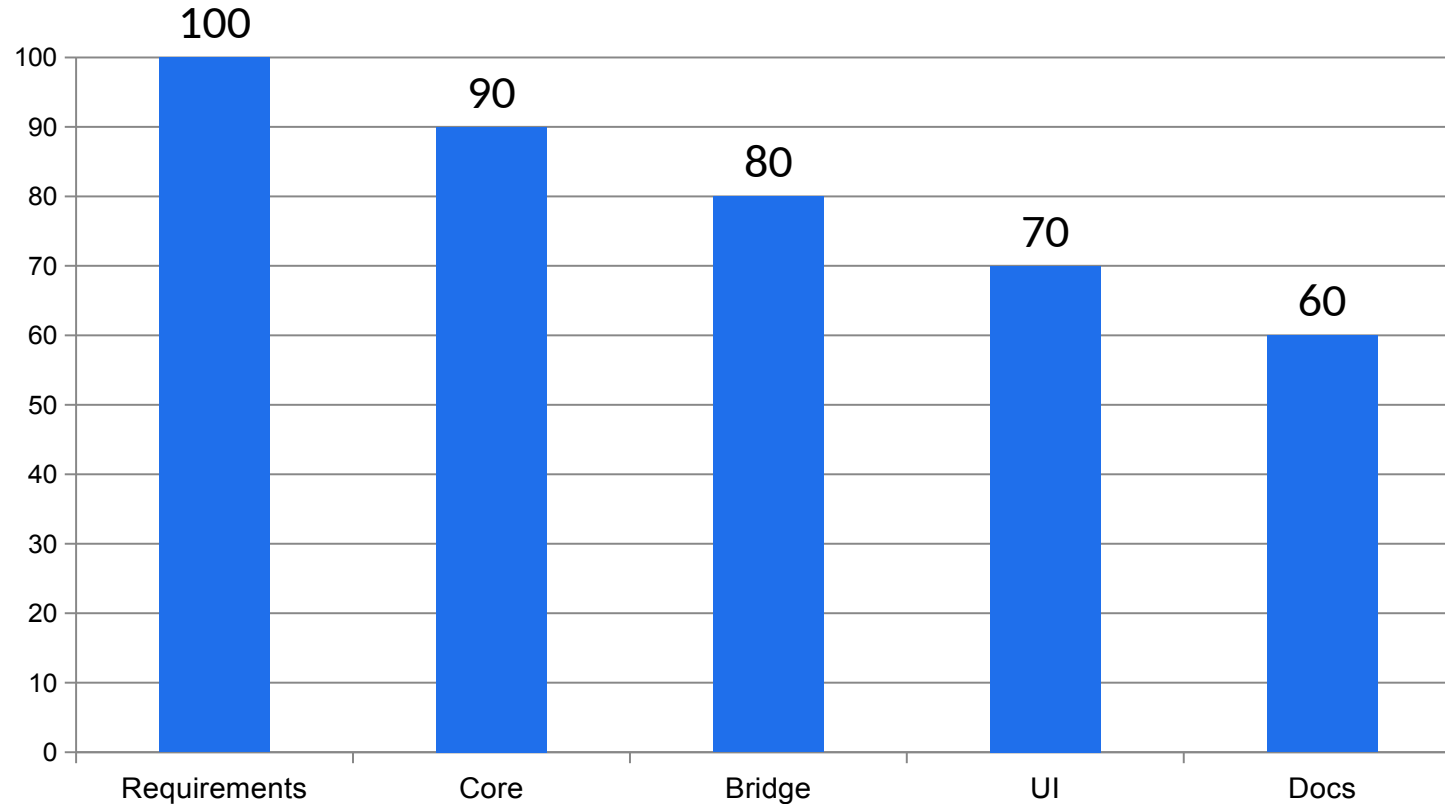
4

affected tests

EVIDENCE METRIC

Changes with explicit downstream impact

Topic 1 · Evidence Coverage by Layer



WHAT THE DATA SHOWS

Coverage should form a connected chain. A polished UI cannot compensate for untested invariants or an undocumented seam.

Editable chart · values are instructional thresholds, not production benchmarks

Activity 1: Requirements and Component Boundary

CHALLENGE

Convert stakeholder needs into testable requirements and design the SwiftUI–Objective-C++–C++ boundary.

REGULATED MAPPING

K1, A1

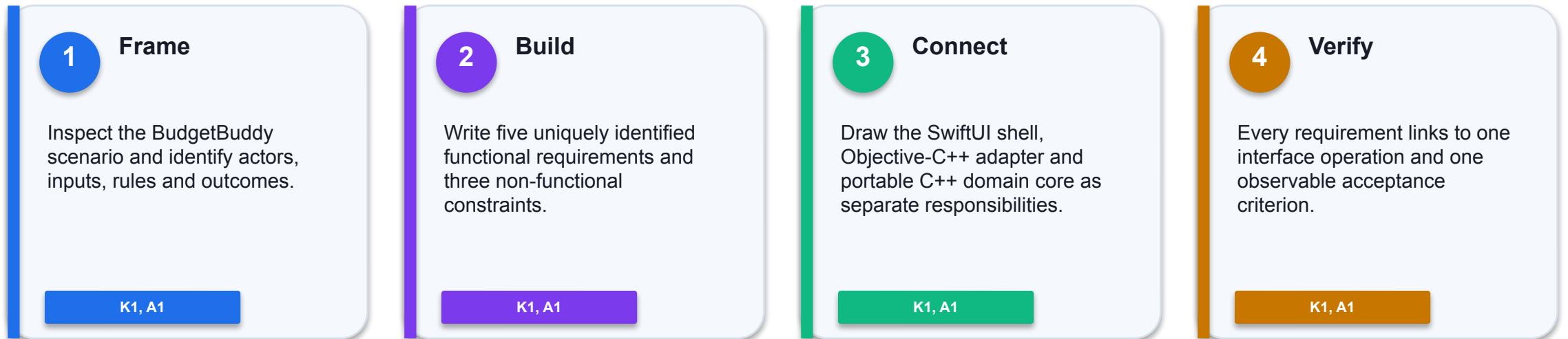
YOU WILL PRODUCE

Requirement matrix, boundary diagram and acceptance criteria

Markdown, diagrams.net or PowerPoint, JSON
fixture

Detailed procedures: [Learner Guide and activities/01-requirements-component-boundary/README.md](#)

Activity 1 · Delivery Workflow



DESIGN JUDGEMENT

The slide communicates the workflow; the Learner Guide carries the detailed procedure and commands.

Activity 1 · Vibe Coding Generation Prompt

GENERATION PROMPT

BudgetBuddy requirements only. Generate REQ-001..REQ-005 with actor, input, integer-cent invariant, observable result, negative case and evidence. Then create the inward dependency boundary from SwiftUI through BudgetViewModel, BudgetEvaluating, BudgetBridge.h/.mm and portable C++. Output each exact path plus the complete file. Do not generate UI code. Verify all IDs, cent units and boundary layers with rg.

PROTECTED FILE SCOPE

requirements-working.txt; boundary-working.txt; acceptance-review.txt

EXPECTED GREEN

``rg -n 'REQ-00[1-5]|SwiftUI|BudgetBridge|C|\\+|cents' requirements-working.txt boundary-working.txt`` shows all five requirements and each boundary layer.

REVIEW / REPAIR

Use the paired review prompt in the Activity guide. Fix the first causal failure, preserve tests, then rerun focused and full checks.

Activity 1 · Evidence Contract

1 Artifact

Requirement matrix, boundary diagram and acceptance criteria

K1, A1

2 Acceptance

Every requirement links to one interface operation and one observable acceptance criterion.

K1, A1

3 Failure signal

If the boundary follows screen widgets, restate each operation as a business verb before redrawing.

K1, A1

4 Cleanup

Retain the versioned design artifacts; remove only disposable sketches.

K1, A1

DESIGN JUDGEMENT

Evidence is a retrievable artifact or test result, not a statement that the activity was completed.

Activity 1 · Verification Gate

PASS WHEN

Every requirement links to one interface operation and one observable acceptance criterion.

IF IT FAILS

If the boundary follows screen widgets, restate each operation as a business verb before redrawing.

LEAVE THE WORKSPACE

Retain the versioned design artifacts; remove only disposable sketches.

Activity 2: Portable C++ Domain Core

CHALLENGE

Implement exact-cent value objects and a small budget policy as a standalone CMake library.

REGULATED MAPPING

K1, A1

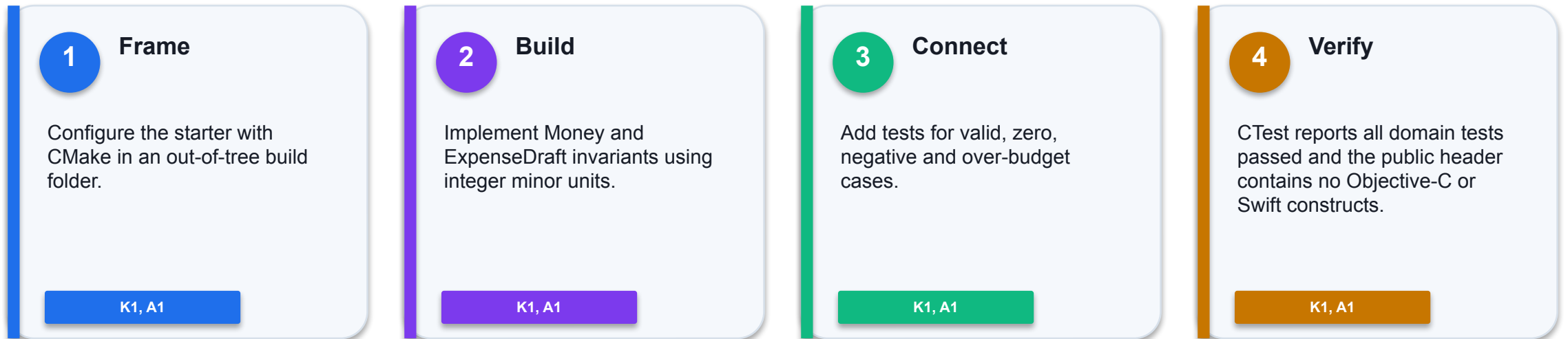
YOU WILL PRODUCE

Compiling C++20 library and domain tests

C++20, CMake, CTest

Detailed procedures: [Learner Guide and activities/02-portable-cpp-domain-core/README.md](#)

Activity 2 · Delivery Workflow



DESIGN JUDGEMENT

The slide communicates the workflow; the Learner Guide carries the detailed procedure and commands.

Activity 2 · Vibe Coding Generation Prompt

GENERATION PROMPT

Generate complete C++20 Money, ExpenseDraft, Decision, DomainError and BudgetPolicy files plus CMake and tests. Preserve `std::int64_t` cents and the approved evaluate signature. Reject invalid, overflow and overspend cases without Apple imports. Output exact path then complete code, with no TODOs. Verify a clean `configure/build/CTest` run and an empty Apple-framework scan.

PROTECTED FILE SCOPE

```
workspace/CMakeLists.txt;  
workspace/include/BudgetPolicy.hpp;  
workspace/src/BudgetPolicy.cpp;  
workspace/tests/BudgetPolicyTests.cpp
```

EXPECTED GREEN

```
`cmake -S workspace -B workspace/build &&  
cmake --build workspace/build && ctest --test-  
dir workspace/build --output-on-failure`; all  
tests pass and Apple import scan is empty.
```

REVIEW / REPAIR

Use the paired review prompt in the Activity guide. Fix the first causal failure, preserve tests, then rerun focused and full checks.

Activity 2 · Evidence Contract

1 Artifact

Compiling C++20 library and domain tests

K1, A1

2 Acceptance

CTest reports all domain tests passed and the public header contains no Objective-C or Swift constructs.

K1, A1

3 Failure signal

Delete only the local build directory and reconfigure; inspect the first compiler diagnostic, not the last.

K1, A1

4 Cleanup

Remove build/; keep source, headers and tests.

K1, A1

DESIGN JUDGEMENT

Evidence is a retrievable artifact or test result, not a statement that the activity was completed.

Activity 2 · Verification Gate

PASS WHEN

CTest reports all domain tests passed and the public header contains no Objective-C or Swift constructs.

IF IT FAILS

Delete only the local build directory and reconfigure; inspect the first compiler diagnostic, not the last.

LEAVE THE WORKSPACE

Remove build/; keep source, headers and tests.

Topic 1 · Review

1 Explain

LO1: Determine basic software components using C++ methodologies to meet functional specifications.

LO1 · K1 · A1

2 Demonstrate

Show the Activity evidence that operationalises the outcome.

LO1 · K1 · A1

3 Diagnose

Name the failure signal and the layer where it originates.

LO1 · K1 · A1

4 Trace

Connect LO1 · K1 · A1 to the assessment and evidence pack.

LO1 · K1 · A1

DESIGN JUDGEMENT

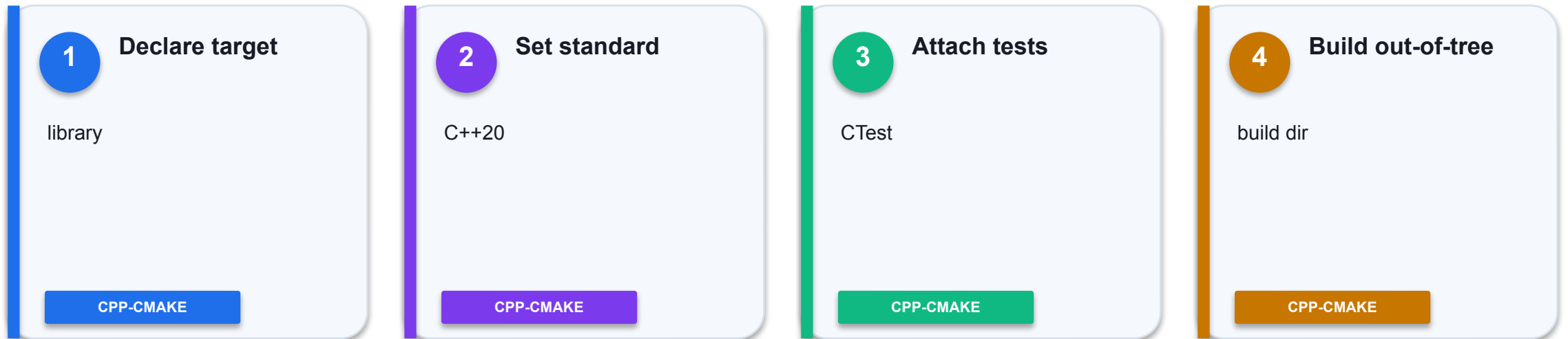
Do not advance on confidence alone; point to the artifact or test result that supports the answer.

TOPIC 2 · LO2 · K2 · A2

Vibe-Generated Portable Logic and Quality Gates

LO2: Apply C++ methodologies and tools for software creation.

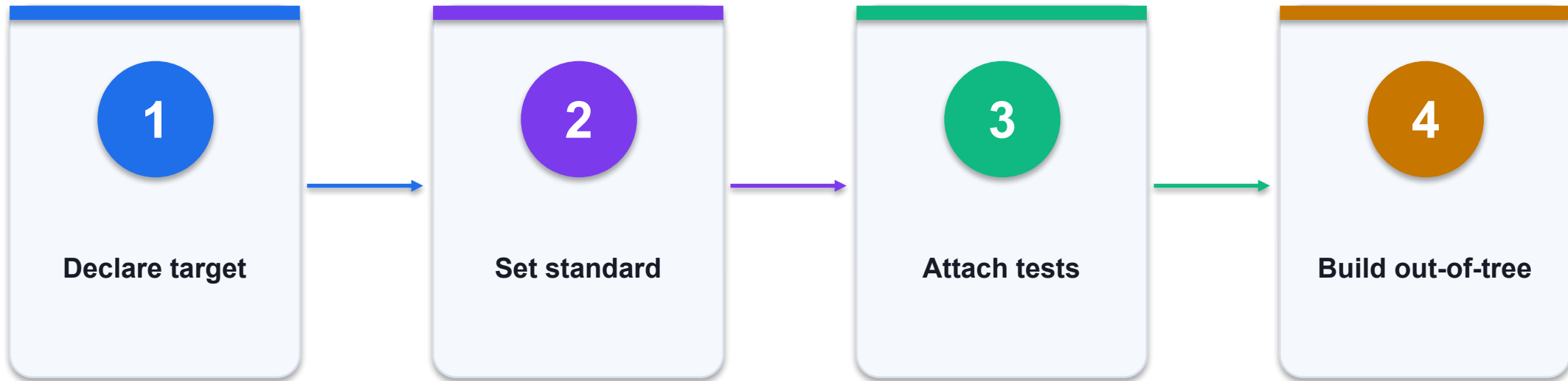
CMake defines a portable build graph



DESIGN JUDGEMENT

Make the core buildable with CMake and add it to Xcode as an explicit dependency.

CMake defines a portable build graph · Mechanism



MECHANISM

Failure mode: Xcode-only configuration hides a broken standalone core. **Control:** Make the core buildable with CMake and add it to Xcode as an explicit dependency.

CMake defines a portable build graph · Technical Anchor

```
add_library(DomainCore STATIC src/Budget.cpp)
```

1

library

2

C++20

3

CTest

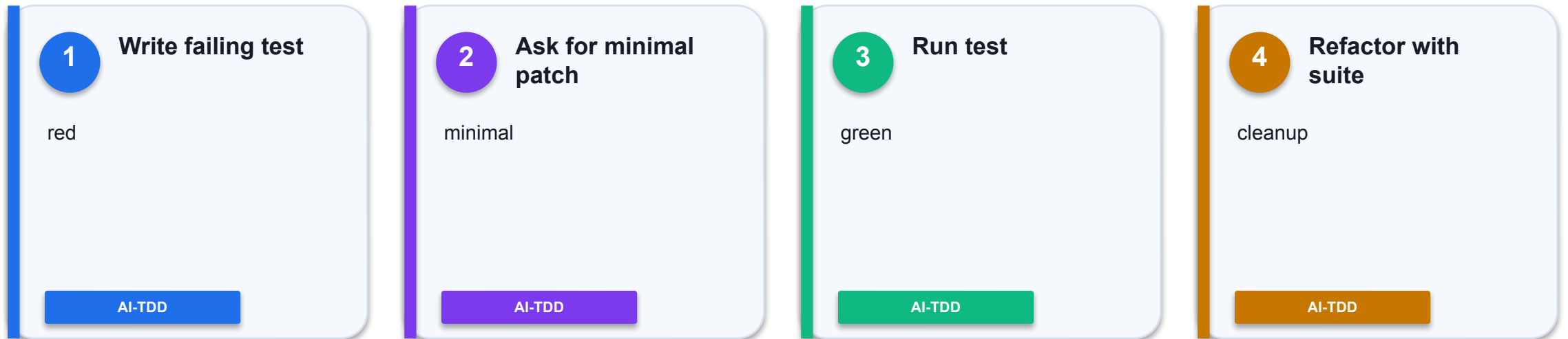
4

build dir

EVIDENCE METRIC

Reproducible build from clean checkout

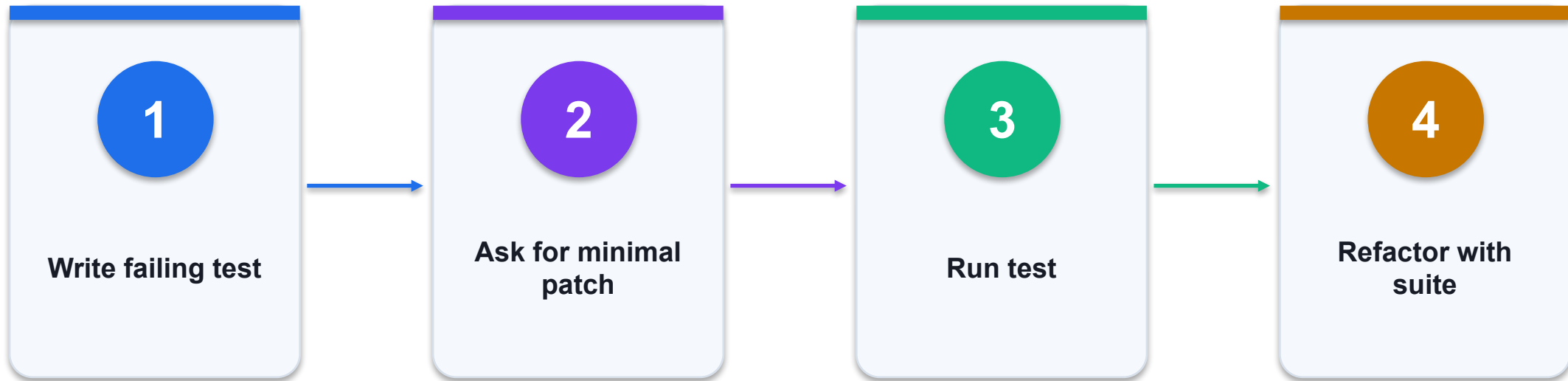
Use Red-Green-Refactor with AI



DESIGN JUDGEMENT

Freeze expected behaviour first and demand the smallest causal patch.

Use Red-Green-Refactor with AI · Mechanism



MECHANISM

Failure mode: The assistant writes implementation and tests that merely echo it. **Control:** Freeze expected behaviour first and demand the smallest causal patch.

Use Red-Green-Refactor with AI · Technical Anchor

```
TEST(BudgetPolicy, RejectsNegativeLimit)
{ EXPECT_THROW(...); }
```

1

red

2

minimal

3

green

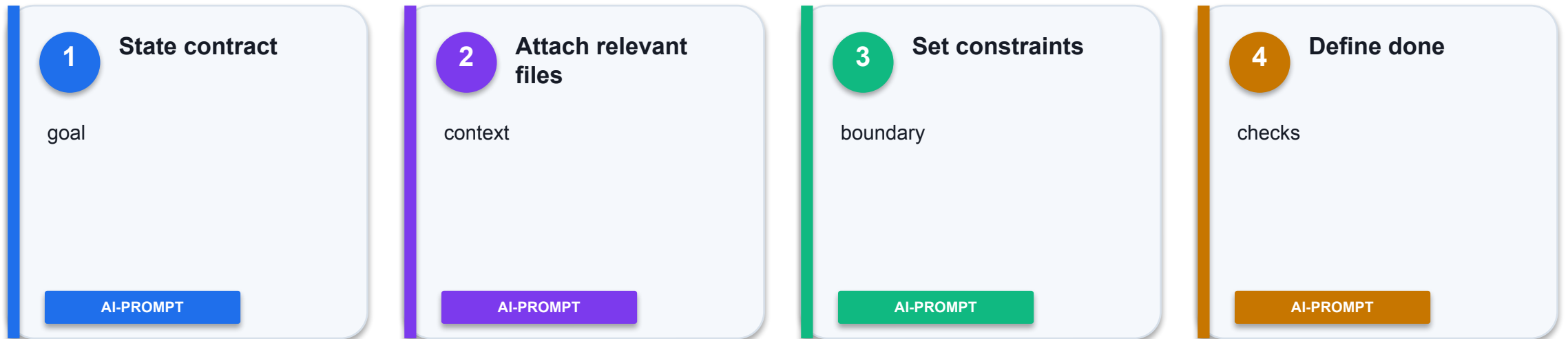
4

cleanup

EVIDENCE METRIC

Every production change preceded by a meaningful failure

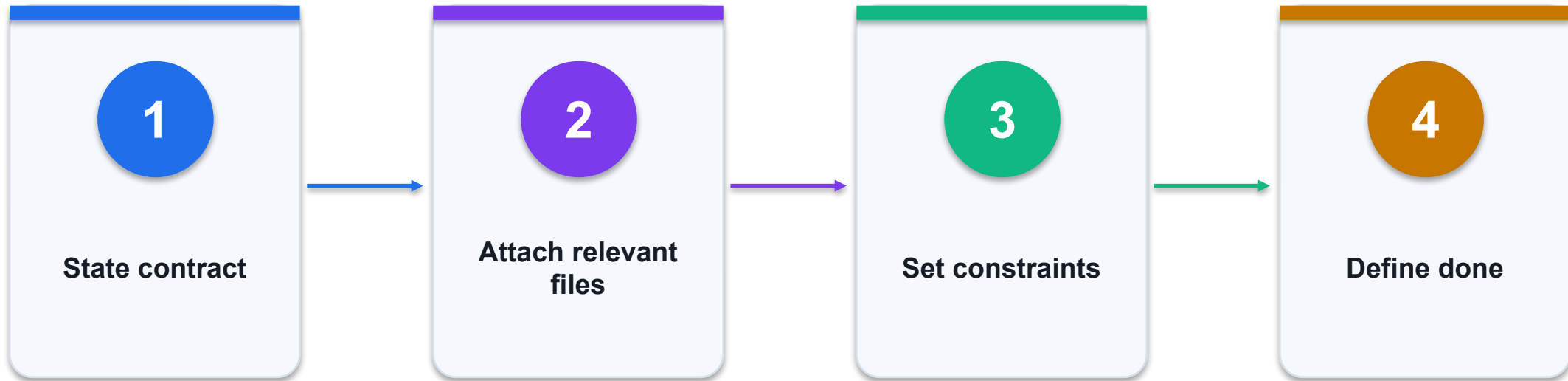
Prompt context is an engineering input



DESIGN JUDGEMENT

Name protected interfaces and exact verification commands.

Prompt context is an engineering input · Mechanism



MECHANISM

Failure mode: A broad prompt invites API churn and unrelated edits. **Control:** Name protected interfaces and exact verification commands.

Prompt context is an engineering input · Technical Anchor

```
Goal: implement evaluate(). Preserve BudgetPolicy.hpp.  
Run ctest.
```

1

goal

2

context

3

boundary

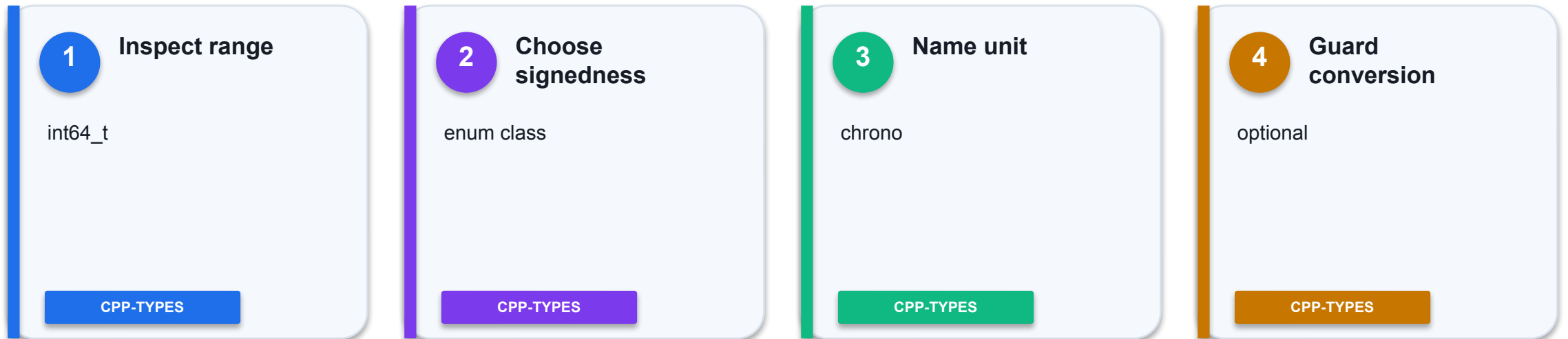
4

checks

EVIDENCE METRIC

Acceptance checks passed on first review

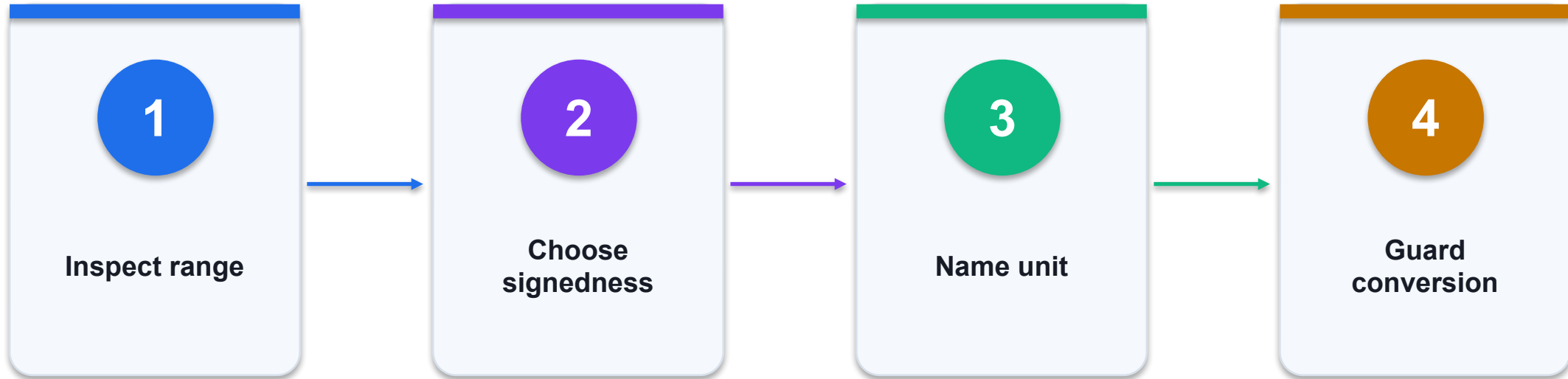
Select types to encode invariants



DESIGN JUDGEMENT

Prefer domain types, scoped enumerations and explicit optional values.

Select types to encode invariants · Mechanism



MECHANISM

Failure mode: Primitive strings and doubles hide units and optionality. **Control:** Prefer domain types, scoped enumerations and explicit optional values.

Select types to encode invariants · Technical Anchor

```
enum class ExpenseKind : std::uint8_t { food, travel,  
other };
```

1

int64_t

2

enum class

3

chrono

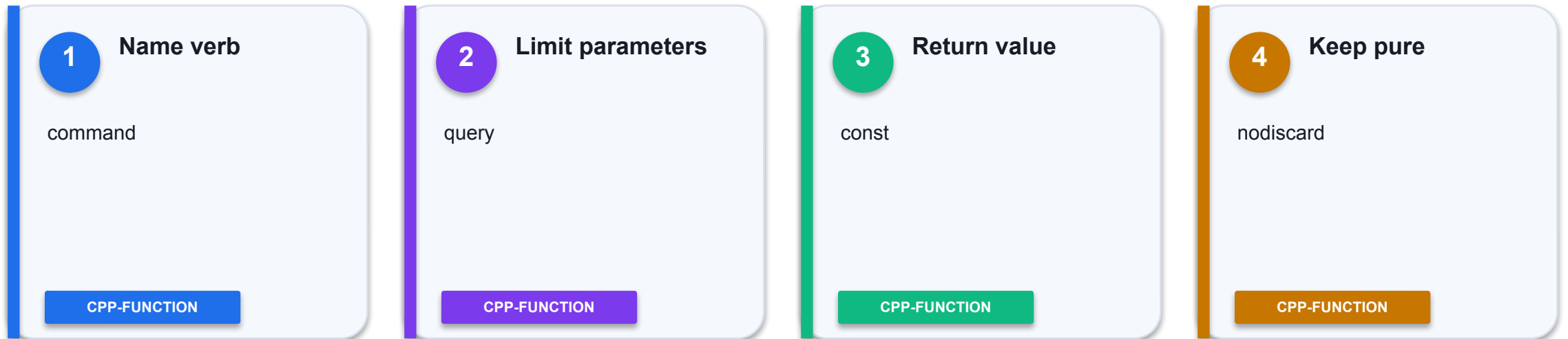
4

optional

EVIDENCE METRIC

Invalid conversions detected at boundaries

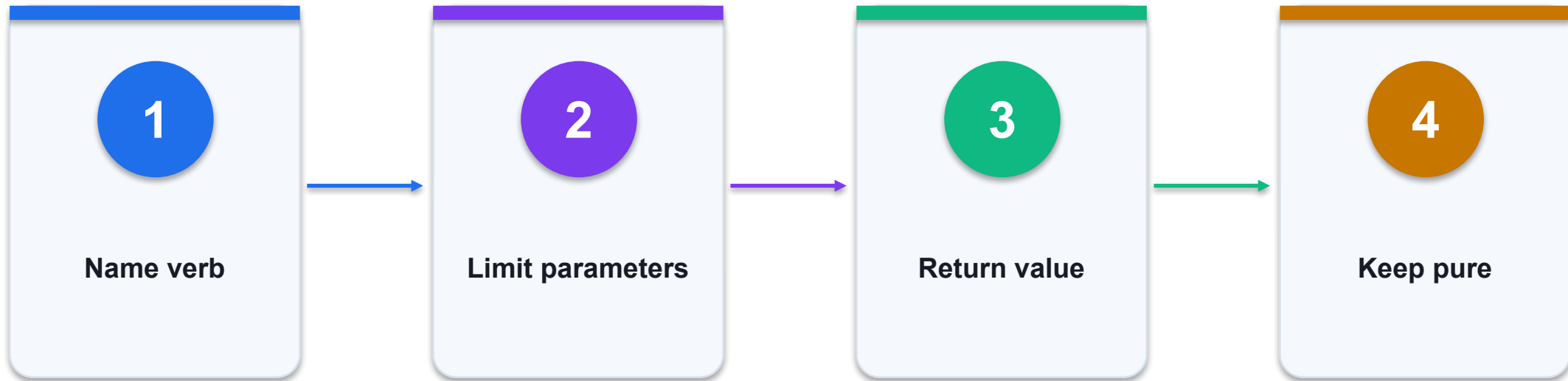
Functions expose intention



DESIGN JUDGEMENT

Return data or typed errors; inject side effects through interfaces.

Functions expose intention · Mechanism



MECHANISM

Failure mode: A function mutates globals and communicates failure by logging. **Control:** Return data or typed errors; inject side effects through interfaces.

Functions expose intention · Technical Anchor

```
[[nodiscard]] Money remaining(Money limit, Money  
spent);
```

1 command

2 query

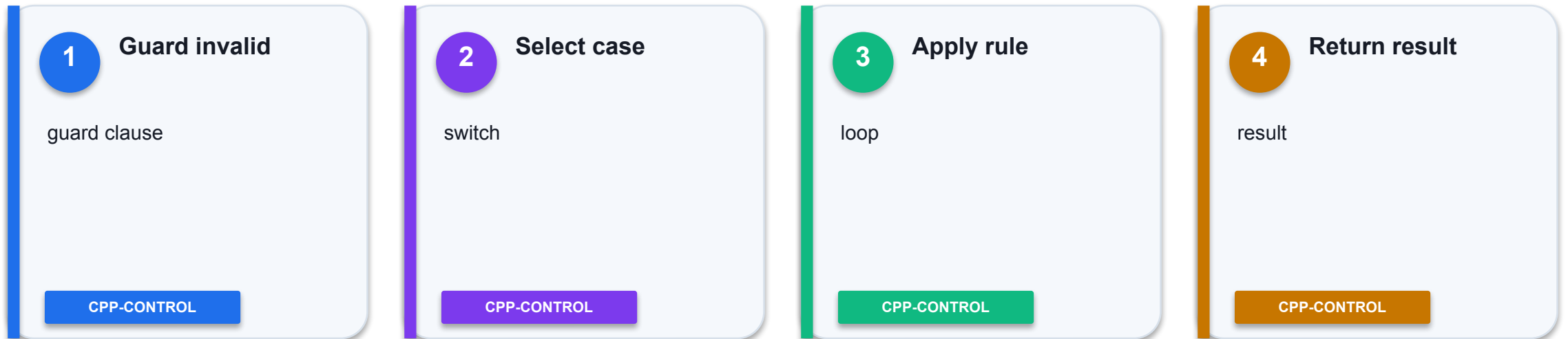
3 const

4 nodiscard

EVIDENCE METRIC

Pure functions covered by deterministic tests

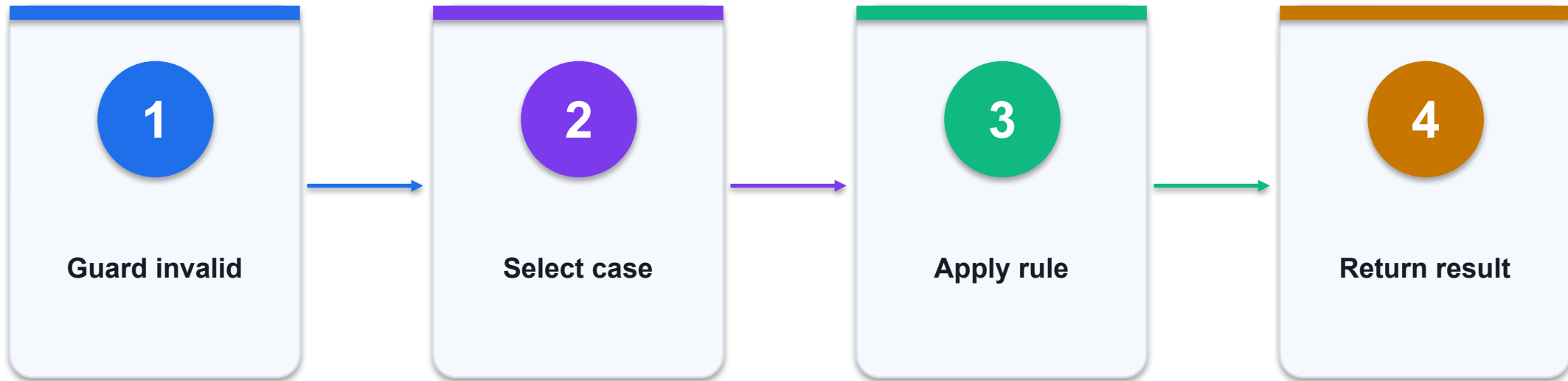
Control flow should reveal policy



DESIGN JUDGEMENT

Use guards and small policy functions; reserve loops for collections.

Control flow should reveal policy · Mechanism



MECHANISM

Failure mode: Deep nesting obscures which condition owns a decision. **Control:** Use guards and small policy functions; reserve loops for collections.

Control flow should reveal policy · Technical Anchor

```
if (draft.amount.cents <= 0) return  
DomainError::invalidAmount;
```

1

guard clause

2

switch

3

loop

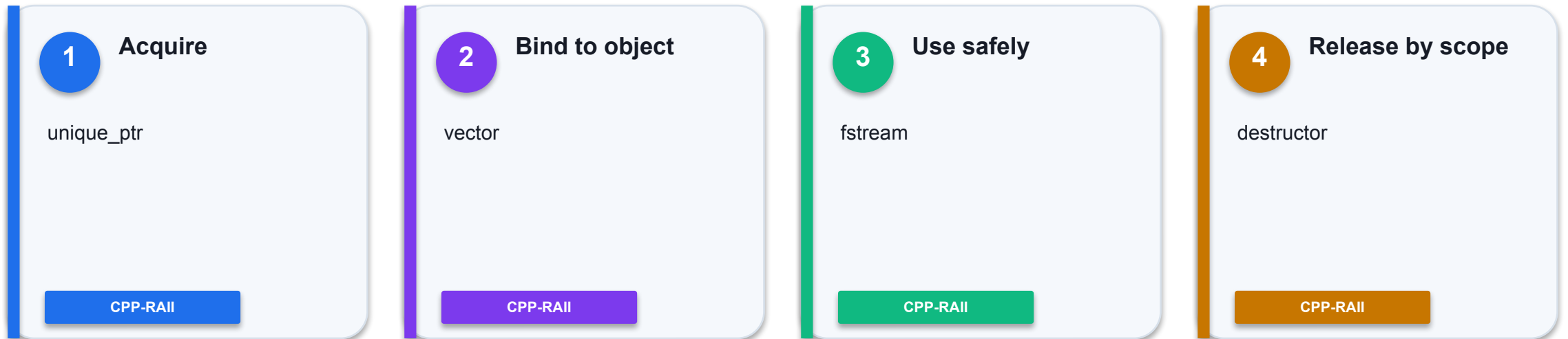
4

result

EVIDENCE METRIC

Cyclomatic paths each have a named test

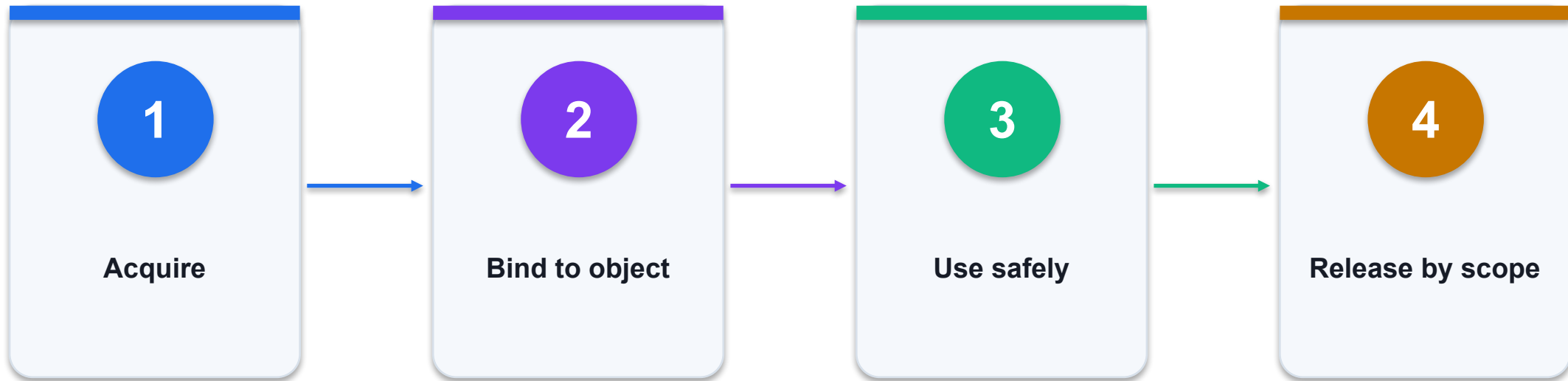
RAII makes ownership visible



DESIGN JUDGEMENT

Keep RAII inside C++; cross the boundary using copied value snapshots.

RAII makes ownership visible · Mechanism



MECHANISM

Failure mode: Raw ownership crosses the bridge and leaks on an error path. **Control:** Keep RAII inside C++; cross the boundary using copied value snapshots.

RAII makes ownership visible · Technical Anchor

```
auto repo =  
std::make_unique<JsonExpenseRepository>(path);
```

1

unique_ptr

2

vector

3

fstream

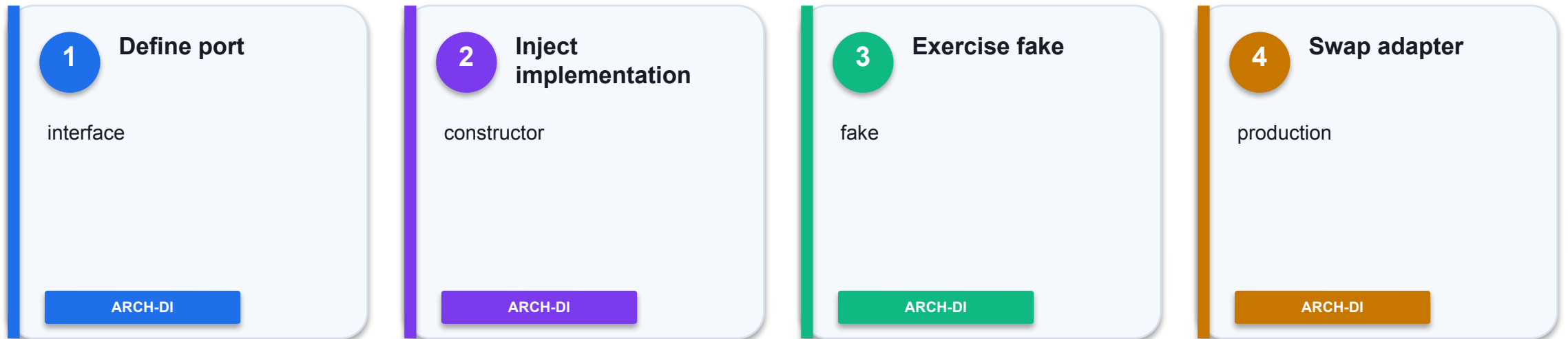
4

destructor

EVIDENCE METRIC

Zero manual delete paths

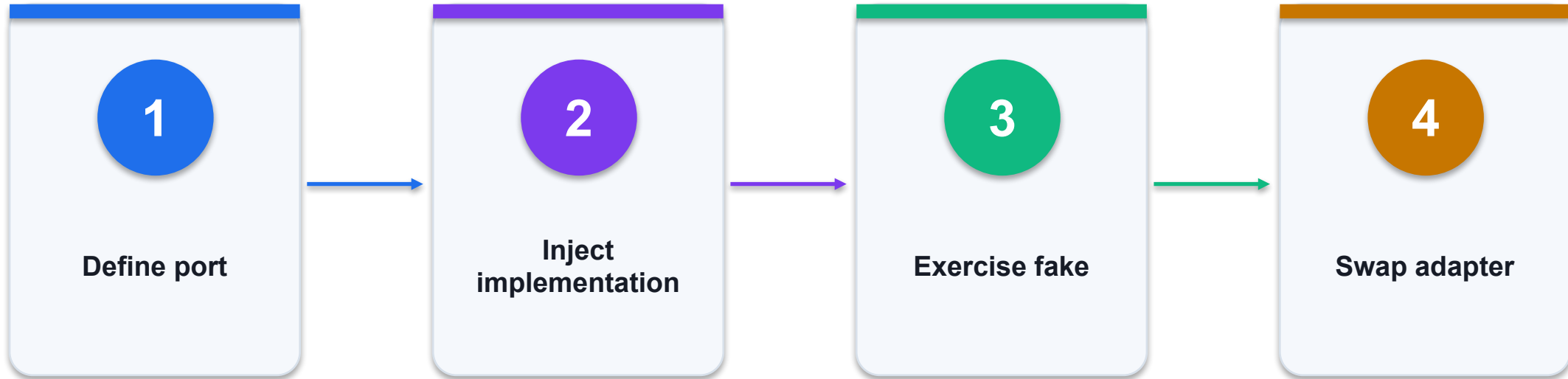
Dependency injection enables substitution



DESIGN JUDGEMENT

Inject repositories and clocks through narrow abstract ports.

Dependency injection enables substitution · Mechanism



MECHANISM

Failure mode: A singleton couples rules to persistence and time. **Control:** Inject repositories and clocks through narrow abstract ports.

Dependency injection enables substitution · Technical Anchor

```
ExpenseService (ExpenseRepository& repo, Clock& clock);
```

1

interface

2

constructor

3

fake

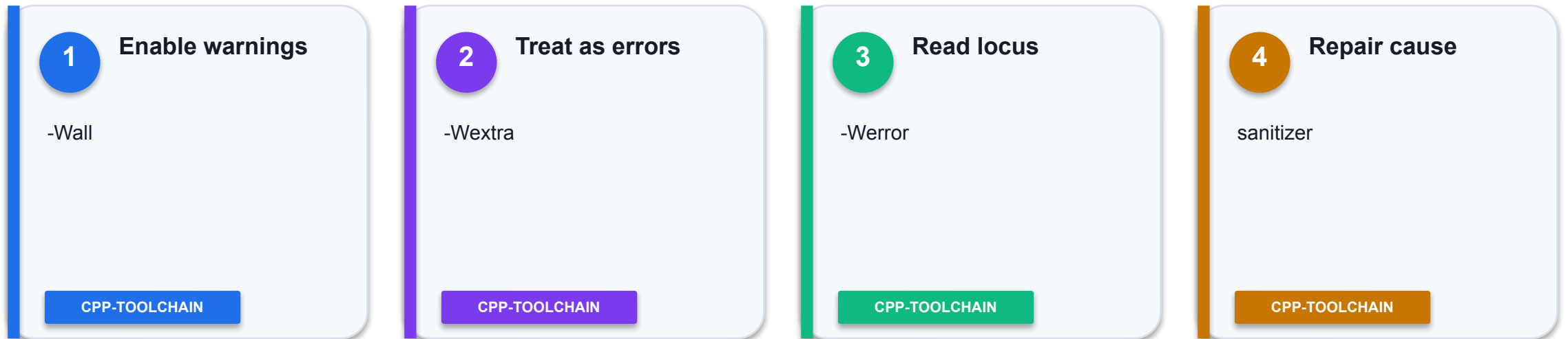
4

production

EVIDENCE METRIC

Core tests run without disk or iOS runtime

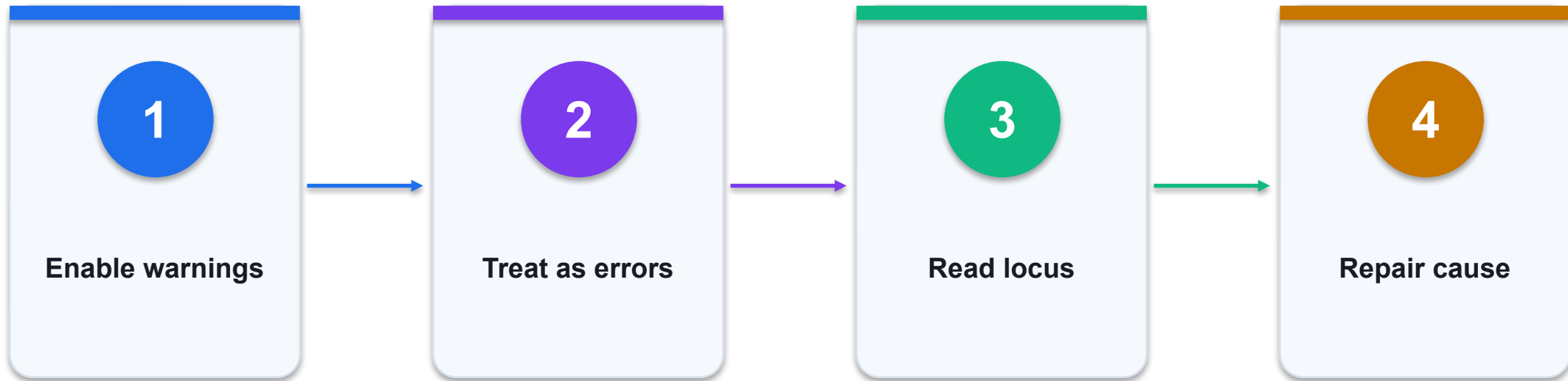
Compiler diagnostics are design feedback



DESIGN JUDGEMENT

Use strict warnings and sanitizers during development; document platform differences.

Compiler diagnostics are design feedback · Mechanism



MECHANISM

Failure mode: Generated code compiles only under permissive defaults. **Control:** Use strict warnings and sanitizers during development; document platform differences.

Compiler diagnostics are design feedback · Technical Anchor

```
target_compile_options(DomainCore PRIVATE -Wall -Wextra  
-Werror)
```

1

-Wall

2

-Wextra

3

-Werror

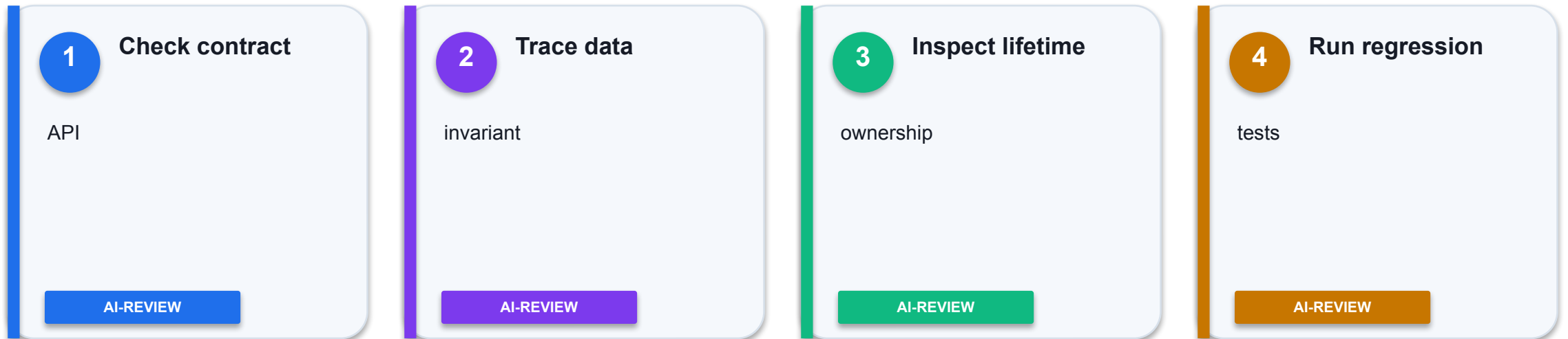
4

sanitizer

EVIDENCE METRIC

Zero warnings in clean build

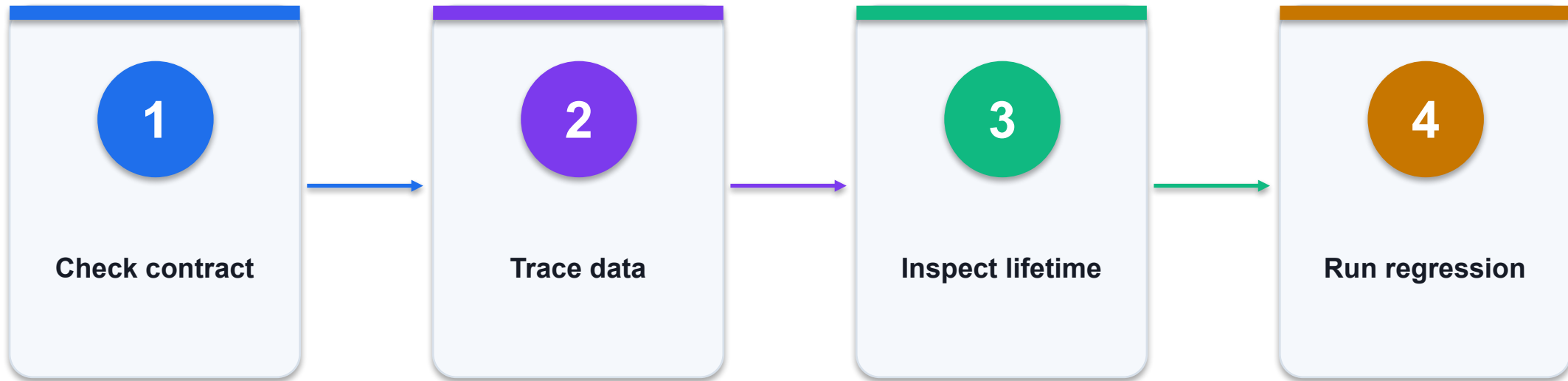
Review AI diffs semantically



DESIGN JUDGEMENT

Review data representation and lifetimes before style.

Review AI diffs semantically · Mechanism



MECHANISM

Failure mode: A plausible patch introduces silent truncation or ownership ambiguity. **Control:** Review data representation and lifetimes before style.

Review AI diffs semantically - Technical Anchor

```
// Review: conversion from Swift Int to int32_t may overflow.
```

1

API

2

invariant

3

ownership

4

tests

EVIDENCE METRIC

Review findings resolved before merge

AI Code Review · Evidence before Acceptance



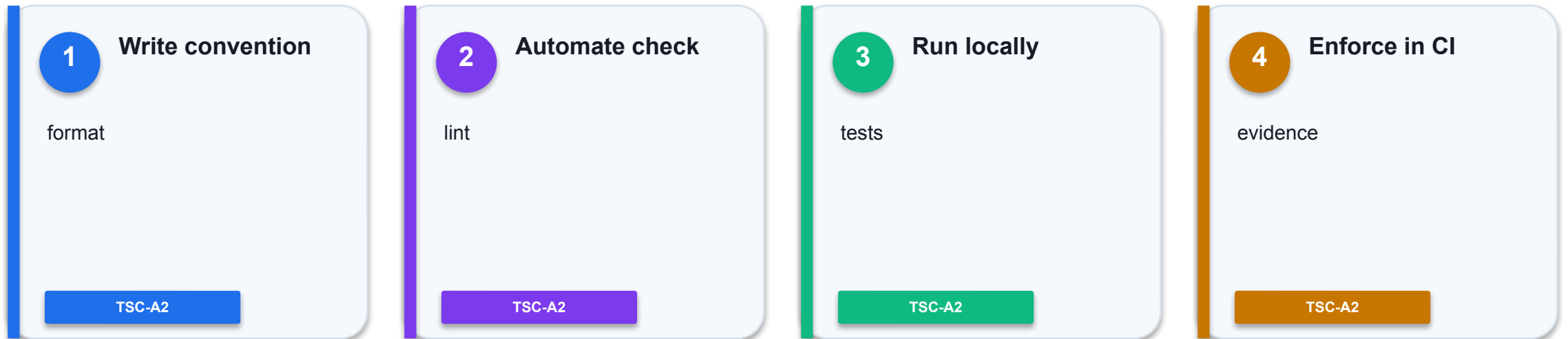
1 Locate the claim in code.

2 Reproduce material findings.

3 Classify severity and owner.

4 Run focused and regression checks.

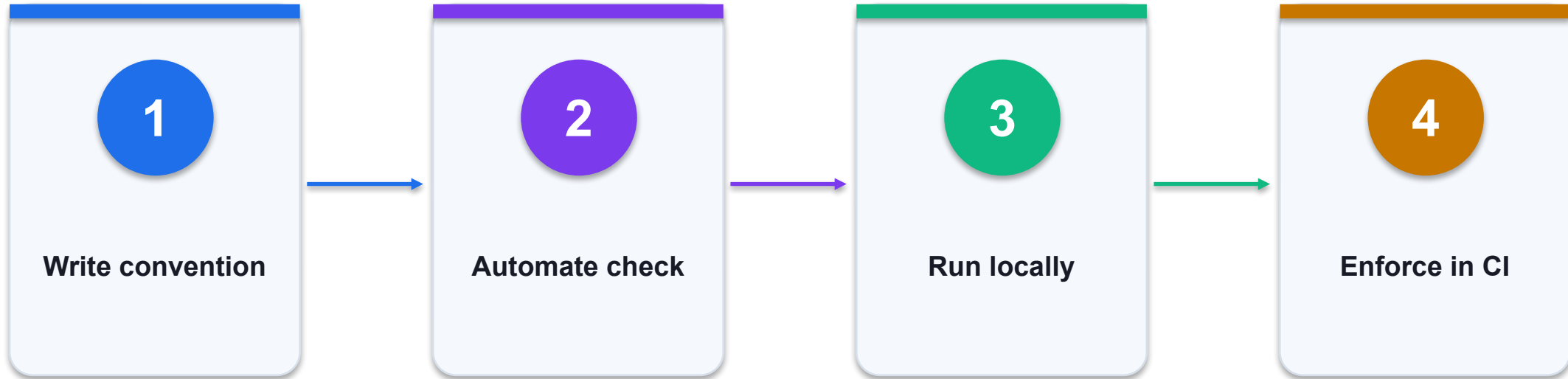
Organisational practice is executable



DESIGN JUDGEMENT

Translate practices into formatter, compiler and test configuration.

Organisational practice is executable · Mechanism



MECHANISM

Failure mode: A design principle remains prose and is ignored under time pressure. **Control:** Translate practices into formatter, compiler and test configuration.

Organisational practice is executable · Technical Anchor

```
cmake --build build && ctest --test-dir build --output-on-failure
```

1

format

2

lint

3

tests

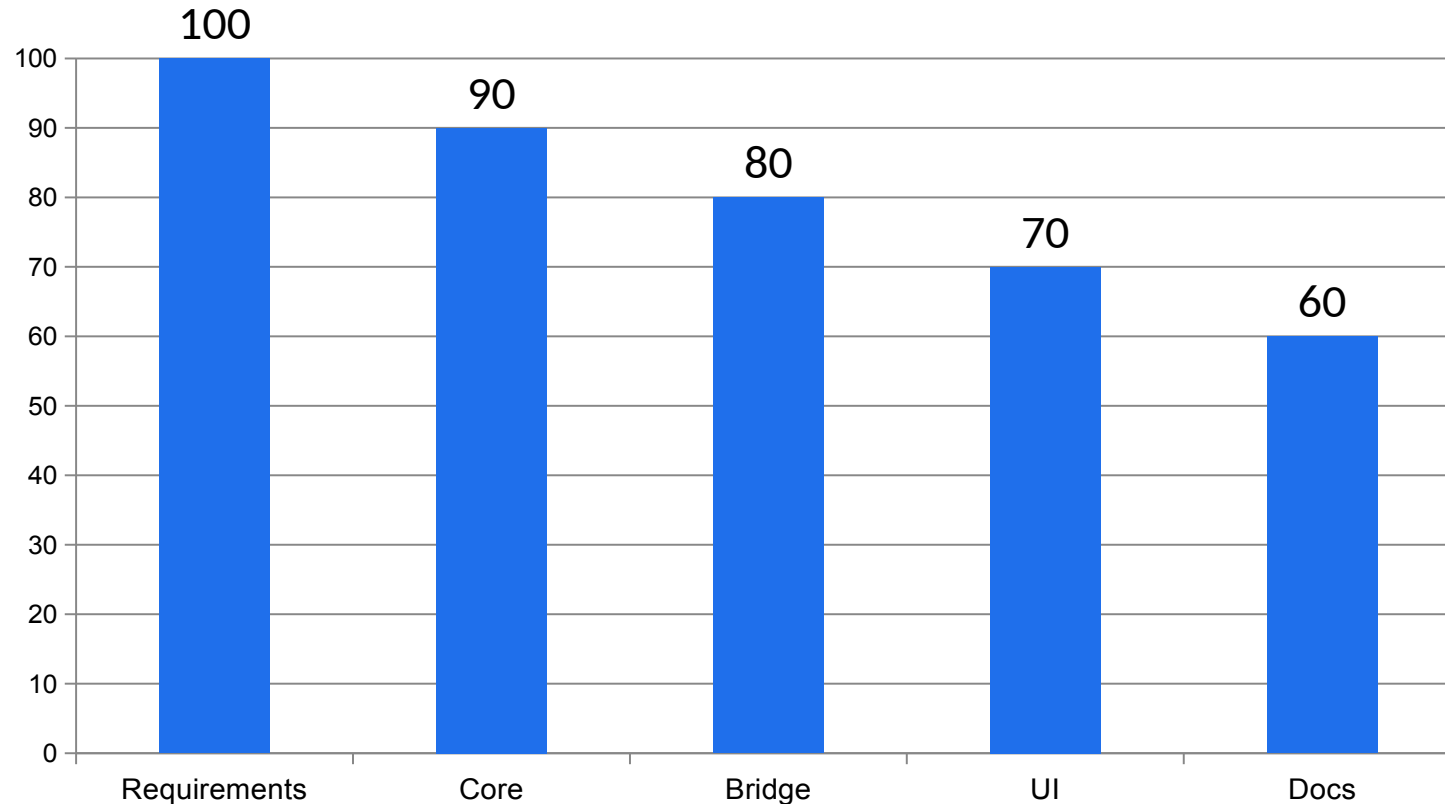
4

evidence

EVIDENCE METRIC

Same checks locally and in CI

Topic 2 · Evidence Coverage by Layer



WHAT THE DATA SHOWS

Coverage should form a connected chain. A polished UI cannot compensate for untested invariants or an undocumented seam.

Editable chart · values are instructional thresholds, not production benchmarks

Activity 3: AI-Assisted TDD for C++

CHALLENGE

Use an evidence-bound AI prompt to implement a rule through Red-Green-Refactor.

REGULATED MAPPING

K2, A2

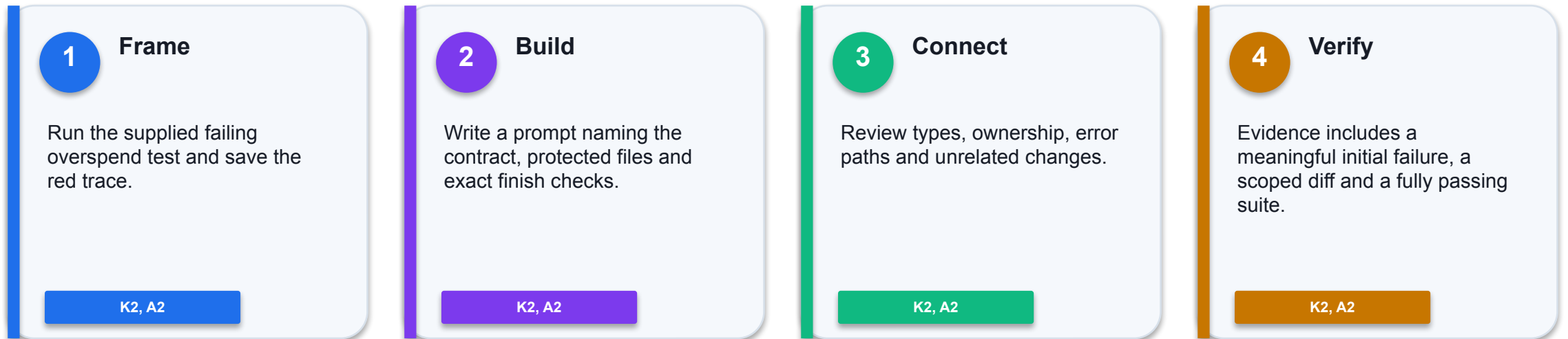
YOU WILL PRODUCE

Prompt contract, failing trace, minimal patch and passing trace

AI coding assistant, CTest, diff review

Detailed procedures: [Learner Guide and activities/03-ai-assisted-tdd/README.md](#)

Activity 3 · Delivery Workflow



DESIGN JUDGEMENT

The slide communicates the workflow; the Learner Guide carries the detailed procedure and commands.

Activity 3 · Vibe Coding Generation Prompt

GENERATION PROMPT

Use the recorded overspend RED trace and current BudgetPolicy files. Produce the smallest causal patch that makes the missing rule pass. Preserve the public signature, integer cents, existing tests and unrelated formatting. Return the complete changed file, focused command and full regression command. Do not weaken the failing assertion. Retain red trace, scoped diff and GREEN trace.

PROTECTED FILE SCOPE

workspace/prompt-contract.txt;
workspace/BudgetPolicy.hpp;
workspace/BudgetPolicy.cpp; workspace/failing-test.cpp;
workspace/red-trace.txt; workspace/green-trace.txt

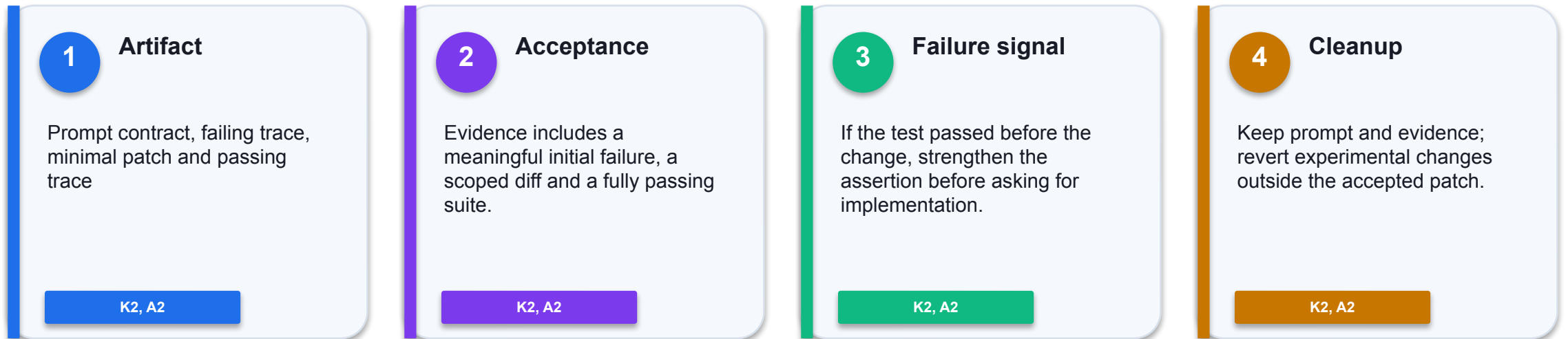
EXPECTED GREEN

The initial test fails for the missing overspend rule; after the minimal patch the focused binary and full domain suite exit 0.

REVIEW / REPAIR

Use the paired review prompt in the Activity guide. Fix the first causal failure, preserve tests, then rerun focused and full checks.

Activity 3 · Evidence Contract



DESIGN JUDGEMENT

Evidence is a retrievable artifact or test result, not a statement that the activity was completed.

Activity 3 · Verification Gate

PASS WHEN

Evidence includes a meaningful initial failure, a scoped diff and a fully passing suite.

IF IT FAILS

If the test passed before the change, strengthen the assertion before asking for implementation.

LEAVE THE WORKSPACE

Keep prompt and evidence; revert experimental changes outside the accepted patch.

Activity 4: Toolchain and Quality Gates

CHALLENGE

Turn organisational design practices into compiler, formatter and test gates.

REGULATED MAPPING

K2, A2

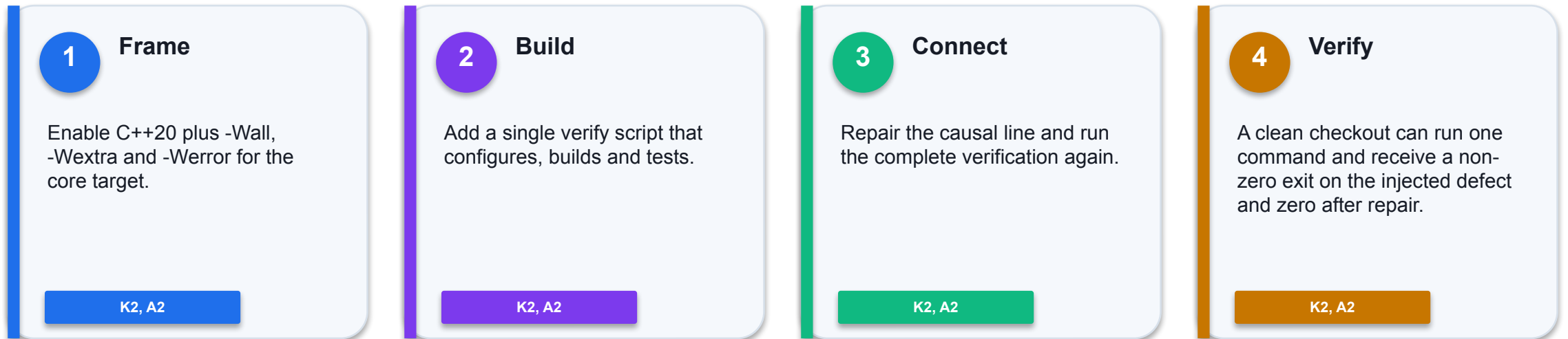
YOU WILL PRODUCE

Warning-clean build, quality script and evidence log

CMake, compiler warnings, sanitizers, shell

Detailed procedures: [Learner Guide and activities/04-toolchain-quality-gates/README.md](#)

Activity 4 · Delivery Workflow



DESIGN JUDGEMENT

The slide communicates the workflow; the Learner Guide carries the detailed procedure and commands.

Activity 4 · Vibe Coding Generation Prompt

GENERATION PROMPT

Generate complete CMake and verify.sh files for C++20, guarded strict warnings, optional AddressSanitizer and CTest. Demonstrate RED with one controlled warning, then repair its causal line and demonstrate GREEN. Do not ignore command failures or use an in-source build. Output complete files and the expected non-zero then zero observations.

PROTECTED FILE SCOPE

`workspace/CMakeLists.txt;`
`workspace/verify.sh; workspace/quality-`
`gates.txt; workspace/evidence-log.txt`

EXPECTED GREEN

``sh workspace/verify.sh`` fails on the injected warning and exits 0 after repair with all CTest cases passing.

REVIEW / REPAIR

Use the paired review prompt in the Activity guide. Fix the first causal failure, preserve tests, then rerun focused and full checks.

Activity 4 · Evidence Contract

1 Artifact

Warning-clean build, quality script and evidence log

K2, A2

2 Acceptance

A clean checkout can run one command and receive a non-zero exit on the injected defect and zero after repair.

K2, A2

3 Failure signal

If Apple Clang rejects a flag, gate it by compiler ID and document the equivalent.

K2, A2

4 Cleanup

Remove generated build and sanitizer output, not the verification script.

K2, A2

DESIGN JUDGEMENT

Evidence is a retrievable artifact or test result, not a statement that the activity was completed.

Activity 4 · Verification Gate

PASS WHEN

A clean checkout can run one command and receive a non-zero exit on the injected defect and zero after repair.

IF IT FAILS

If Apple Clang rejects a flag, gate it by compiler ID and document the equivalent.

LEAVE THE WORKSPACE

Remove generated build and sanitizer output, not the verification script.

Topic 2 · Review

1 Explain

LO2: Apply C++ methodologies and tools for software creation.

LO2 · K2 · A2

2 Demonstrate

Show the Activity evidence that operationalises the outcome.

LO2 · K2 · A2

3 Diagnose

Name the failure signal and the layer where it originates.

LO2 · K2 · A2

4 Trace

Connect LO2 · K2 · A2 to the assessment and evidence pack.

LO2 · K2 · A2

DESIGN JUDGEMENT

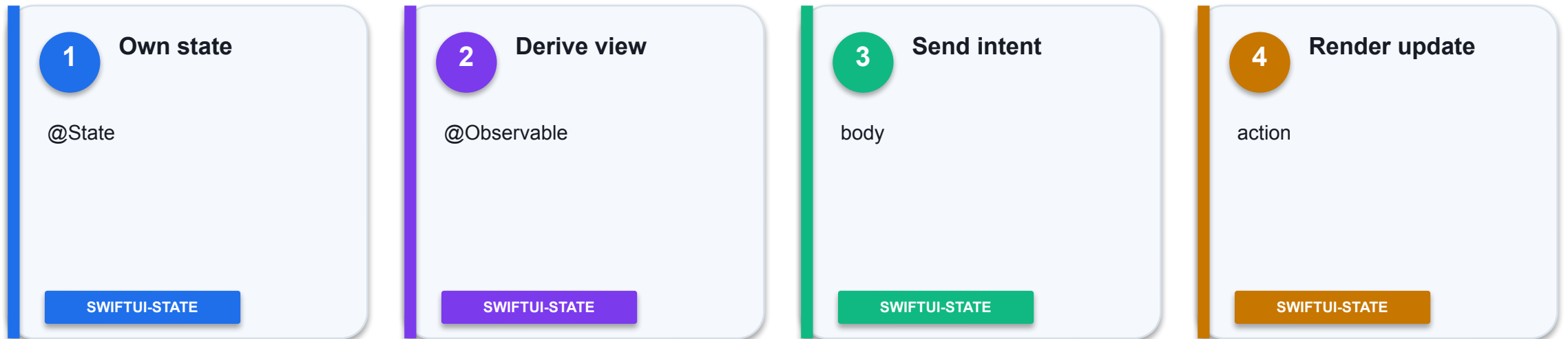
Do not advance on confidence alone; point to the artifact or test result that supports the answer.

TOPIC 3 · LO3 · K3 · A3

AI-Generated SwiftUI App Shell and Controls

LO3: Select essential C++ controls and features to meet software design requirements.

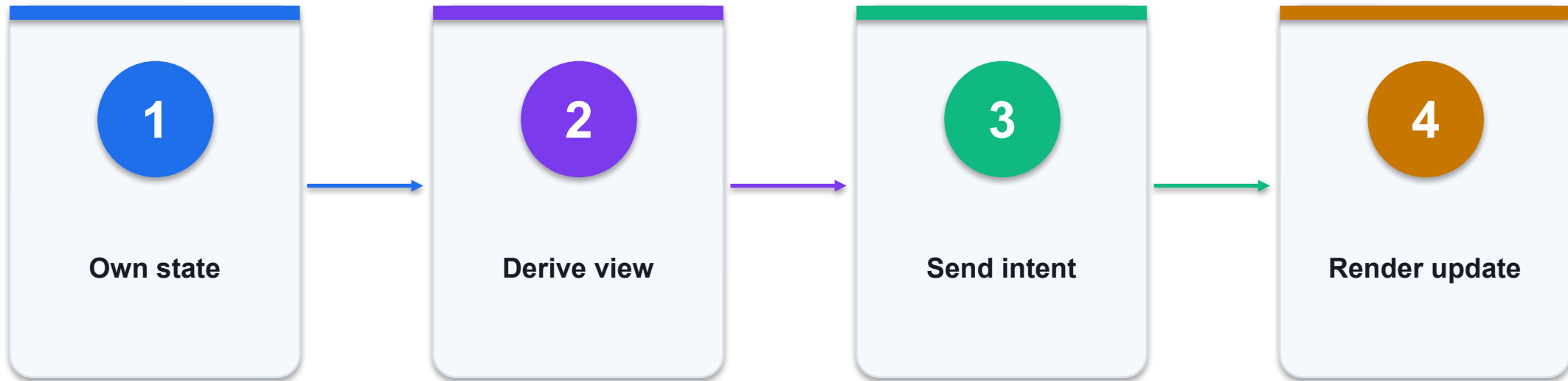
SwiftUI renders state as a function



DESIGN JUDGEMENT

Keep View declarative; send intent to a state owner that calls the bridge.

SwiftUI renders state as a function · Mechanism



MECHANISM

Failure mode: Business rules execute inside body and repeat on every render. **Control:** Keep View declarative; send intent to a state owner that calls the bridge.

SwiftUI renders state as a function · Technical Anchor

```
@Observable final class BudgetViewModel { private(set)  
var snapshot: Snapshot }
```

1

@State

2

@Observable

3

body

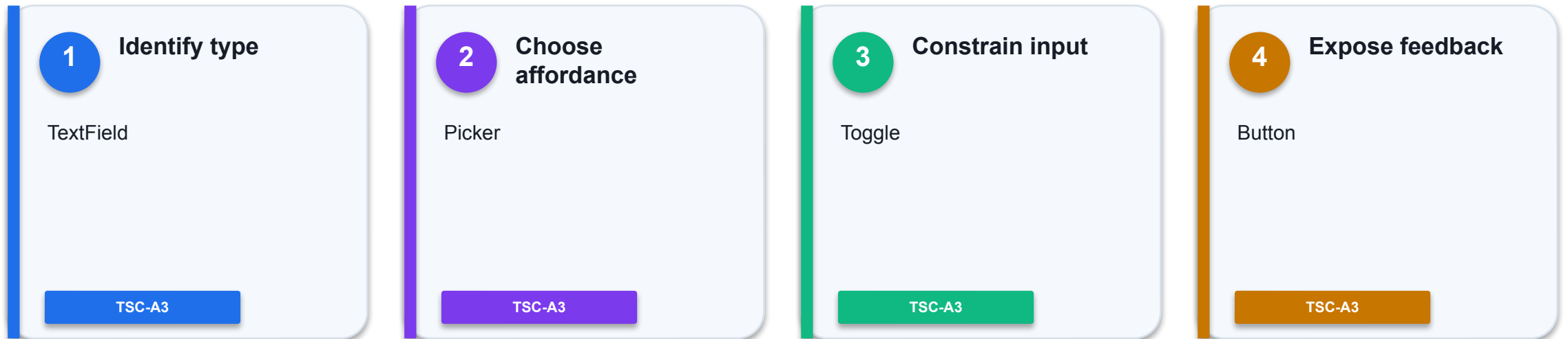
4

action

EVIDENCE METRIC

UI states covered by previews or tests

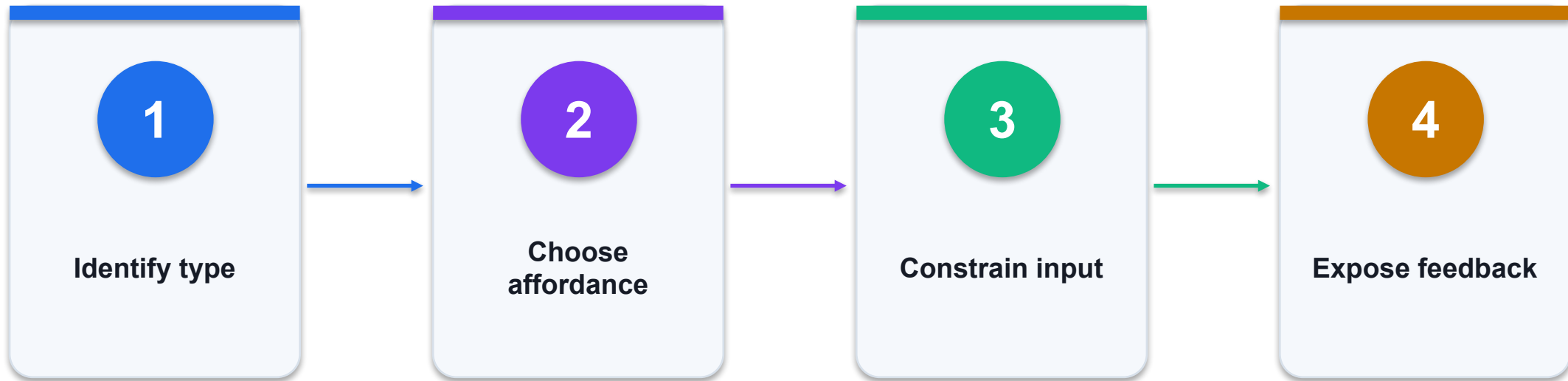
Select controls from the data contract



DESIGN JUDGEMENT

Choose the control that communicates range, optionality and consequence.

Select controls from the data contract · Mechanism



MECHANISM

Failure mode: A free-text field captures a value from a small controlled set. **Control:** Choose the control that communicates range, optionality and consequence.

Select controls from the data contract · Technical Anchor

```
Picker("Category", selection: $draft.category) { ... }
```

1

TextField

2

Picker

3

Toggle

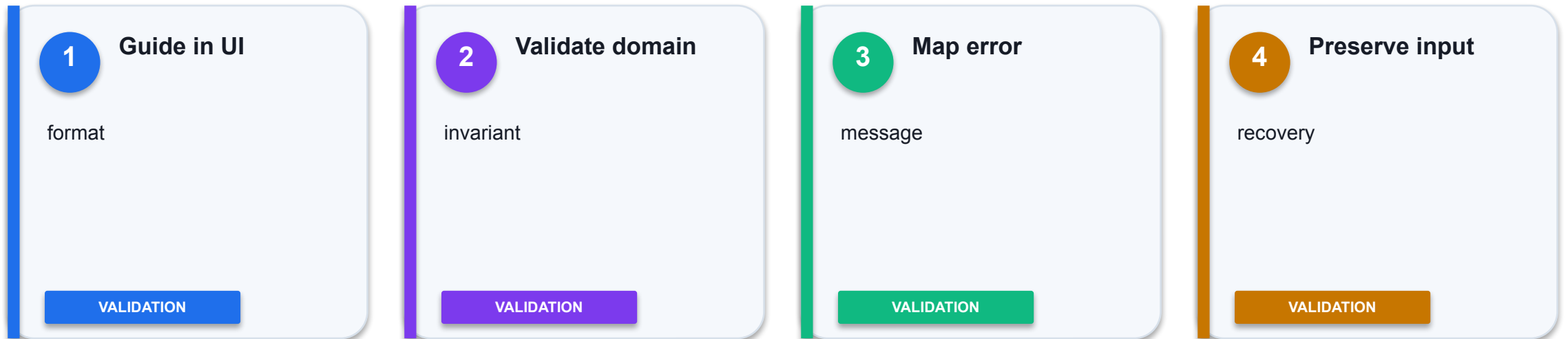
4

Button

EVIDENCE METRIC

Invalid states prevented or explained

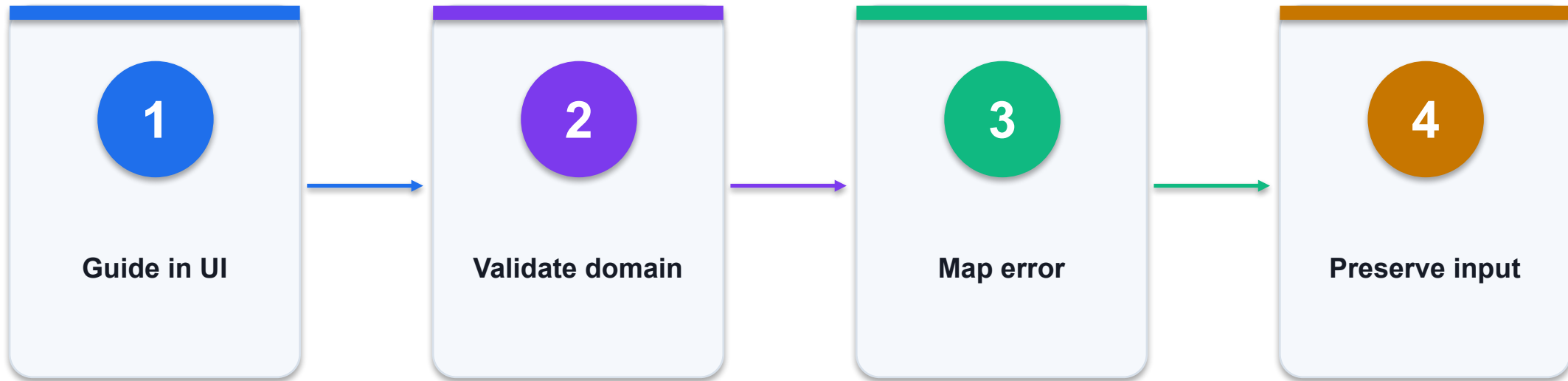
Validation belongs at two boundaries



DESIGN JUDGEMENT

Provide immediate SwiftUI guidance and enforce invariants again in C++.

Validation belongs at two boundaries · Mechanism



MECHANISM

Failure mode: UI validation is treated as sufficient security or correctness. **Control:** Provide immediate SwiftUI guidance and enforce invariants again in C++.

Validation belongs at two boundaries · Technical Anchor

```
guard let cents = MoneyParser.cents(text) else  
{ message = .invalidAmount }
```

1

format

2

invariant

3

message

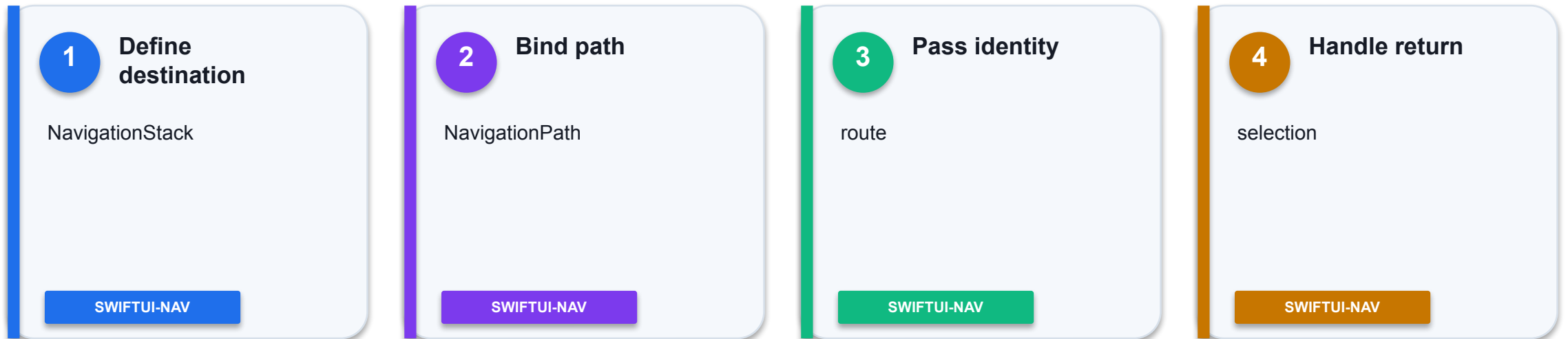
4

recovery

EVIDENCE METRIC

Rejected invalid inputs with recoverable state

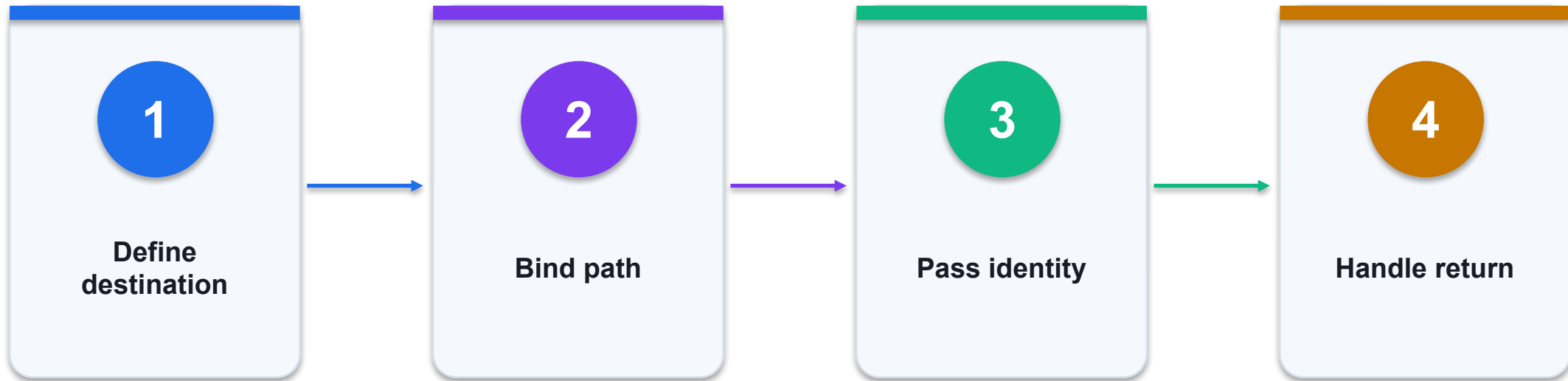
Navigation models workflow state



DESIGN JUDGEMENT

Route with identifiers and reload authoritative snapshots at boundaries.

Navigation models workflow state · Mechanism



MECHANISM

Failure mode: Views pass mutable models through an implicit navigation chain. **Control:** Route with identifiers and reload authoritative snapshots at boundaries.

Navigation models workflow state - Technical Anchor

```
enum Route: Hashable { case expenseDetail(UUID); case settings }
```

1

NavigationStack

2

NavigationPath

3

route

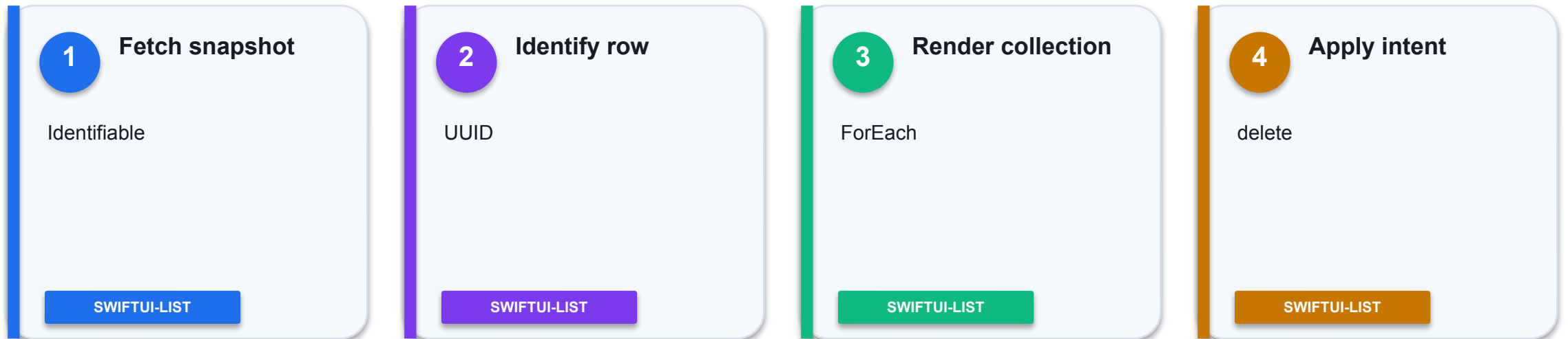
4

selection

EVIDENCE METRIC

Deep link and back path produce valid state

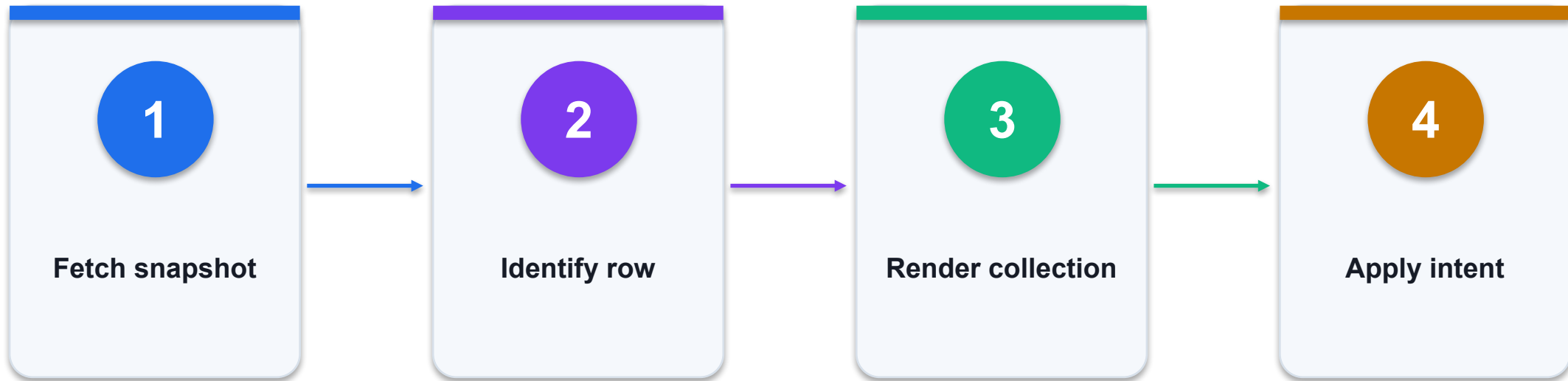
Lists need stable identity



DESIGN JUDGEMENT

Expose stable IDs from the core and reconcile snapshots by identity.

Lists need stable identity · Mechanism



MECHANISM

Failure mode: Array indices are used as identity and shift after deletion. **Control:** Expose stable IDs from the core and reconcile snapshots by identity.

Lists need stable identity · Technical Anchor

```
ForEach(model.snapshot.expenses, id: \.id)
{ ExpenseRow($0) }
```

1

Identifiable

2

UUID

3

ForEach

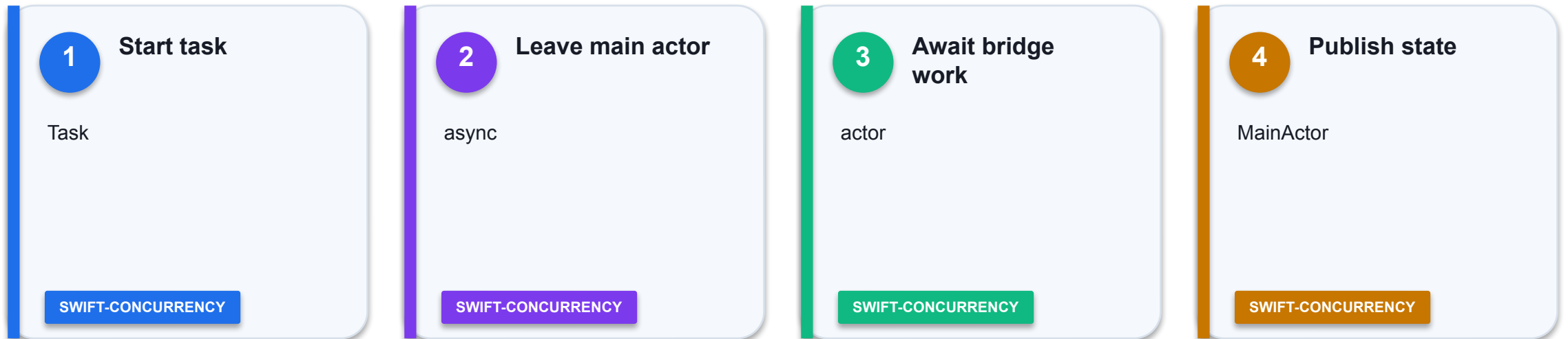
4

delete

EVIDENCE METRIC

Row updates affect exactly one domain entity

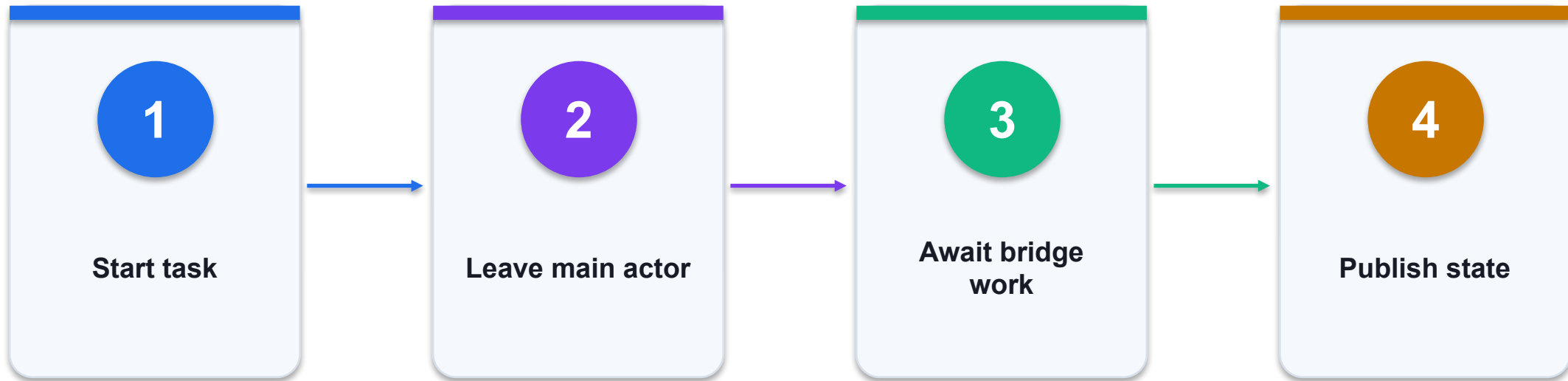
Swift concurrency guards responsiveness



DESIGN JUDGEMENT

Isolate expensive work and publish UI state on the main actor.

Swift concurrency guards responsiveness · Mechanism



MECHANISM

Failure mode: Disk or model inference runs synchronously from a button action. **Control:** Isolate expensive work and publish UI state on the main actor.

Swift concurrency guards responsiveness - Technical Anchor

```
Task { await model.refresh() }
```

1

Task

2

async

3

actor

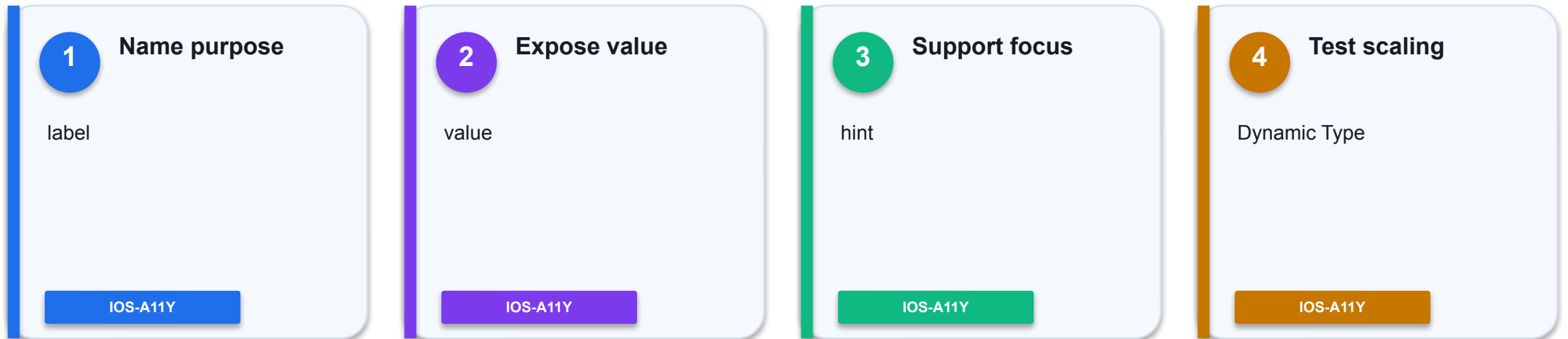
4

MainActor

EVIDENCE METRIC

No blocking work on main thread

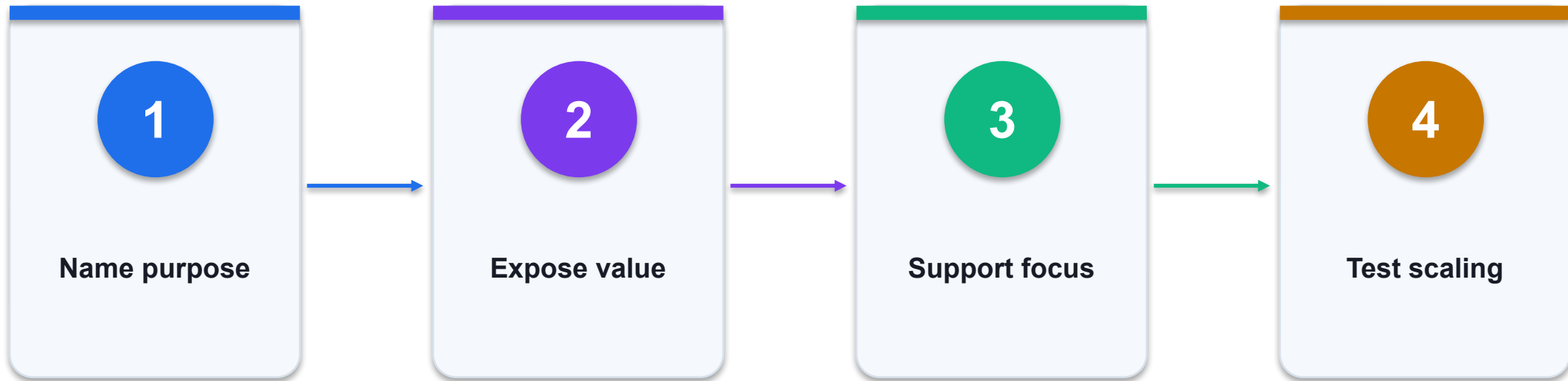
Accessibility is part of control design



DESIGN JUDGEMENT

Treat semantic labels and adaptable layout as acceptance criteria.

Accessibility is part of control design · Mechanism



MECHANISM

Failure mode: An icon-only control has no accessible name. **Control:** Treat semantic labels and adaptable layout as acceptance criteria.

Accessibility is part of control design · Technical Anchor

```
.accessibilityLabel("Delete expense")
```

1

label

2

value

3

hint

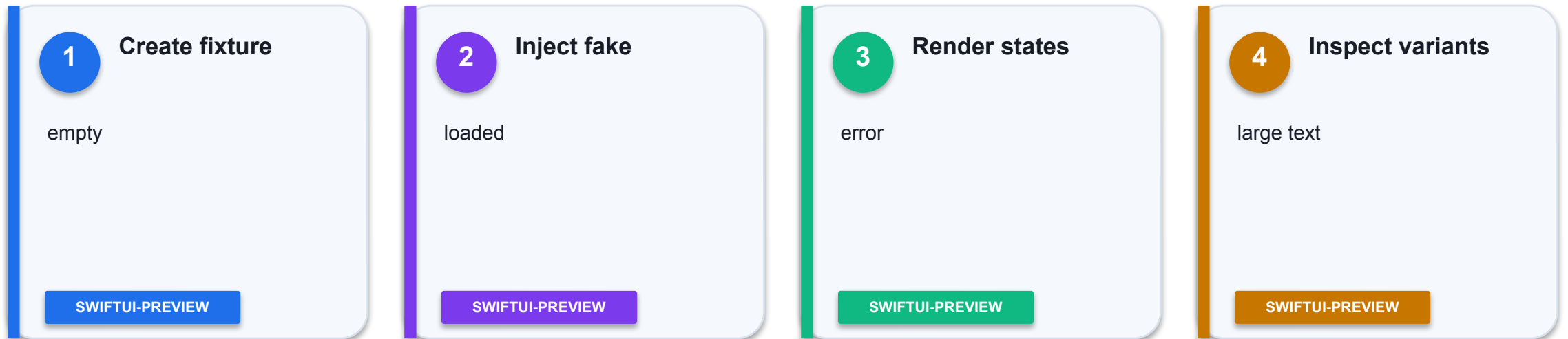
4

Dynamic Type

EVIDENCE METRIC

Critical flow works with VoiceOver and large text

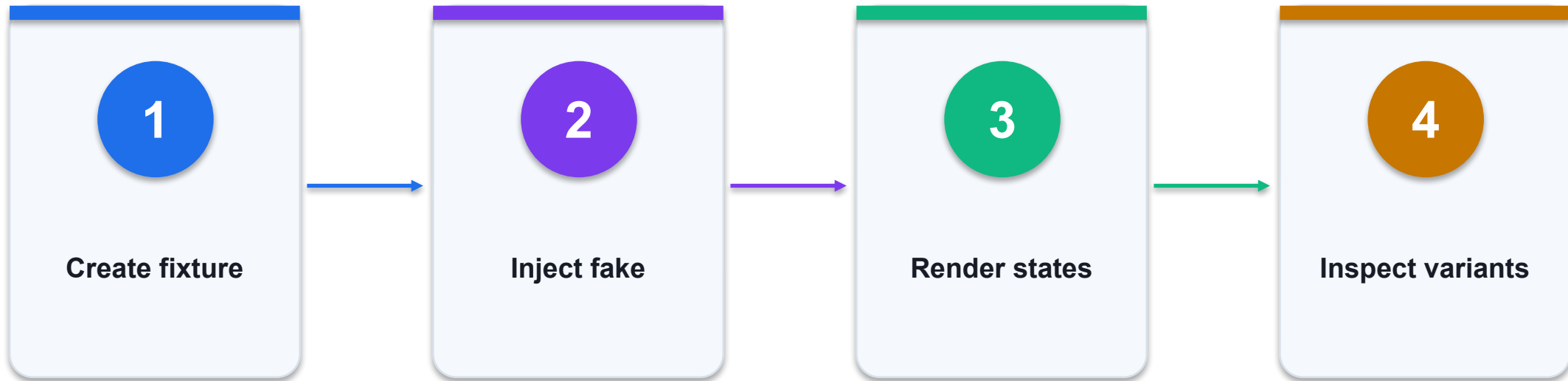
Previews are executable design probes



DESIGN JUDGEMENT

Inject deterministic snapshots and display state extremes.

Previews are executable design probes · Mechanism



MECHANISM

Failure mode: A preview depends on production data and never reaches failure states. **Control:** Inject deterministic snapshots and display state extremes.

Previews are executable design probes · Technical Anchor

```
#Preview("Error")  
{ BudgetScreen(model: .fixture(.error)) }
```

1

empty

2

loaded

3

error

4

large text

EVIDENCE METRIC

Representative states preview without network or storage

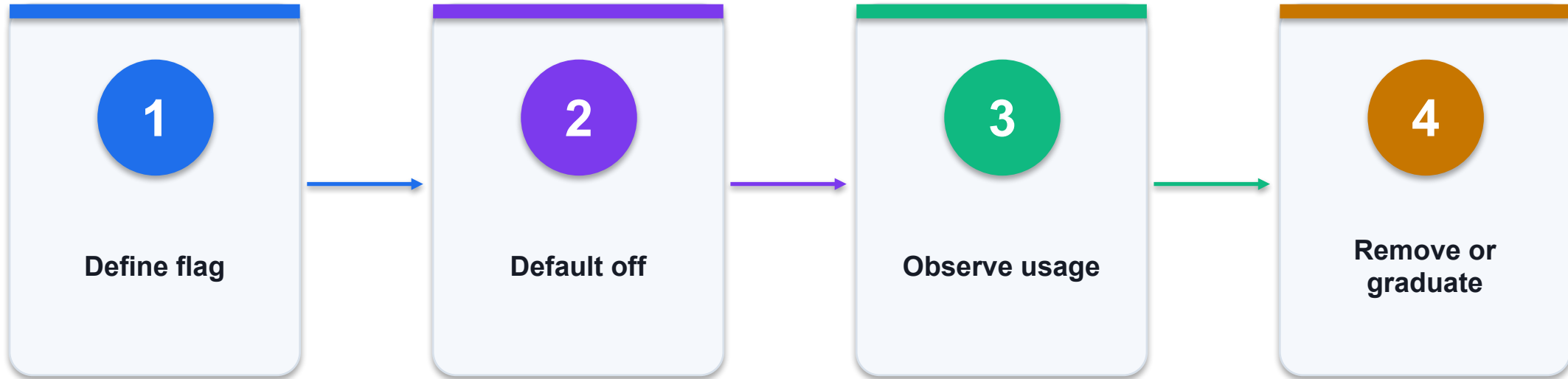
Feature flags bound experimental AI



DESIGN JUDGEMENT

Gate experiments, capture evidence and set a review date.

Feature flags bound experimental AI · Mechanism



MECHANISM

Failure mode: An AI-generated feature becomes permanent hidden complexity. **Control:** Gate experiments, capture evidence and set a review date.

Feature flags bound experimental AI - Technical Anchor

```
if flags.isEnabled(.smartCategorySuggestion) { ... }
```

1

identifier

2

default

3

telemetry

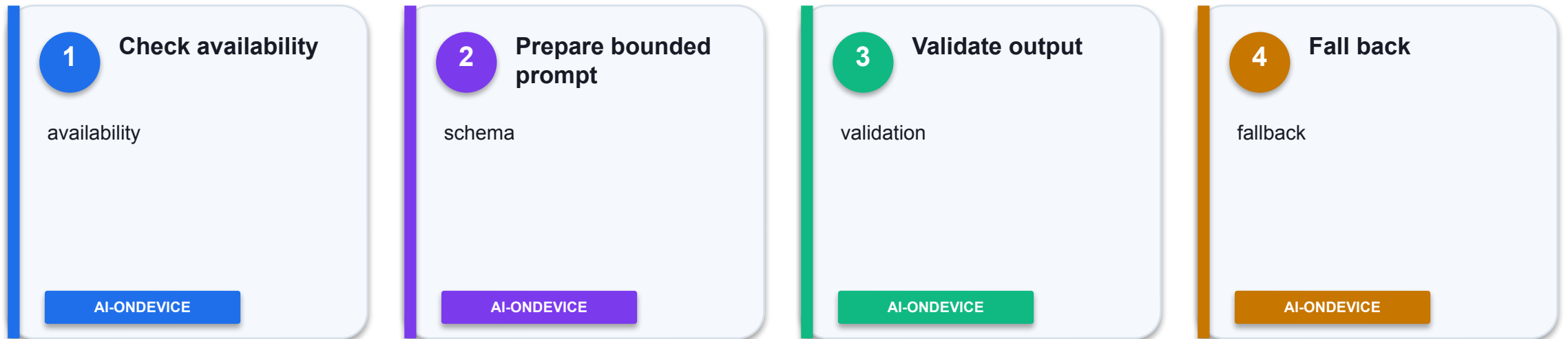
4

owner

EVIDENCE METRIC

Experimental path has owner and expiry

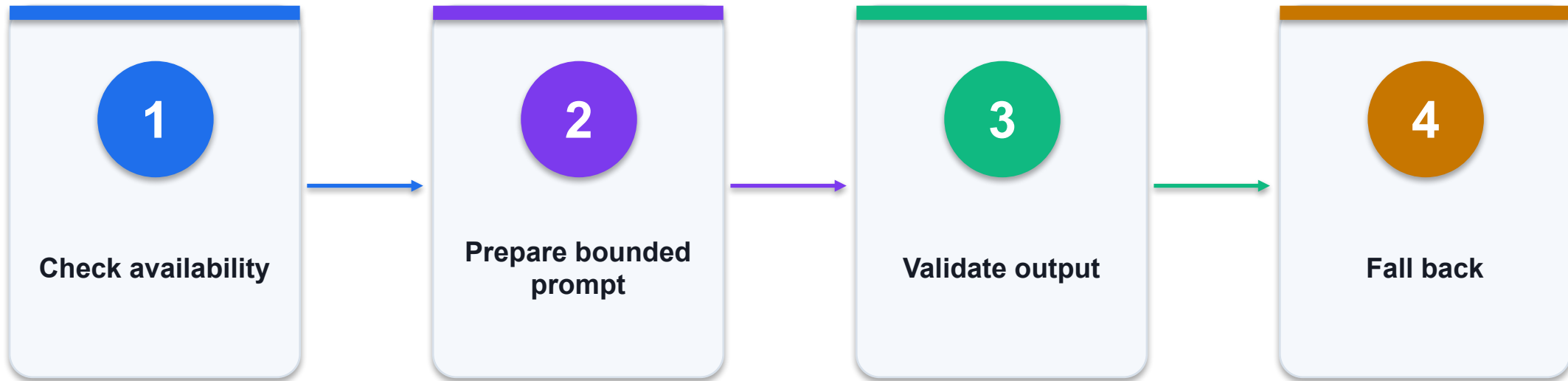
Local Foundation Models stay supplemental



DESIGN JUDGEMENT

Use model output as a suggestion; validate before committing to the core.

Local Foundation Models stay supplemental · Mechanism



MECHANISM

Failure mode: A local model becomes a hidden source of business truth. **Control:** Use model output as a suggestion; validate before committing to the core.

Local Foundation Models stay supplemental · Technical Anchor

```
guard modelSystem.isAvailable else  
{ return .manualEntry }
```

1

availability

2

schema

3

validation

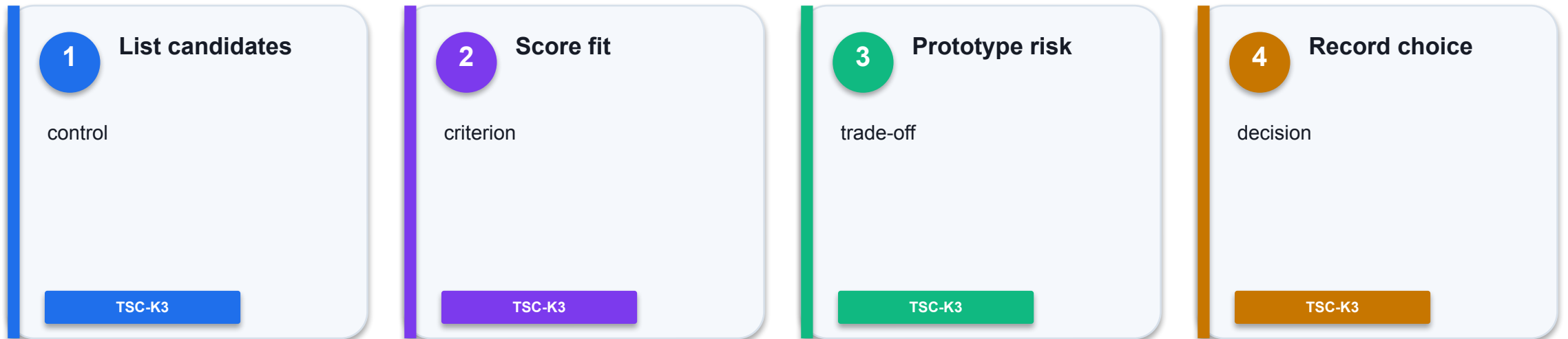
4

fallback

EVIDENCE METRIC

Deterministic core works when model is unavailable

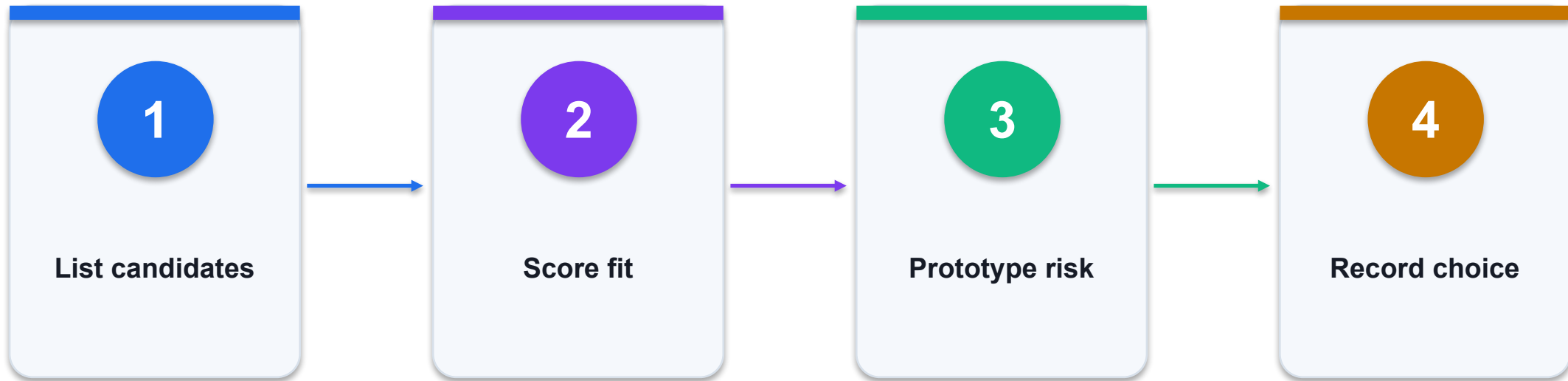
Design selections require rationale



DESIGN JUDGEMENT

Record why the chosen control better enforces the requirement.

Design selections require rationale · Mechanism



MECHANISM

Failure mode: Controls are selected because they are familiar or visually fashionable. **Control:** Record why the chosen control better enforces the requirement.

Design selections require rationale · Technical Anchor

```
// Decision: Picker because category is closed and required.
```

1

control

2

criterion

3

trade-off

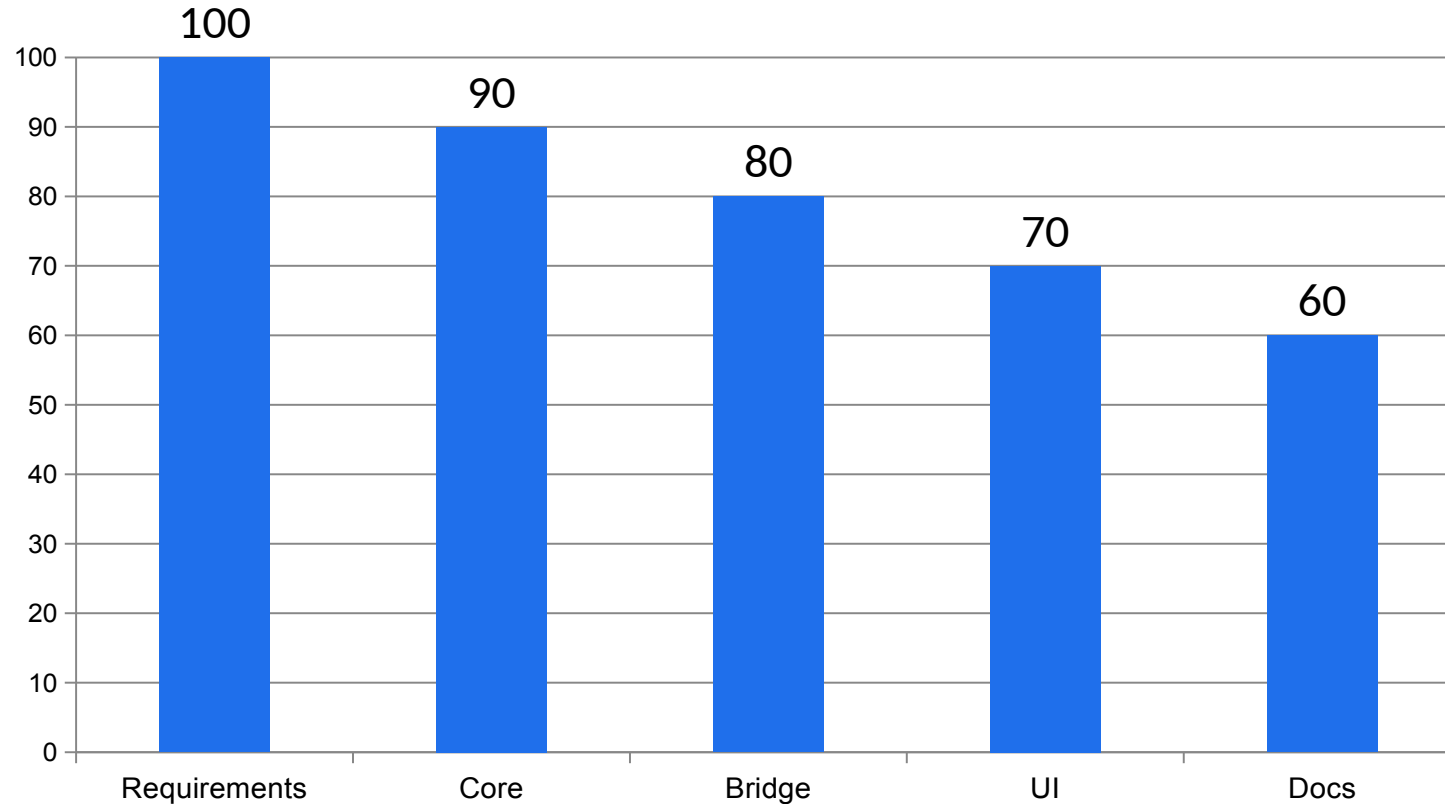
4

decision

EVIDENCE METRIC

Each material control linked to objective

Topic 3 · Evidence Coverage by Layer



WHAT THE DATA SHOWS

Coverage should form a connected chain. A polished UI cannot compensate for untested invariants or an undocumented seam.

Editable chart · values are instructional thresholds, not production benchmarks

Activity 5: Objective-C++ Adapter

CHALLENGE

Expose the portable domain core to Swift through a narrow Objective-C-compatible API.

REGULATED MAPPING

K3, A3

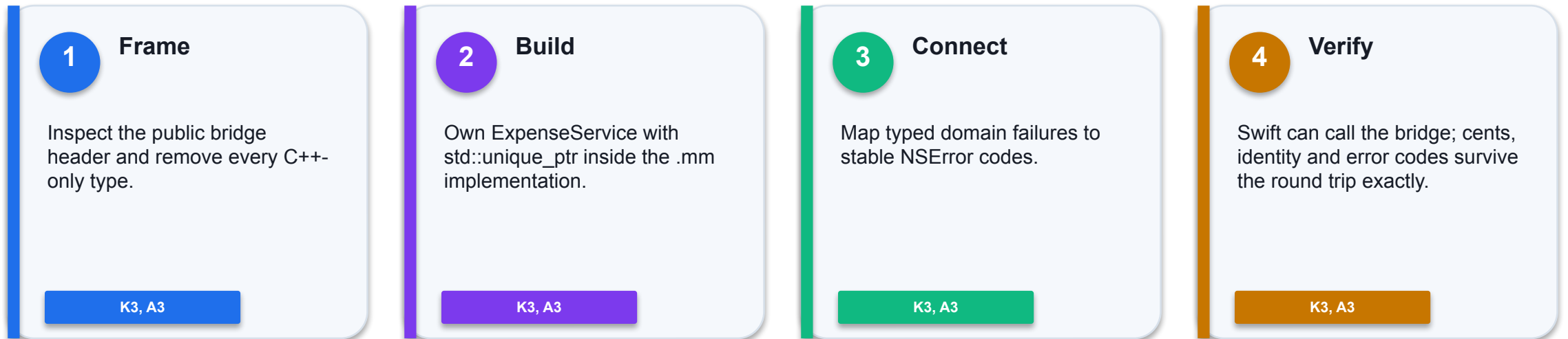
YOU WILL PRODUCE

Bridge header, .mm adapter and round-trip test

Xcode, Objective-C++, XCTest

Detailed procedures: [Learner Guide and activities/05-objective-cpp-adapter/README.md](#)

Activity 5 · Delivery Workflow



DESIGN JUDGEMENT

The slide communicates the workflow; the Learner Guide carries the detailed procedure and commands.

Activity 5 · Vibe Coding Generation Prompt

GENERATION PROMPT

```
Generate BudgetBridge.h, BudgetBridge.mm and a bridge contract test.  
Keep the header Objective-C compatible. Put C++ includes and  
std::unique_ptr ownership in .mm. Preserve exact cents and map each  
DomainError to a stable NSError code plus recovery description. Output  
complete files. Verify iOS build, 10_000/2_500/1_250 -> 6_250 and every  
error mapping.
```

PROTECTED FILE SCOPE

**BudgetBridge.h; BudgetBridge.mm; bridge-
test.swift; error-mapping.md**

EXPECTED GREEN

**iOS build and focused bridge tests exit 0; the
public header contains no
STL/template/reference/owning-pointer type
and exact cents survive the round trip.**

REVIEW / REPAIR

**Use the paired review prompt in the Activity
guide. Fix the first causal failure, preserve
tests, then rerun focused and full checks.**

Activity 5 · Evidence Contract

1 Artifact

Bridge header, .mm adapter and round-trip test

K3, A3

2 Acceptance

Swift can call the bridge; cents, identity and error codes survive the round trip exactly.

K3, A3

3 Failure signal

If Swift cannot import the header, check target membership and confirm C++ includes appear only in .mm.

K3, A3

4 Cleanup

Clean derived build data only if needed; retain tests and mapping table.

K3, A3

DESIGN JUDGEMENT

Evidence is a retrievable artifact or test result, not a statement that the activity was completed.

Activity 5 · Verification Gate

PASS WHEN

Swift can call the bridge; cents, identity and error codes survive the round trip exactly.

IF IT FAILS

If Swift cannot import the header, check target membership and confirm C++ includes appear only in .mm.

LEAVE THE WORKSPACE

Clean derived build data only if needed; retain tests and mapping table.

Activity 6: SwiftUI Controls and State

CHALLENGE

Build an accessible SwiftUI expense-entry shell whose controls reflect the domain contract.

REGULATED MAPPING

K3, A3

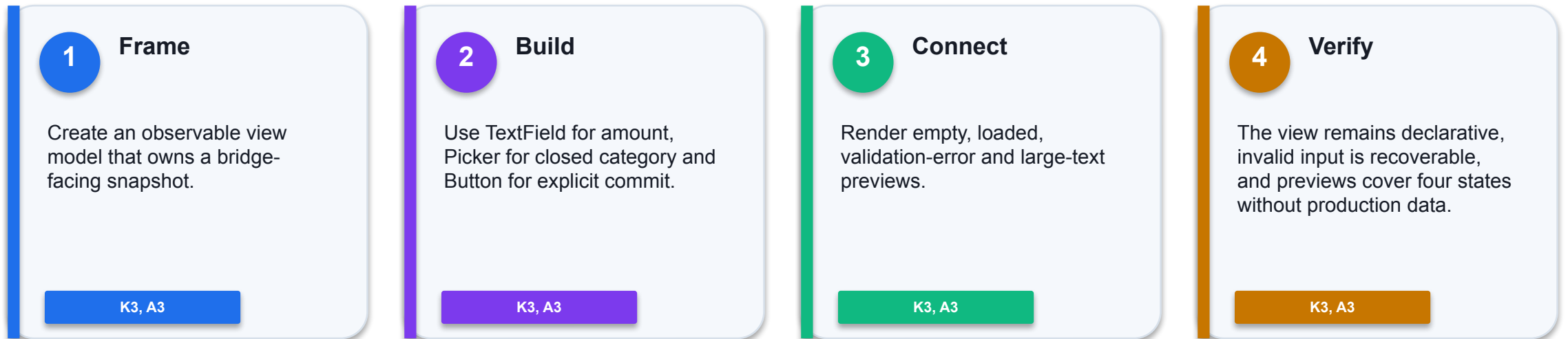
YOU WILL PRODUCE

SwiftUI entry flow, deterministic previews and validation states

SwiftUI, Observation, Xcode previews

Detailed procedures: [Learner Guide and activities/06-swiftui-controls-state/README.md](#)

Activity 6 · Delivery Workflow



DESIGN JUDGEMENT

The slide communicates the workflow; the Learner Guide carries the detailed procedure and commands.

Activity 6 · Vibe Coding Generation Prompt

GENERATION PROMPT

```
Generate BudgetEvaluating, BudgetBridgeClient, FakeBudgetBridge, BudgetModel and @Observable BudgetViewModel. Then generate BudgetBuddyApp, AppRoot, BudgetScreen, summary/form/list/row views and deterministic previews. Use accessible TextField, Picker and Button identifiers. Keep policy in C++. Output complete files. Verify Swift Testing state tests, iOS 17+ build and empty/loaded/error/large-text previews.
```

PROTECTED FILE SCOPE

BudgetEvaluating.swift; BudgetBridgeClient.swift; FakeBudgetBridge.swift; BudgetModel.swift; BudgetViewModel.swift; BudgetBuddyApp.swift; AppRoot.swift; BudgetScreen.swift; supporting views and previews

EXPECTED GREEN

Swift Testing proves parsing/state/error/retry with a fake; iOS 17+ build succeeds; four deterministic previews construct; required accessibility identifiers exist.

REVIEW / REPAIR

Use the paired review prompt in the Activity guide. Fix the first causal failure, preserve tests, then rerun focused and full checks.

Activity 6 · Evidence Contract

1 Artifact

SwiftUI entry flow, deterministic previews and validation states

K3, A3

2 Acceptance

The view remains declarative, invalid input is recoverable, and previews cover four states without production data.

K3, A3

3 Failure signal

If previews fail, inject the fixture model and remove filesystem work from initialisation.

K3, A3

4 Cleanup

Retain preview fixtures; remove screenshots containing personal data.

K3, A3

DESIGN JUDGEMENT

Evidence is a retrievable artifact or test result, not a statement that the activity was completed.

Activity 6 · Verification Gate

PASS WHEN

The view remains declarative, invalid input is recoverable, and previews cover four states without production data.

IF IT FAILS

If previews fail, inject the fixture model and remove filesystem work from initialisation.

LEAVE THE WORKSPACE

Retain preview fixtures; remove screenshots containing personal data.

Topic 3 · Review

1 Explain

LO3: Select essential C++ controls and features to meet software design requirements.

LO3 · K3 · A3

2 Demonstrate

Show the Activity evidence that operationalises the outcome.

LO3 · K3 · A3

3 Diagnose

Name the failure signal and the layer where it originates.

LO3 · K3 · A3

4 Trace

Connect LO3 · K3 · A3 to the assessment and evidence pack.

LO3 · K3 · A3

DESIGN JUDGEMENT

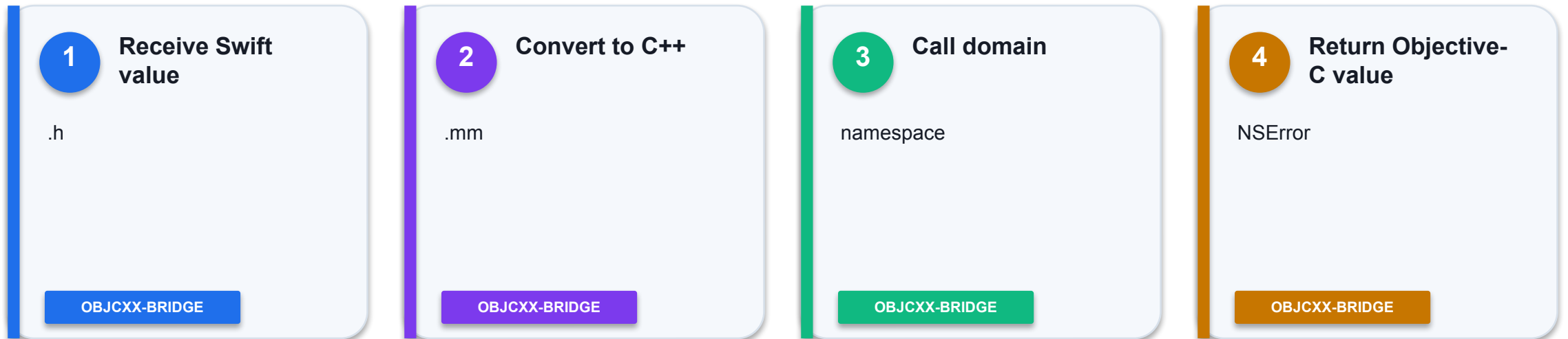
Do not advance on confidence alone; point to the artifact or test result that supports the answer.

TOPIC 4 · LO4 · K4 · A4

iOS Interoperability, Persistence and Validation

LO4: Examine the interoperability and functionality of C++ software components.

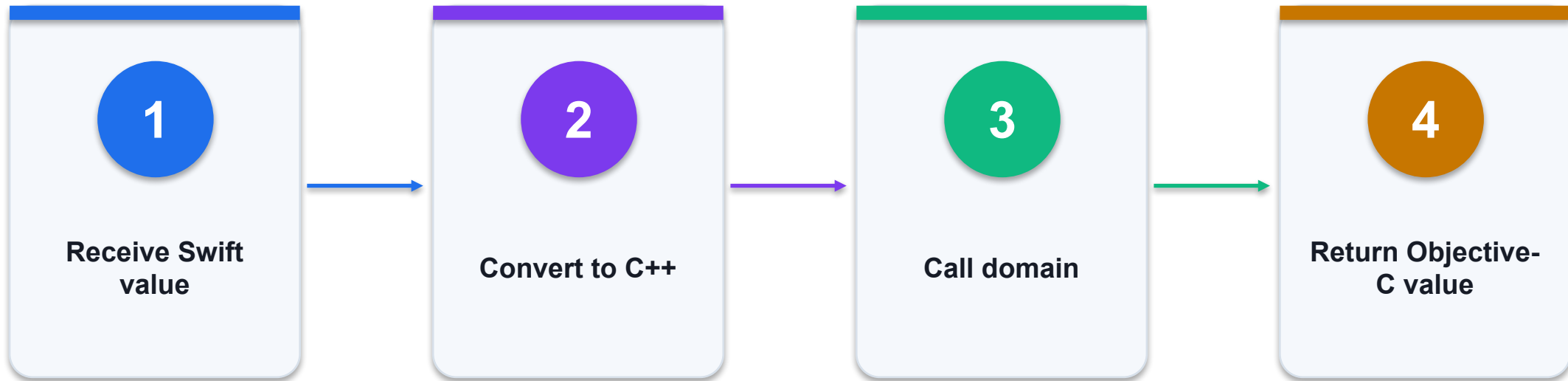
Objective-C++ owns language translation



DESIGN JUDGEMENT

Keep the header Objective-C compatible and put C++ includes in `.mm`.

Objective-C++ owns language translation · Mechanism



MECHANISM

Failure mode: C++ types leak through the public Objective-C header. Control: Keep the header Objective-C compatible and put C++ includes in .mm.

Objective-C++ owns language translation · Technical Anchor

```
// BudgetBridge.mm  
auto result = core_>add(toCpp(draft));
```

1

.h

2

.mm

3

namespace

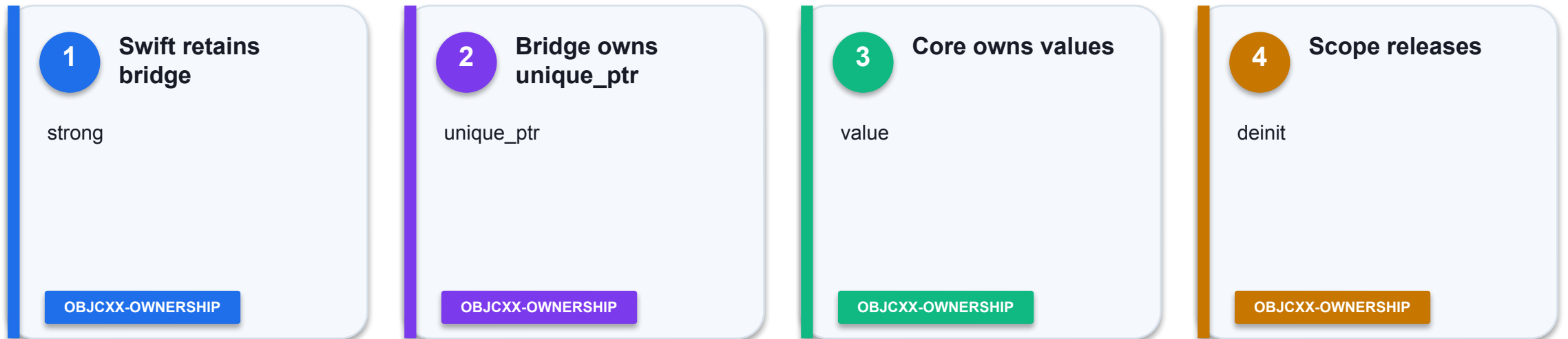
4

NSError

EVIDENCE METRIC

One translation point per operation

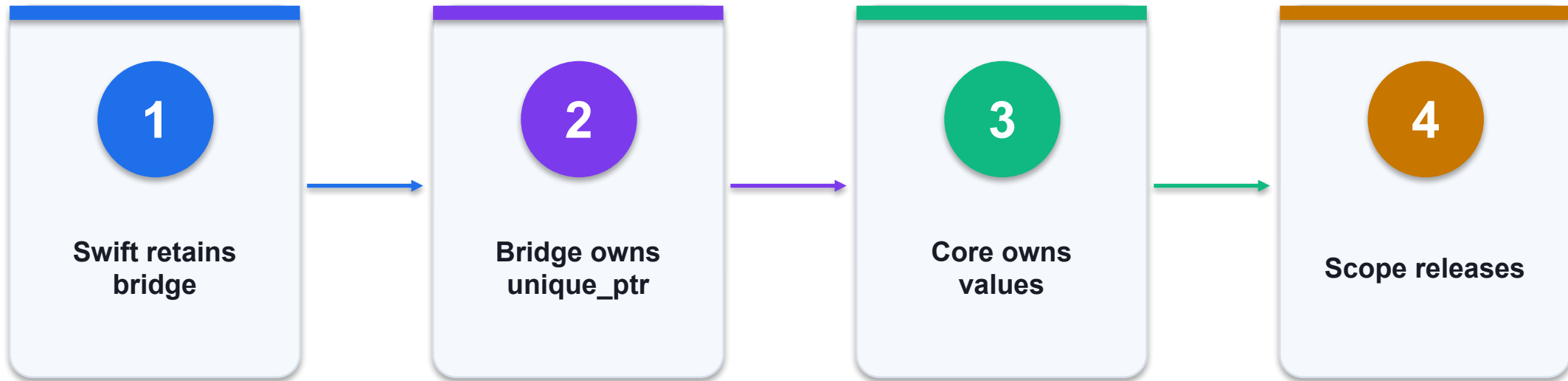
Ownership across ARC and RAI



DESIGN JUDGEMENT

Let the adapter own long-lived C++ services; return copied immutable DTOs.

Ownership across ARC and RAII · Mechanism



MECHANISM

Failure mode: Swift stores a pointer to a temporary C++ object. **Control:** Let the adapter own long-lived C++ services; return copied immutable DTOs.

Ownership across ARC and RAII - Technical Anchor

```
std::unique_ptr<ExpenseService> service_;
```

1

strong

2

unique_ptr

3

value

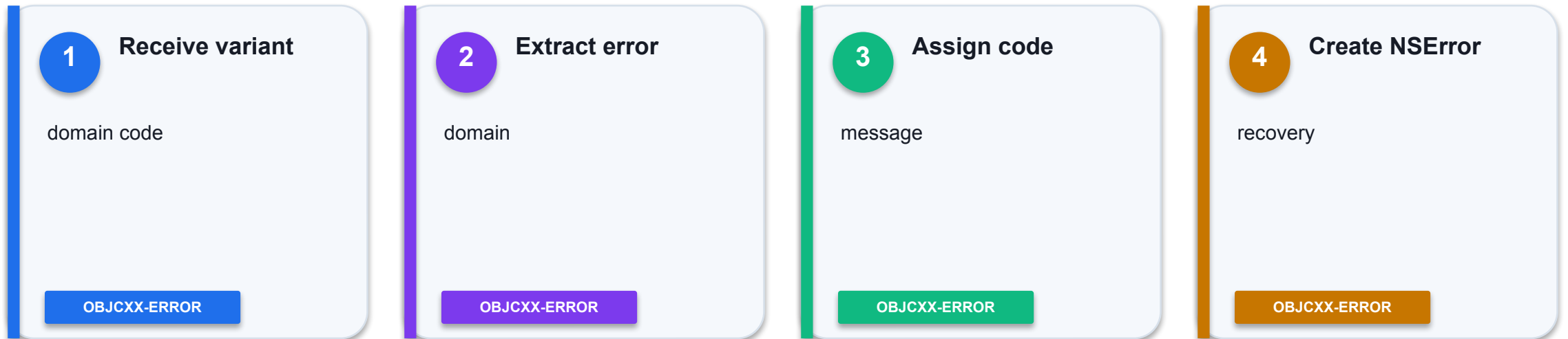
4

deinit

EVIDENCE METRIC

No leaks or dangling pointers under repeated use

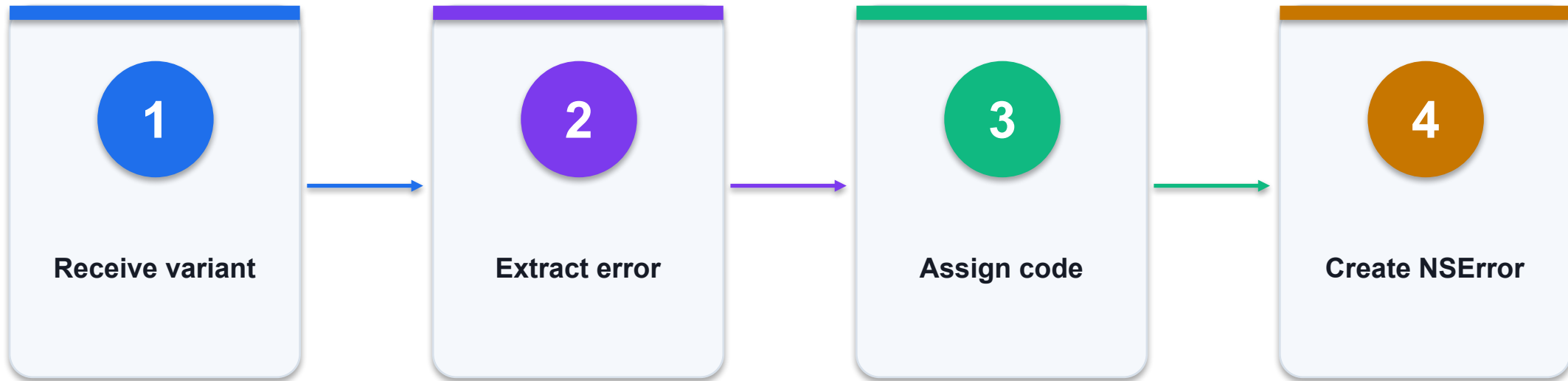
Map typed errors once



DESIGN JUDGEMENT

Centralise DomainError-to-NSError mapping in the adapter.

Map typed errors once · Mechanism



MECHANISM

Failure mode: Every screen invents a different string for the same domain failure. **Control:** Centralise DomainError-to-NSError mapping in the adapter.

Map typed errors once · Technical Anchor

```
NSError *error = [NSError errorWithDomain:BudgetDomain  
code:code userInfo:info];
```

1

domain code

2

domain

3

message

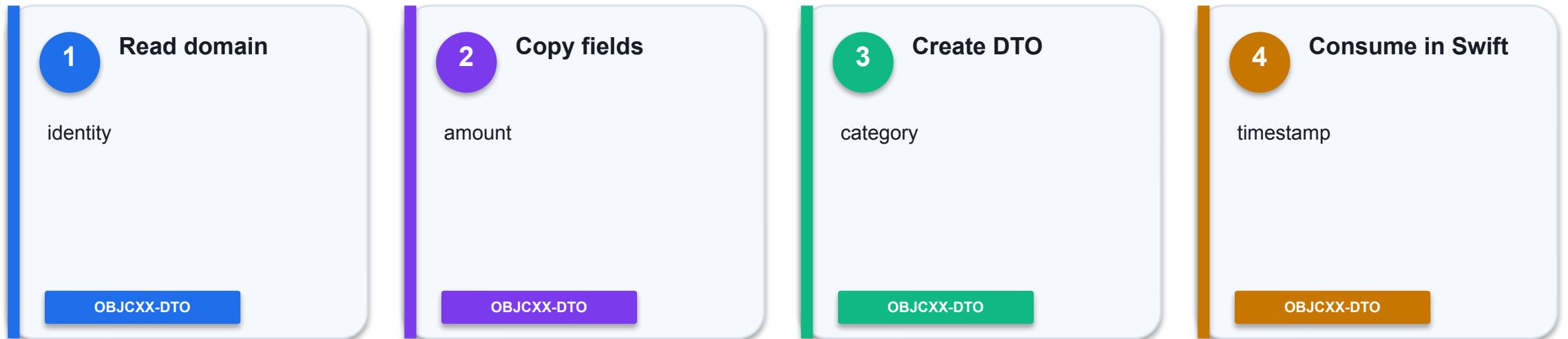
4

recovery

EVIDENCE METRIC

Stable error codes reach Swift tests

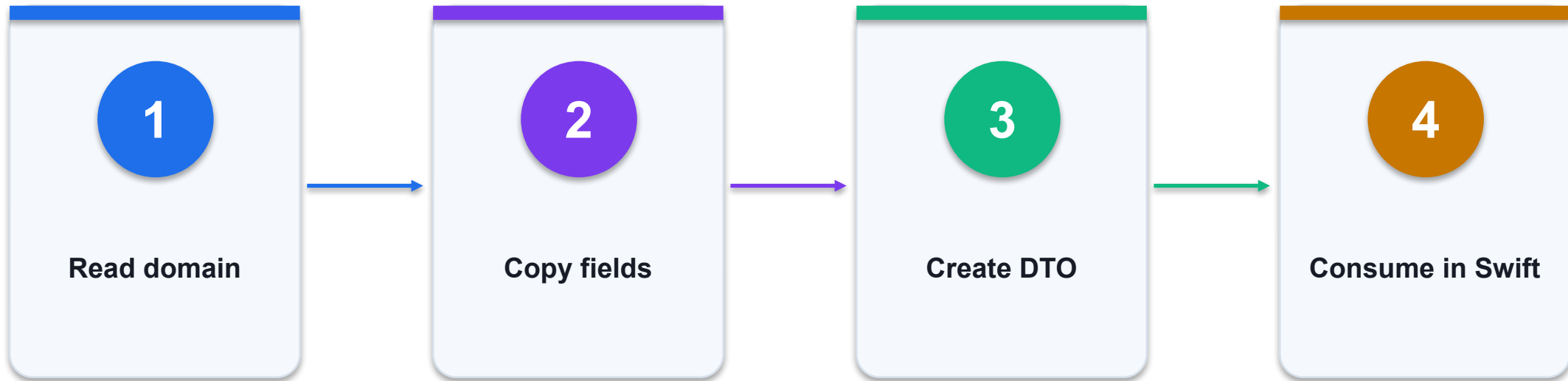
DTOs freeze the boundary shape



DESIGN JUDGEMENT

Use explicit DTO objects with stable identity and documented units.

DTOs freeze the boundary shape · Mechanism



MECHANISM

Failure mode: Swift receives multiple arrays whose indices must stay aligned. **Control:** Use explicit DTO objects with stable identity and documented units.

DTOs freeze the boundary shape · Technical Anchor

```
@interface ExpenseDTO : NSObject @property(readonly)  
int64_t cents; @end
```

1

identity

2

amount

3

category

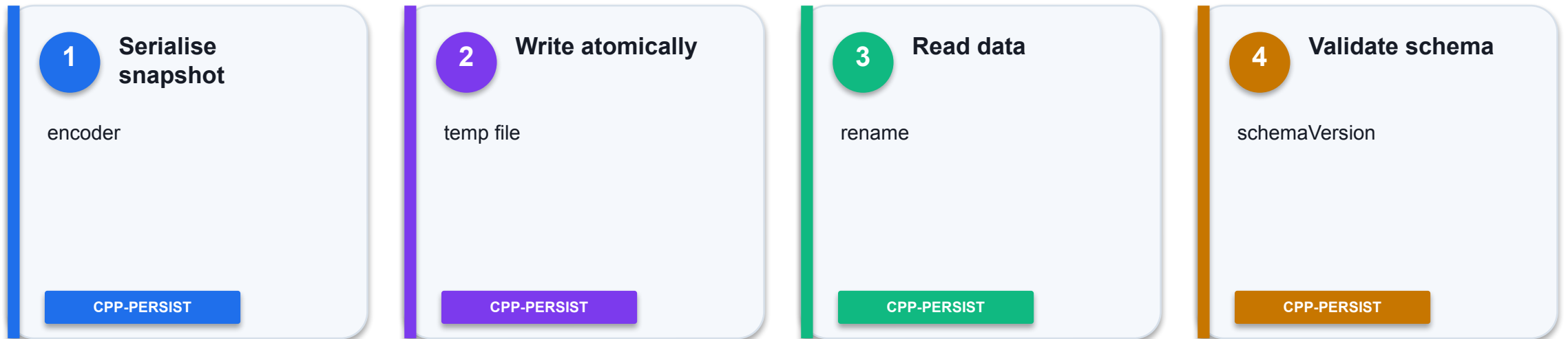
4

timestamp

EVIDENCE METRIC

Swift sees immutable, complete records

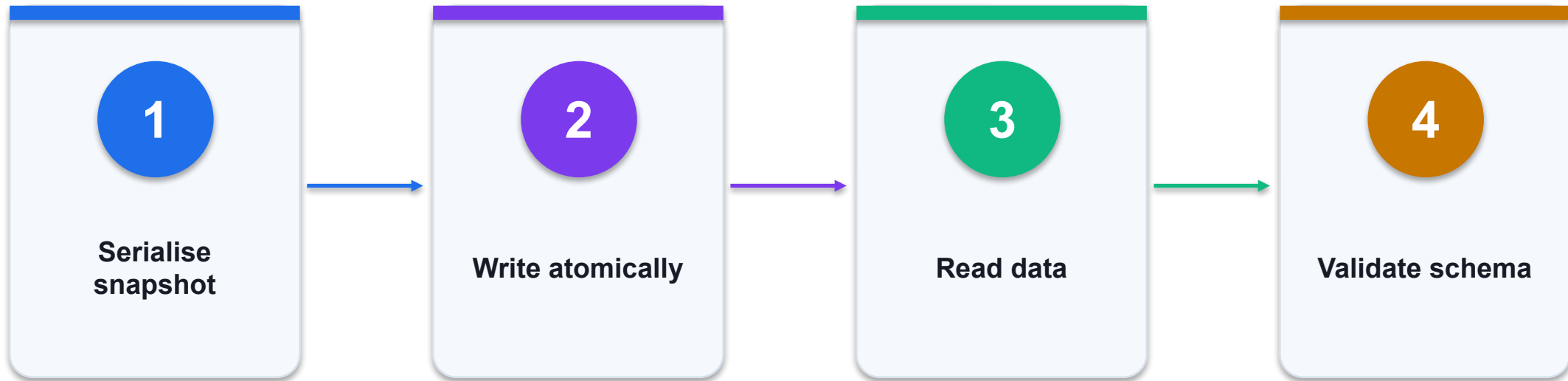
JSON persistence stays behind a port



DESIGN JUDGEMENT

Write to a temporary file, fsync where relevant, then replace atomically.

JSON persistence stays behind a port · Mechanism



MECHANISM

Failure mode: A partial write corrupts the only saved file. **Control:** Write to a temporary file, fsync where relevant, then replace atomically.

JSON persistence stays behind a port · Technical Anchor

```
repository.save(snapshot); // port hides JSON and  
filesystem
```

1

encoder

2

temp file

3

rename

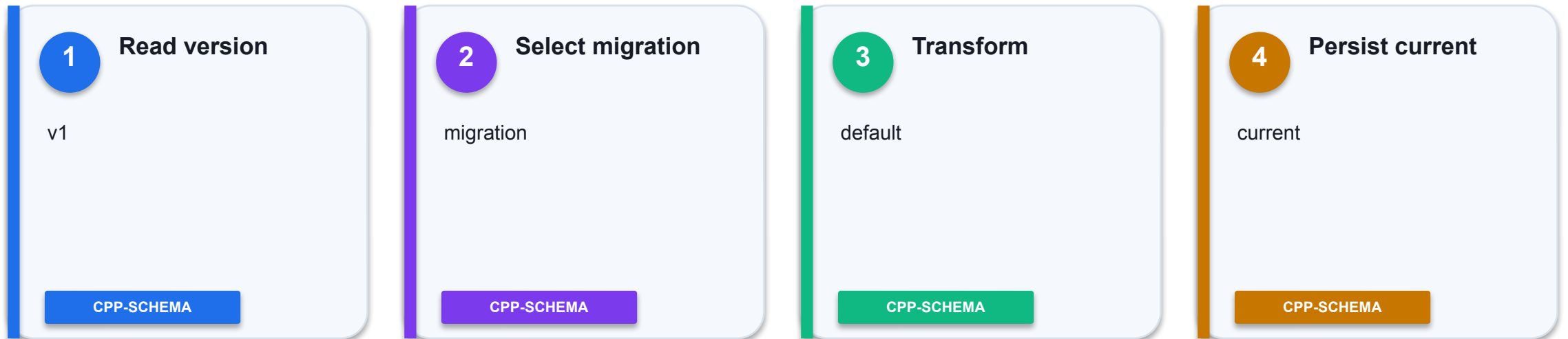
4

schemaVersion

EVIDENCE METRIC

Restart reproduces last accepted state

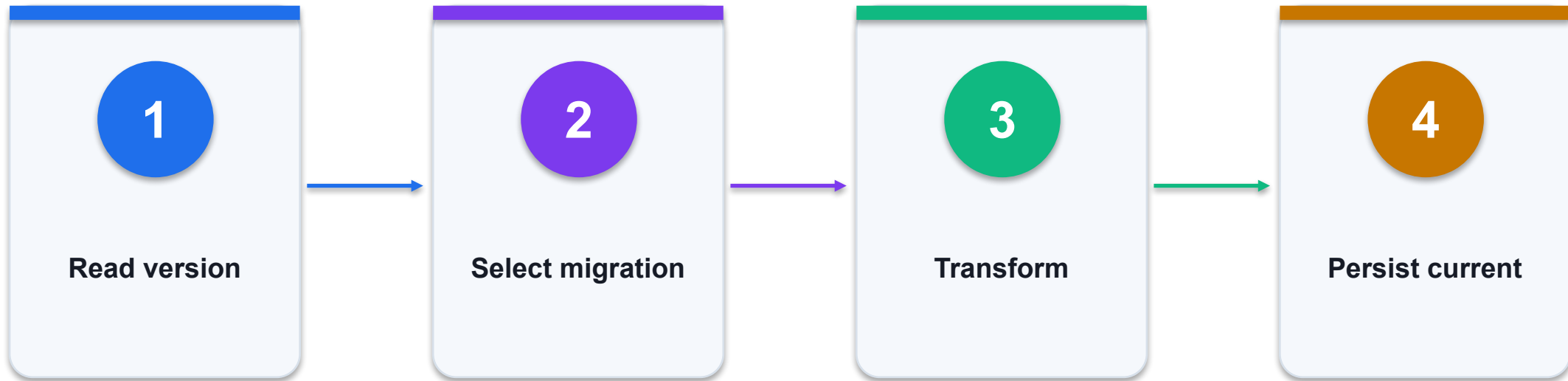
Schema evolution needs a version



DESIGN JUDGEMENT

Version persisted documents and test every supported migration path.

Schema evolution needs a version · Mechanism



MECHANISM

Failure mode: A new field makes existing user data unreadable. **Control:** Version persisted documents and test every supported migration path.

Schema evolution needs a version · Technical Anchor

```
if (doc.schemaVersion == 1) return migrateV1(doc);
```

1

v1

2

migration

3

default

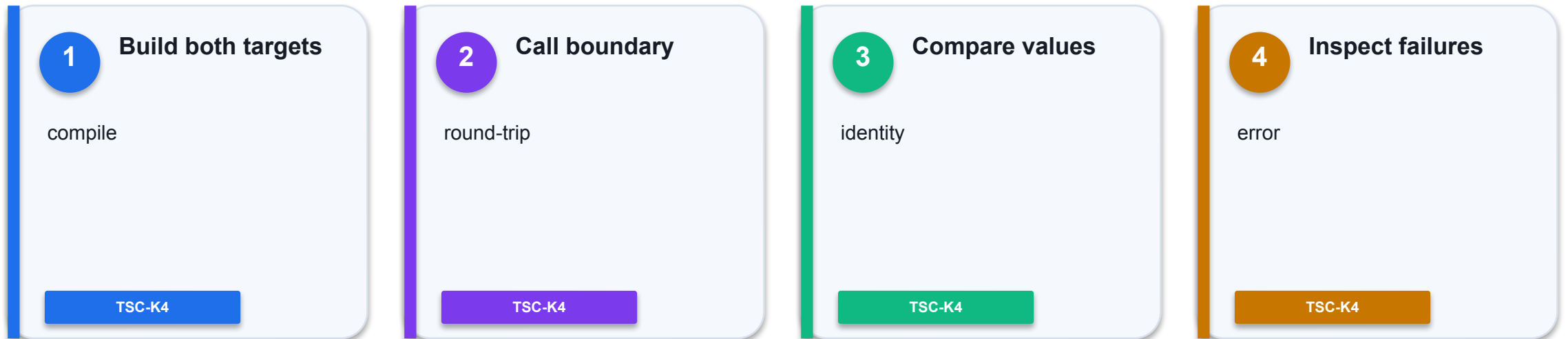
4

current

EVIDENCE METRIC

Legacy fixtures load into current model

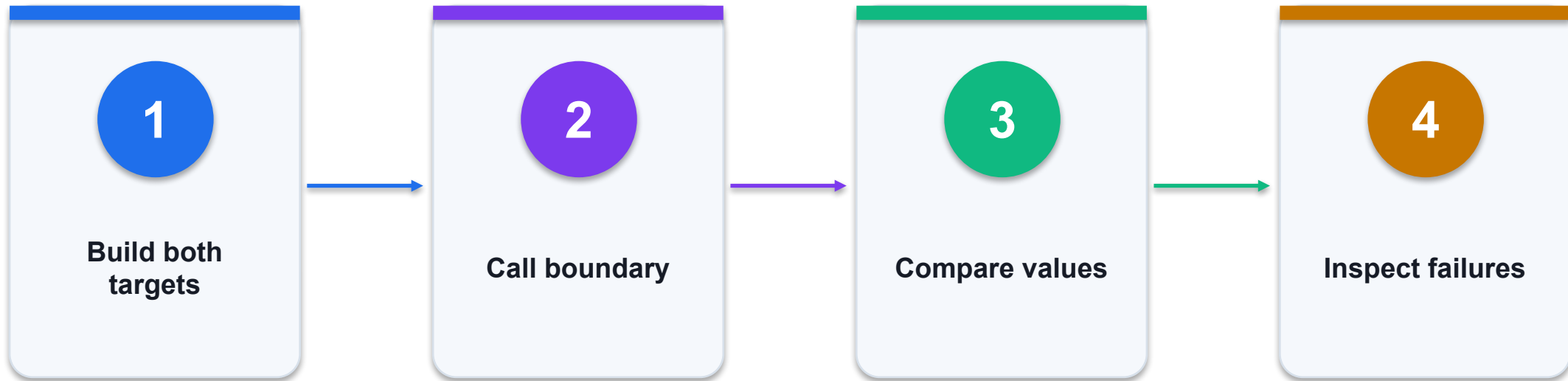
Interoperability has observable indicators



DESIGN JUDGEMENT

Measure compile compatibility, value fidelity, error fidelity and lifecycle safety.

Interoperability has observable indicators · Mechanism



MECHANISM

Failure mode: Only a happy-path simulator demo is used as evidence. **Control:** Measure compile compatibility, value fidelity, error fidelity and lifecycle safety.

Interoperability has observable indicators · Technical Anchor

```
EXPECT_EQ(swiftRoundTrip.amountCents,  
cppInput.amount.cents);
```

1

compile

2

round-trip

3

identity

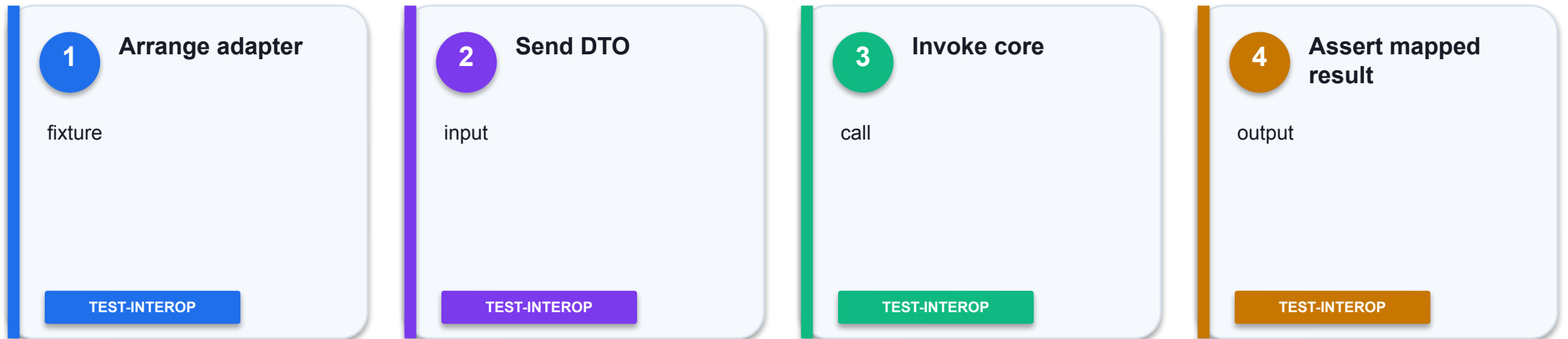
4

error

EVIDENCE METRIC

Contract tests pass for success and failure paths

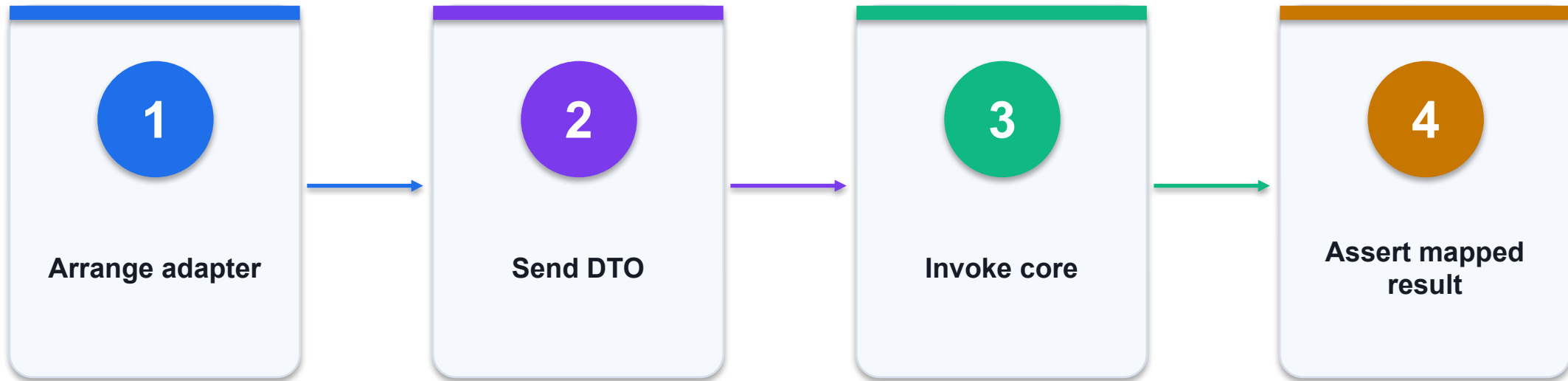
Contract tests isolate the seam



DESIGN JUDGEMENT

Test the translation seam directly with deterministic core doubles.

Contract tests isolate the seam · Mechanism



MECHANISM

Failure mode: Bridge tests are postponed until full end-to-end testing. **Control:** Test the translation seam directly with deterministic core doubles.

Contract tests isolate the seam · Technical Anchor

```
func testNegativeAmountMapsToStableErrorCode() throws {  
    ...  
}
```

1

fixture

2

input

3

call

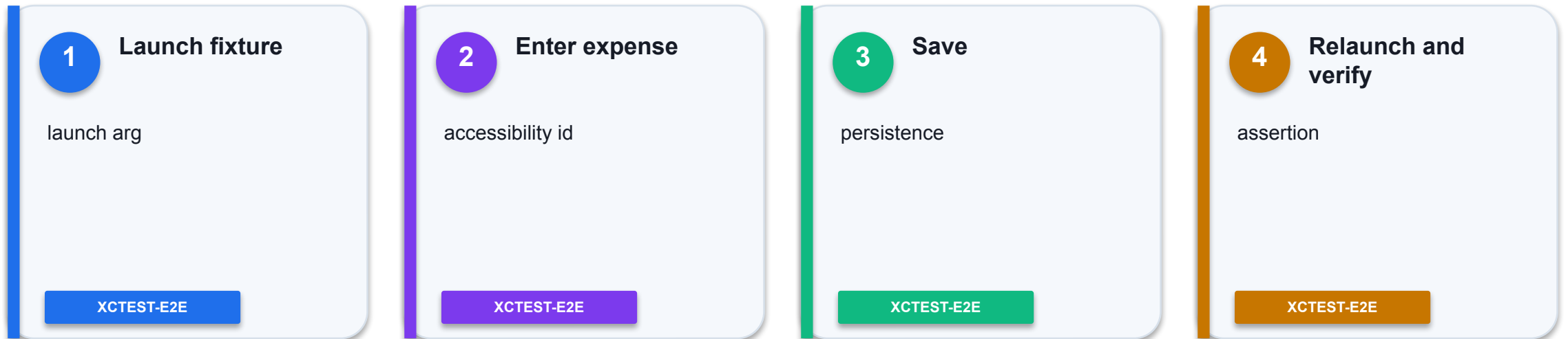
4

output

EVIDENCE METRIC

Adapter regression detected without UI automation

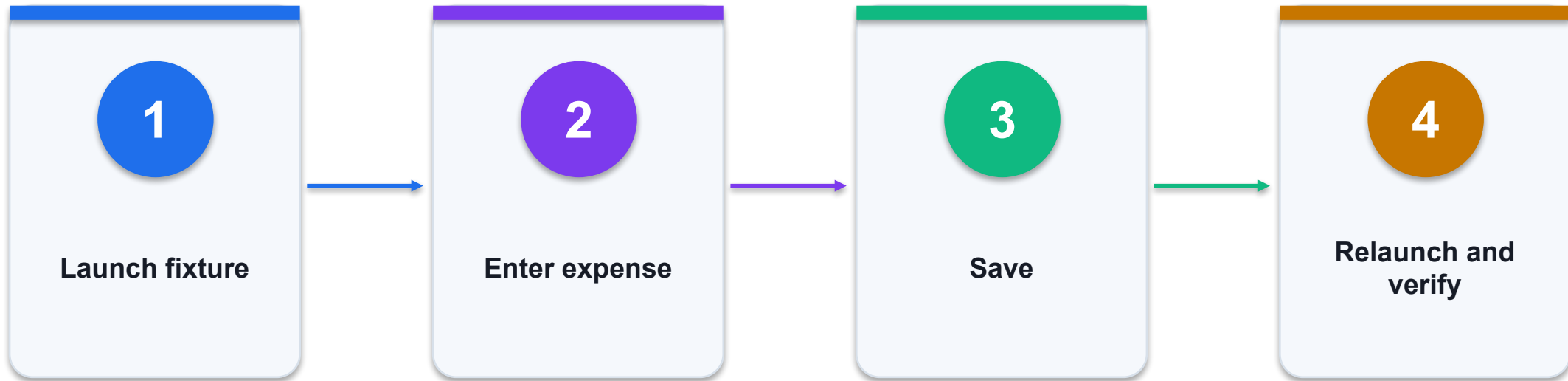
End-to-end tests prove user value



DESIGN JUDGEMENT

Choose a thin golden path that crosses every architectural boundary.

End-to-end tests prove user value · Mechanism



MECHANISM

Failure mode: A UI test checks labels but not the C++ decision or persistence result. **Control:** Choose a thin golden path that crosses every architectural boundary.

End-to-end tests prove user value · Technical Anchor

```
app.buttons["saveExpense"].tap();  
XCTAssertTrue(app.staticTexts["$12.50"].exists)
```

1

launch arg

2

accessibility id

3

persistence

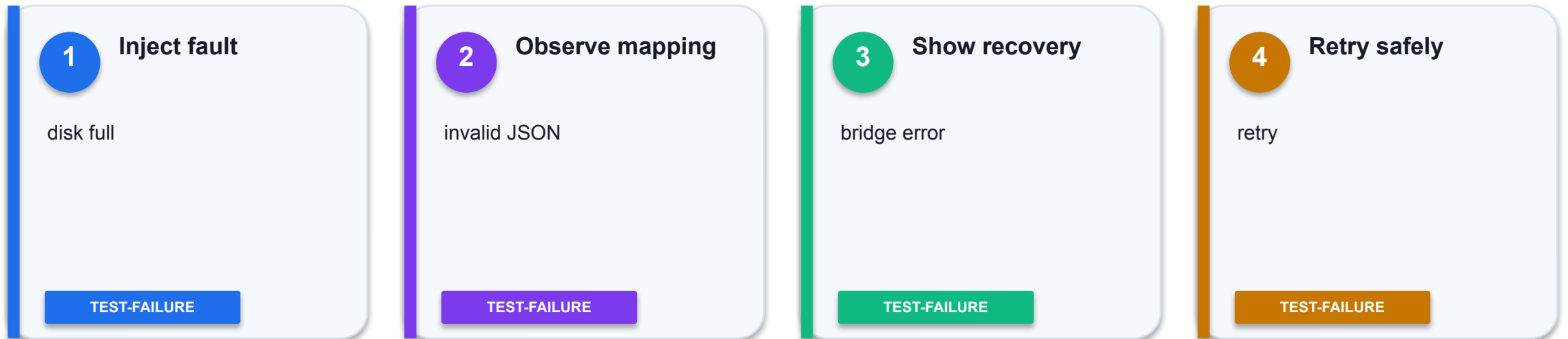
4

assertion

EVIDENCE METRIC

Critical journey passes from control to persisted core state

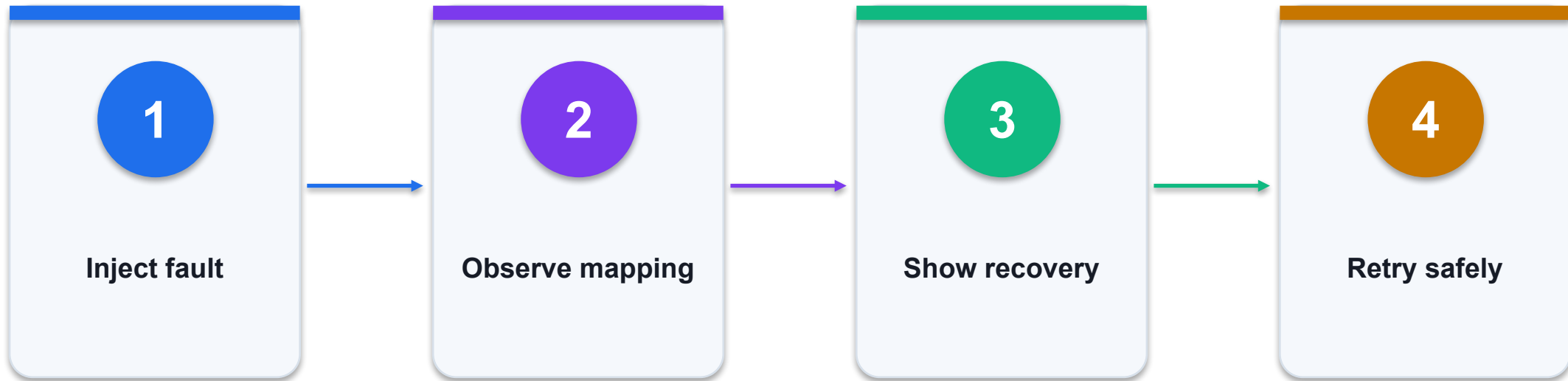
Failure injection tests recovery



DESIGN JUDGEMENT

Inject controlled faults and assert state transitions plus recovery action.

Failure injection tests recovery · Mechanism



MECHANISM

Failure mode: Errors are logged but the UI remains stuck in a loading state. **Control:** Inject controlled faults and assert state transitions plus recovery action.

Failure injection tests recovery · Technical Anchor

```
repo.failNextSave = true // deterministic test double
```

1

disk full

2

invalid JSON

3

bridge error

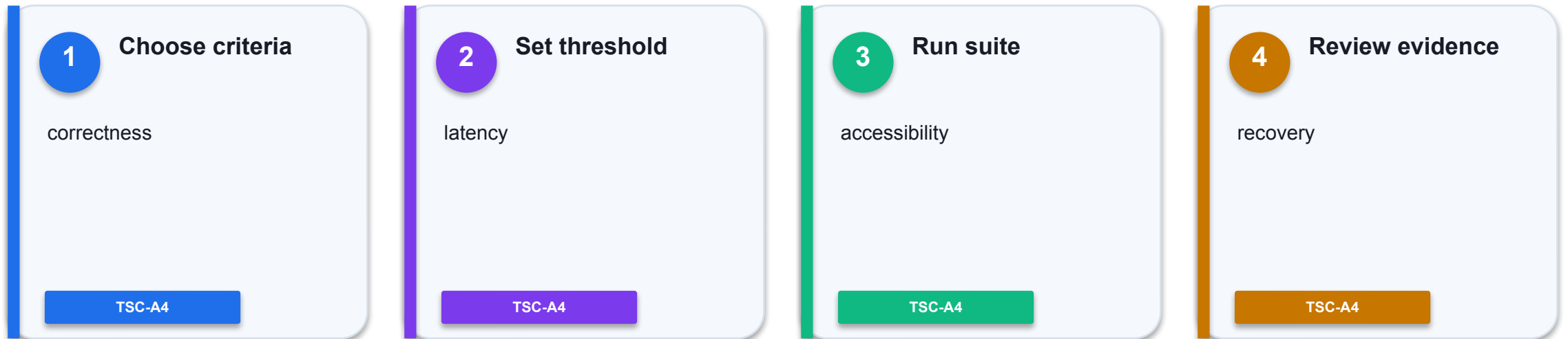
4

retry

EVIDENCE METRIC

Failure states preserve user input and offer a valid next action

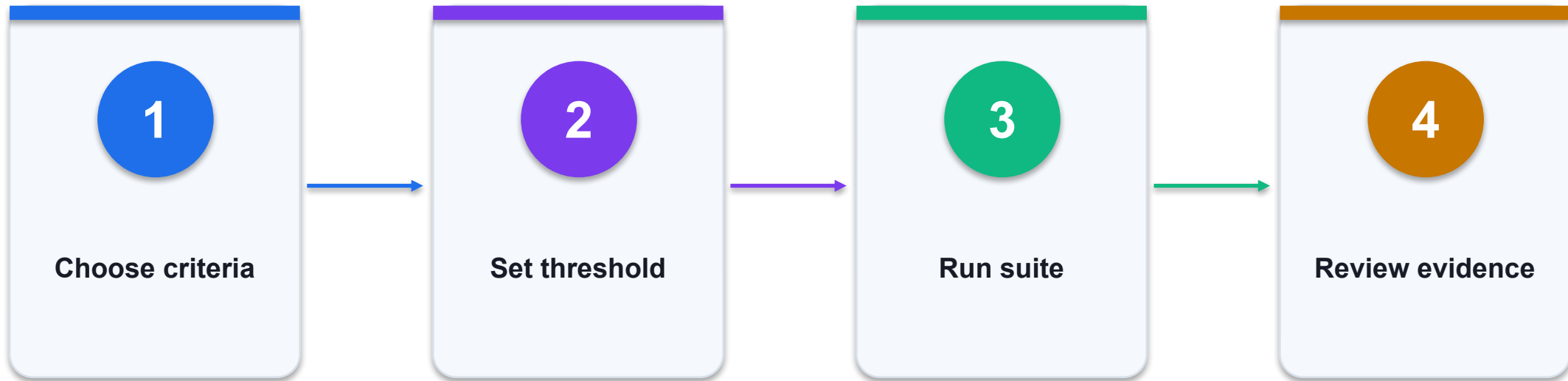
Functionality needs a scorecard



DESIGN JUDGEMENT

Combine domain, bridge, UI and quality indicators into one release decision.

Functionality needs a scorecard · Mechanism



MECHANISM

Failure mode: A build is declared ready because it compiles. **Control:** Combine domain, bridge, UI and quality indicators into one release decision.

Functionality needs a scorecard - Technical Anchor

```
ReleaseGate{.domain=1.0, .bridge=1.0, .ui=0.95, .crashes=0};
```

1

correctness

2

latency

3

accessibility

4

recovery

EVIDENCE METRIC

All release gates meet declared thresholds

Application Validation · Layered Evidence



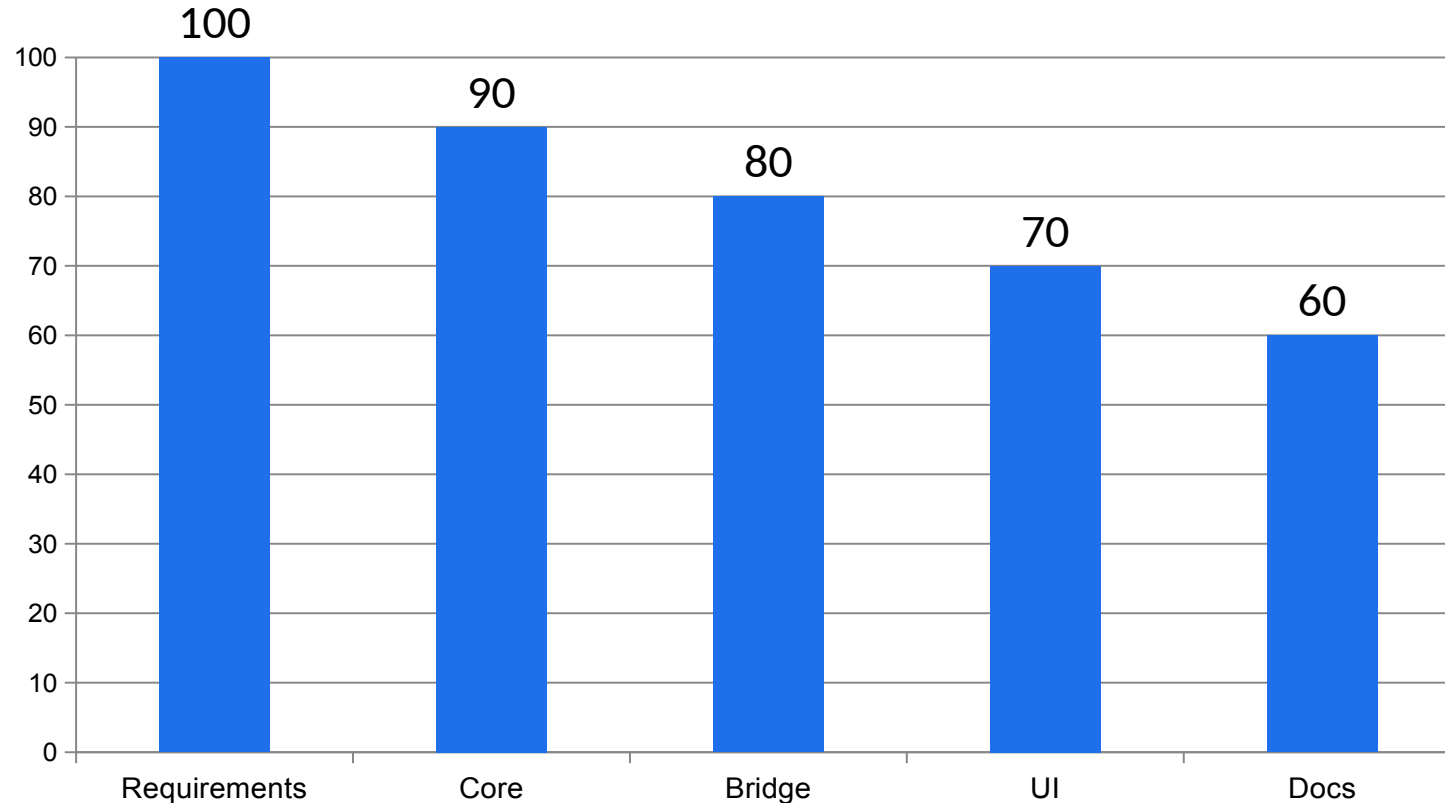
1 Domain rules first.

2 Bridge contract second.

3 Golden UI journey third.

4 Release decision last.

Topic 4 · Evidence Coverage by Layer



WHAT THE DATA SHOWS

Coverage should form a connected chain. A polished UI cannot compensate for untested invariants or an undocumented seam.

Editable chart · values are instructional thresholds, not production benchmarks

Activity 7: Persistence and Error Recovery

CHALLENGE

Persist domain snapshots safely and prove recovery from invalid data and write failure.

REGULATED MAPPING

K4, A4

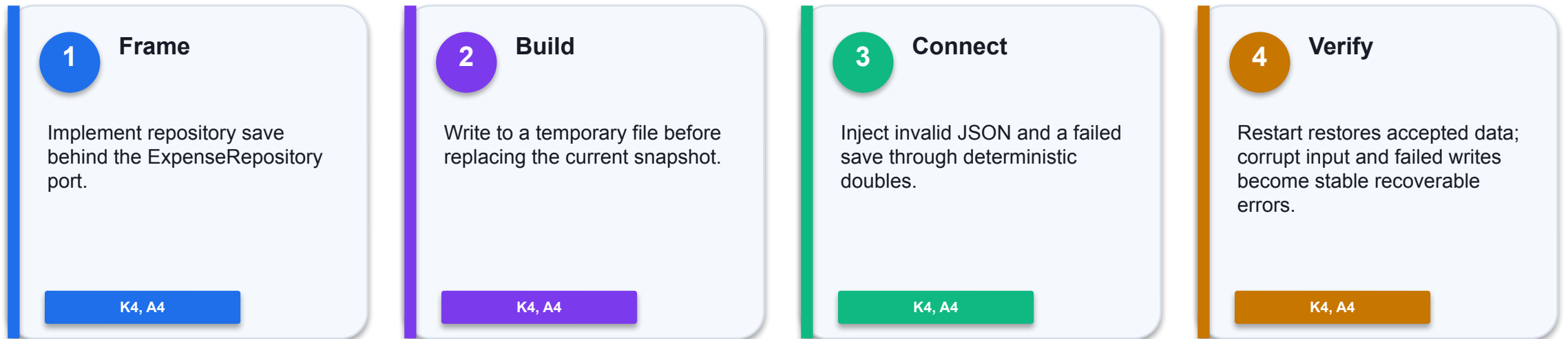
YOU WILL PRODUCE

Versioned JSON repository, migration fixture and recovery tests

C++ filesystem, JSON fixture, CTest

Detailed procedures: [Learner Guide and activities/07-persistence-error-recovery/README.md](#)

Activity 7 · Delivery Workflow



DESIGN JUDGEMENT

The slide communicates the workflow; the Learner Guide carries the detailed procedure and commands.

Activity 7 · Vibe Coding Generation Prompt

GENERATION PROMPT

Generate versioned atomic JSON persistence and v1-to-v2 migration with exact Int64 cents and stable IDs. Write a temporary file before replacement, keep tests inside a supplied temporary directory and map corrupt/read/write failures to typed recovery. Output complete repository, fixture, schema and test files. Verify migration, save/reload, failed replacement and retry with CTest.

PROTECTED FILE SCOPE

JsonExpenseRepository.hpp.cpp; snapshot-v1.json; persistence-schema.md; JsonExpenseRepositoryTests.cpp

EXPECTED GREEN

Clean CTest passes migration, round-trip and injected-failure tests; the previous snapshot remains readable after a failed replacement.

REVIEW / REPAIR

Use the paired review prompt in the Activity guide. Fix the first causal failure, preserve tests, then rerun focused and full checks.

Activity 7 · Evidence Contract

1

Artifact

Versioned JSON repository, migration fixture and recovery tests

K4, A4

2

Acceptance

Restart restores accepted data; corrupt input and failed writes become stable recoverable errors.

K4, A4

3

Failure signal

If tests touch the real home directory, inject a temporary path and fail the test on escape.

K4, A4

4

Cleanup

Delete only temporary test stores; retain migration fixtures.

K4, A4

DESIGN JUDGEMENT

Evidence is a retrievable artifact or test result, not a statement that the activity was completed.

Activity 7 · Verification Gate

PASS WHEN

Restart restores accepted data; corrupt input and failed writes become stable recoverable errors.

IF IT FAILS

If tests touch the real home directory, inject a temporary path and fail the test on escape.

LEAVE THE WORKSPACE

Delete only temporary test stores; retain migration fixtures.

Activity 8: Interoperability Validation Scorecard

CHALLENGE

Assess the C++/Objective-C++/SwiftUI system with domain, bridge and end-to-end evidence.

REGULATED MAPPING

K4, A4

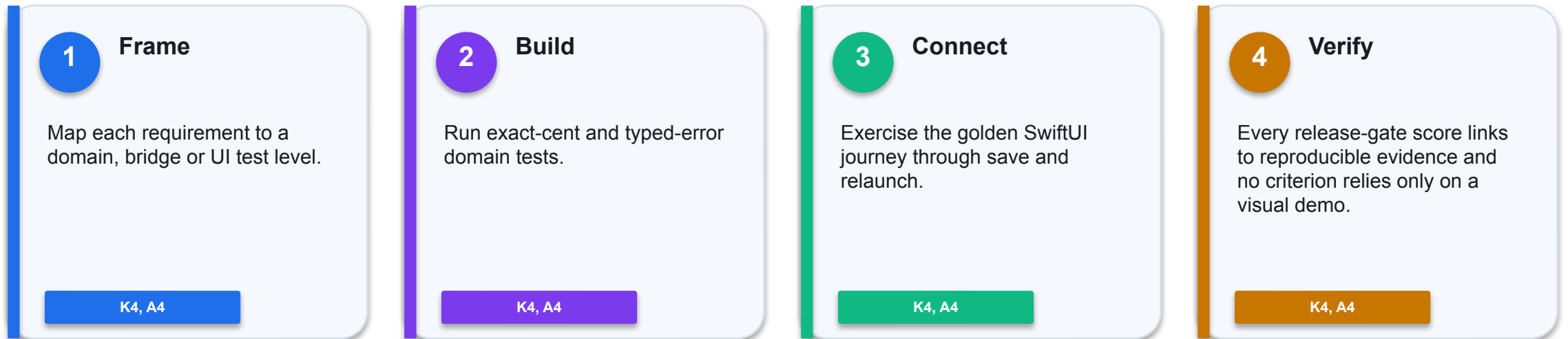
YOU WILL PRODUCE

Test matrix, execution evidence and release scorecard

CTest, XCTest, XCUITest, Instruments

Detailed procedures: [Learner Guide and activities/08-interop-validation-scorecard/README.md](#)

Activity 8 · Delivery Workflow



DESIGN JUDGEMENT

The slide communicates the workflow; the Learner Guide carries the detailed procedure and commands.

Activity 8 · Vibe Coding Generation Prompt

GENERATION PROMPT

Generate the layered validation matrix. Use CTest for portable code, Swift Testing for Swift unit/integration behaviour and XCTest with XCUIAutomation for the visible journey. Map REQ-001..REQ-005 to exact test names, commands, thresholds and evidence. Reject screenshot-only proof or PASS with an empty path. Output complete matrix, scorecard and test files.

PROTECTED FILE SCOPE

test-matrix.csv; scorecard.json; Swift Testing test files; BudgetBuddyUITests.swift; evidence logs

EXPECTED GREEN

Every requirement has domain, bridge/integration, or UI evidence as appropriate; all mandatory thresholds resolve to reproducible passing commands.

REVIEW / REPAIR

Use the paired review prompt in the Activity guide. Fix the first causal failure, preserve tests, then rerun focused and full checks.

Activity 8 · Evidence Contract

1 Artifact

Test matrix, execution evidence and release scorecard

K4, A4

2 Acceptance

Every release-gate score links to reproducible evidence and no criterion relies only on a visual demo.

K4, A4

3 Failure signal

If a UI failure is ambiguous, reproduce it one layer lower before changing the architecture.

K4, A4

4 Cleanup

Remove disposable run artifacts; retain the scorecard and final evidence links.

K4, A4

DESIGN JUDGEMENT

Evidence is a retrievable artifact or test result, not a statement that the activity was completed.

Activity 8 · Verification Gate

PASS WHEN

Every release-gate score links to reproducible evidence and no criterion relies only on a visual demo.

IF IT FAILS

If a UI failure is ambiguous, reproduce it one layer lower before changing the architecture.

LEAVE THE WORKSPACE

Remove disposable run artifacts; retain the scorecard and final evidence links.

Topic 4 · Review

1 Explain

LO4: Examine the interoperability and functionality of C++ software components.

LO4 · K4 · A4

2 Demonstrate

Show the Activity evidence that operationalises the outcome.

LO4 · K4 · A4

3 Diagnose

Name the failure signal and the layer where it originates.

LO4 · K4 · A4

4 Trace

Connect LO4 · K4 · A4 to the assessment and evidence pack.

LO4 · K4 · A4

DESIGN JUDGEMENT

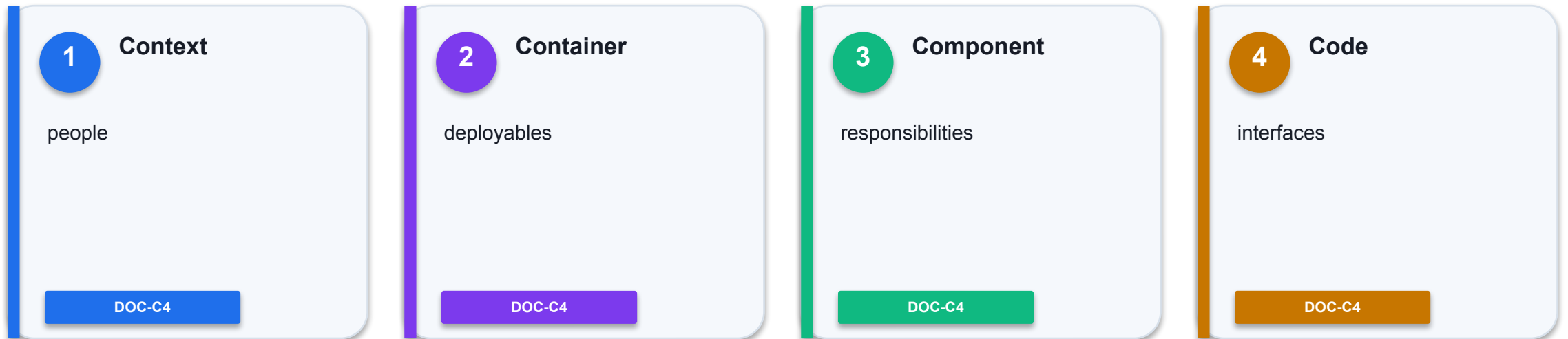
Do not advance on confidence alone; point to the artifact or test result that supports the answer.

TOPIC 5 · LO5 · K5 · A5

Architecture Evidence, Documentation and Deployment

LO5: Generate C++ design documentation aligned with user specifications.

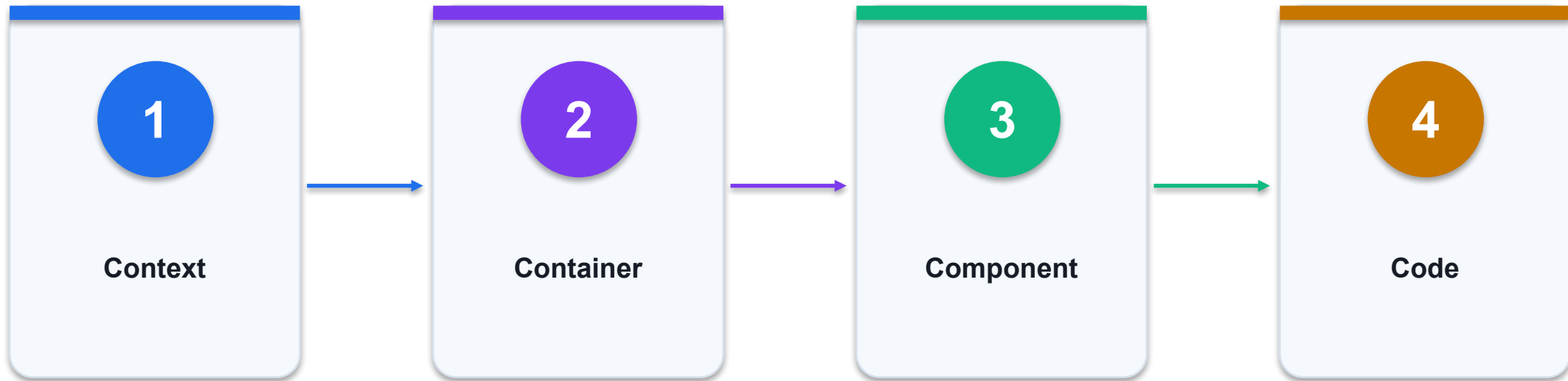
C4 views explain different questions



DESIGN JUDGEMENT

Use layered diagrams and state the audience and decision for each.

C4 views explain different questions · Mechanism



MECHANISM

Failure mode: A single dense diagram mixes business actors with classes. **Control:** Use layered diagrams and state the audience and decision for each.

C4 views explain different questions · Technical Anchor

```
Context -> iOS App -> SwiftUI | Objective-C++ | C++  
Core
```

1

people

2

deployables

3

responsibilities

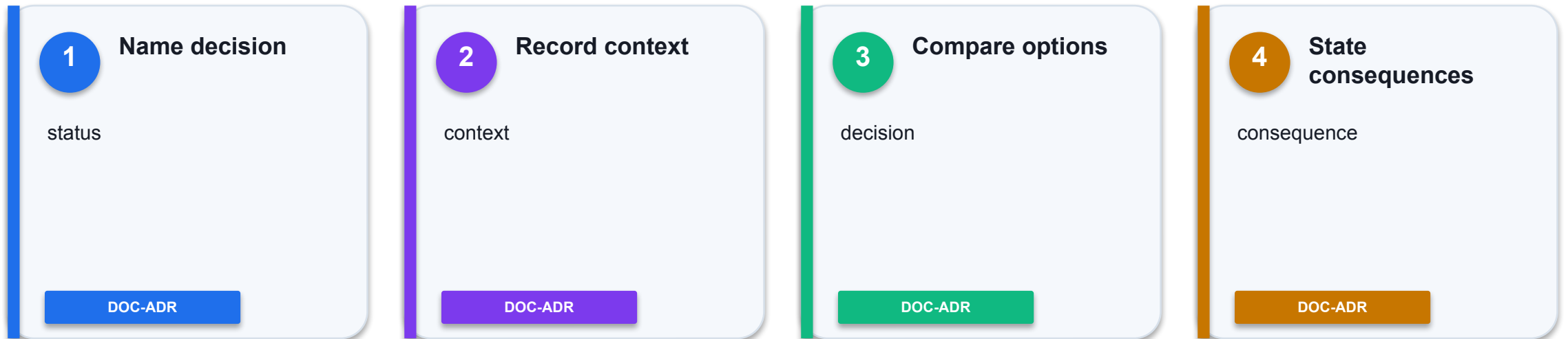
4

interfaces

EVIDENCE METRIC

Stakeholder question answered at correct zoom

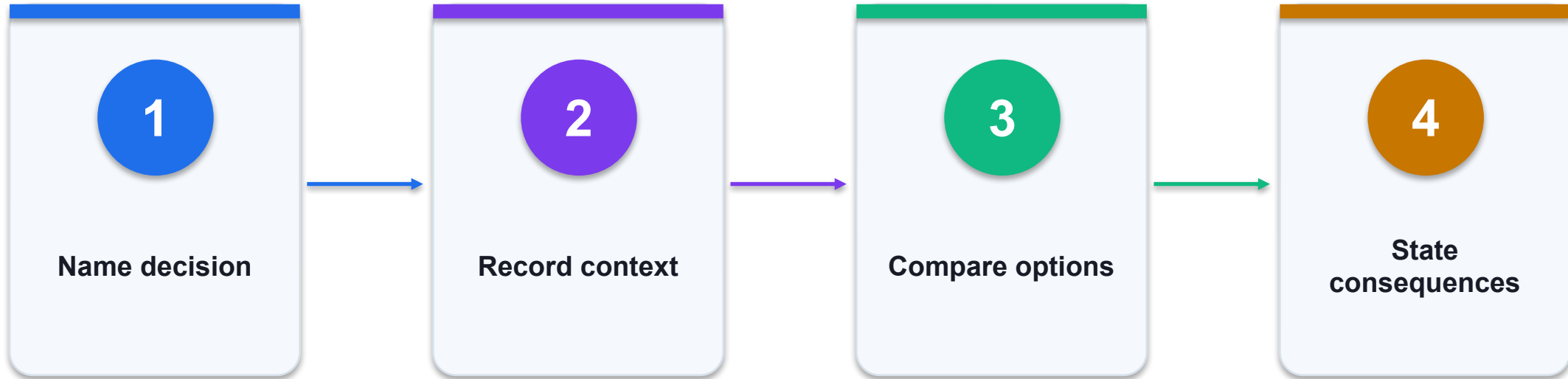
ADRs preserve design reasoning



DESIGN JUDGEMENT

Write a short ADR for boundaries, persistence and AI usage.

ADRs preserve design reasoning · Mechanism



MECHANISM

Failure mode: Documentation states what exists but not why alternatives were rejected. **Control:** Write a short ADR for boundaries, persistence and AI usage.

ADRs preserve design reasoning · Technical Anchor

```
# ADR-002: Use Objective-C++ as the sole C++  
translation boundary
```

1

status

2

context

3

decision

4

consequence

EVIDENCE METRIC

Material choices have retrievable rationale

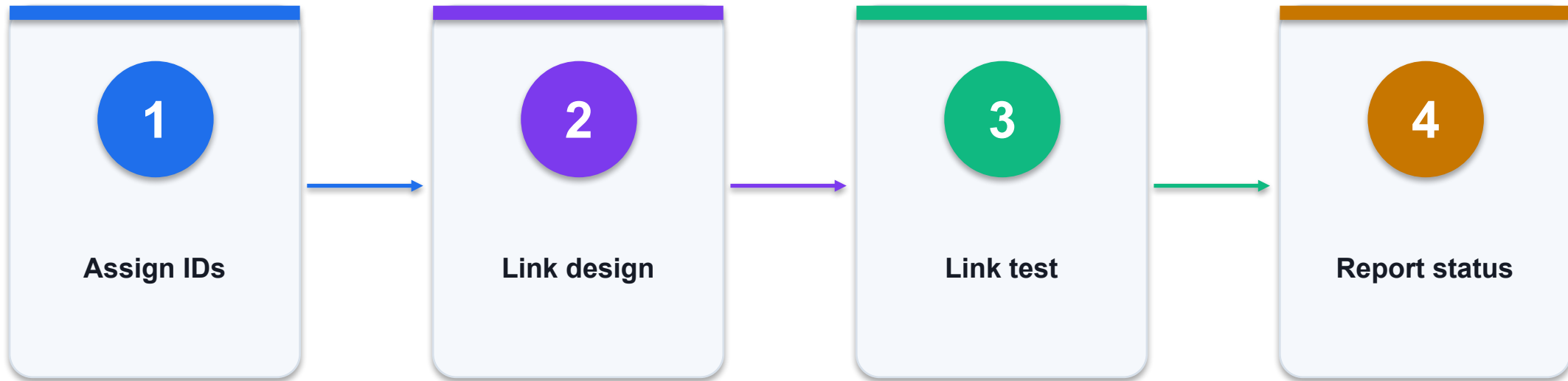
Traceability closes the specification loop



DESIGN JUDGEMENT

Maintain a bidirectional matrix from requirement to implementation and test.

Traceability closes the specification loop · Mechanism



MECHANISM

Failure mode: A feature is marked done without proving the approved requirement. **Control:** Maintain a bidirectional matrix from requirement to implementation and test.

Traceability closes the specification loop · Technical Anchor

```
REQ-004 | BudgetPolicy |  
BudgetPolicyTests/RejectsNegative | PASS
```

1

REQ

2

component

3

test

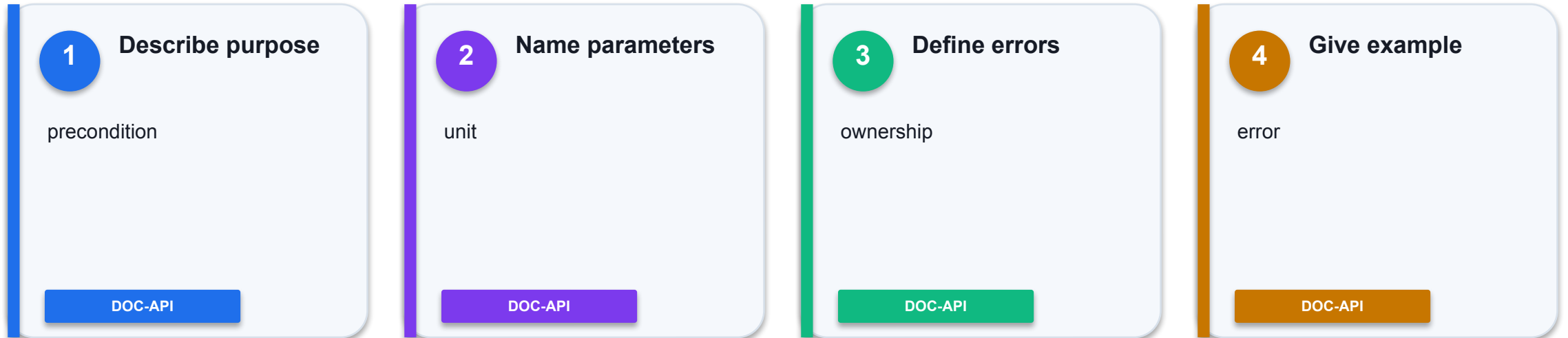
4

evidence

EVIDENCE METRIC

Requirements with design and passing evidence

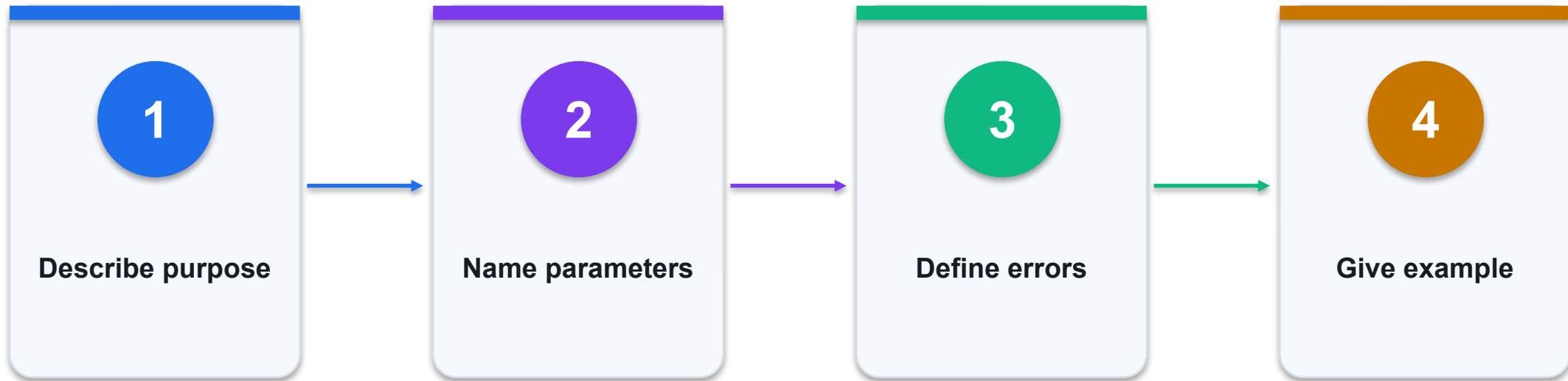
API documentation states contracts



DESIGN JUDGEMENT

Document invariants, units, lifetime and failure semantics.

API documentation states contracts · Mechanism



MECHANISM

Failure mode: Generated comments repeat the method name and omit units or ownership. **Control:** Document invariants, units, lifetime and failure semantics.

API documentation states contracts · Technical Anchor

```
/// Evaluates a draft. Amount is cents; returns typed  
error; no mutation.
```

1

precondition

2

unit

3

ownership

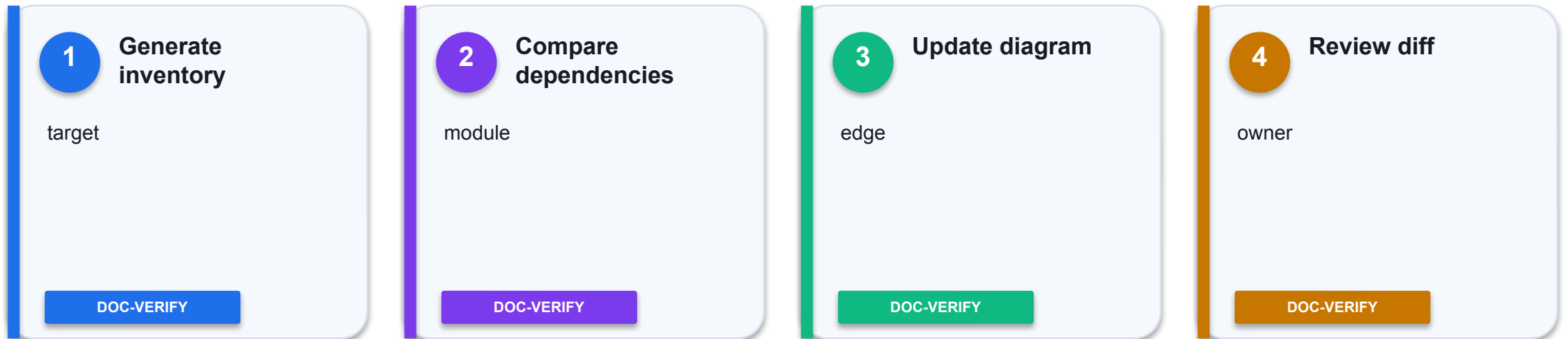
4

error

EVIDENCE METRIC

Consumers implement without reading source

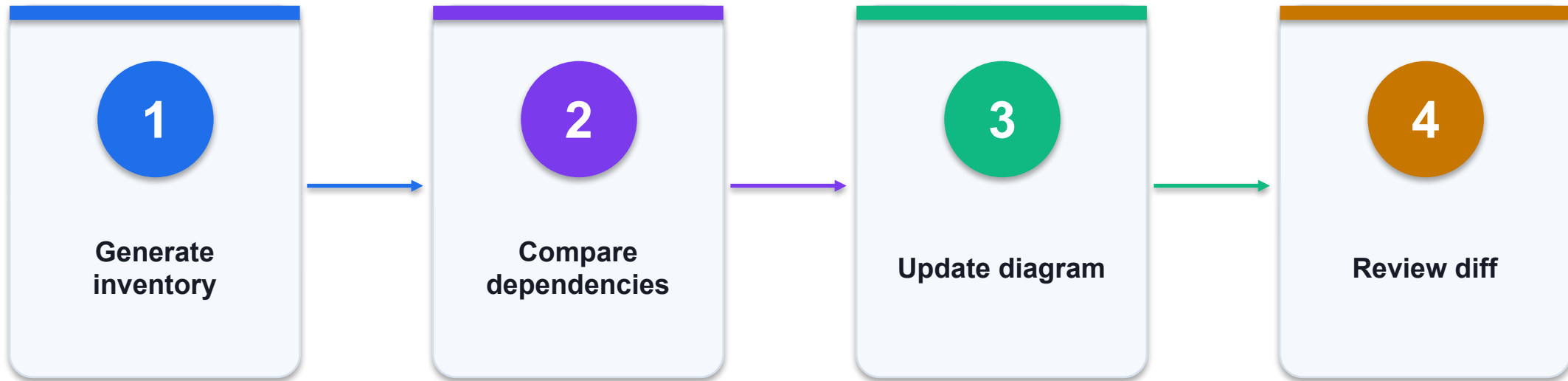
Architecture diagrams must match code



DESIGN JUDGEMENT

Validate diagrams against target membership and imports before release.

Architecture diagrams must match code · Mechanism



MECHANISM

Failure mode: A polished diagram describes an architecture the build no longer uses. **Control:** Validate diagrams against target membership and imports before release.

Architecture diagrams must match code · Technical Anchor

```
SwiftUI -> BudgetBridge -> DomainCore; no reverse  
dependency
```

1

target

2

module

3

edge

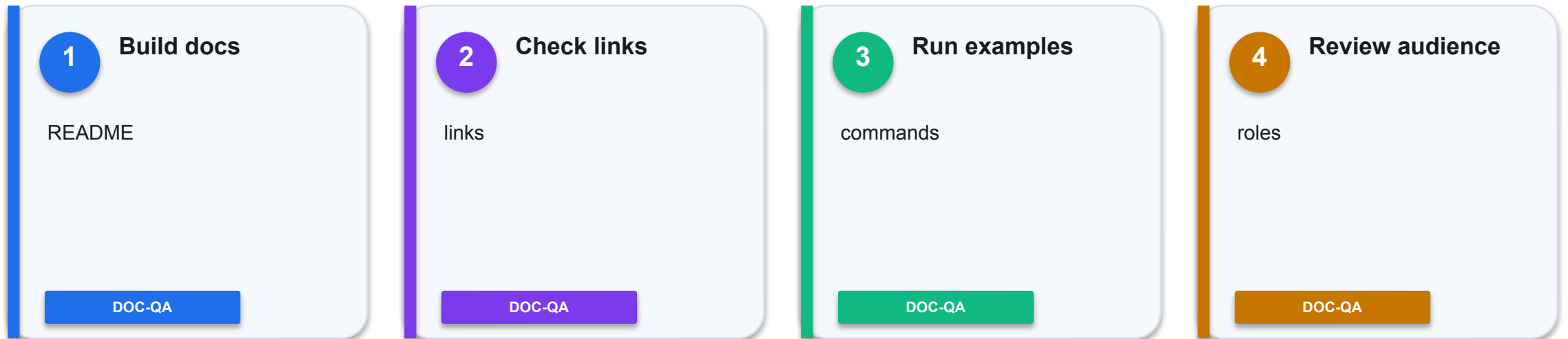
4

owner

EVIDENCE METRIC

Zero undocumented or impossible dependency arrows

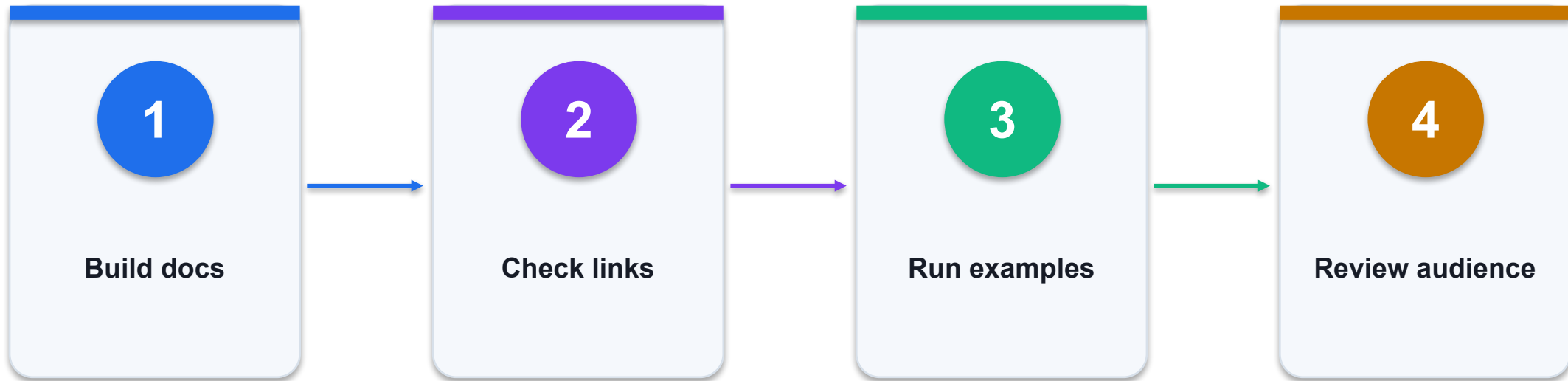
Documentation is a tested deliverable



DESIGN JUDGEMENT

Exercise documentation in a clean environment and capture the transcript.

Documentation is a tested deliverable · Mechanism



MECHANISM

Failure mode: Setup steps depend on an unstated local path or secret. **Control:** Exercise documentation in a clean environment and capture the transcript.

Documentation is a tested deliverable · Technical Anchor

```
cmake -S DomainCore -B build && cmake --build build &&  
ctest --test-dir build
```

1

README

2

links

3

commands

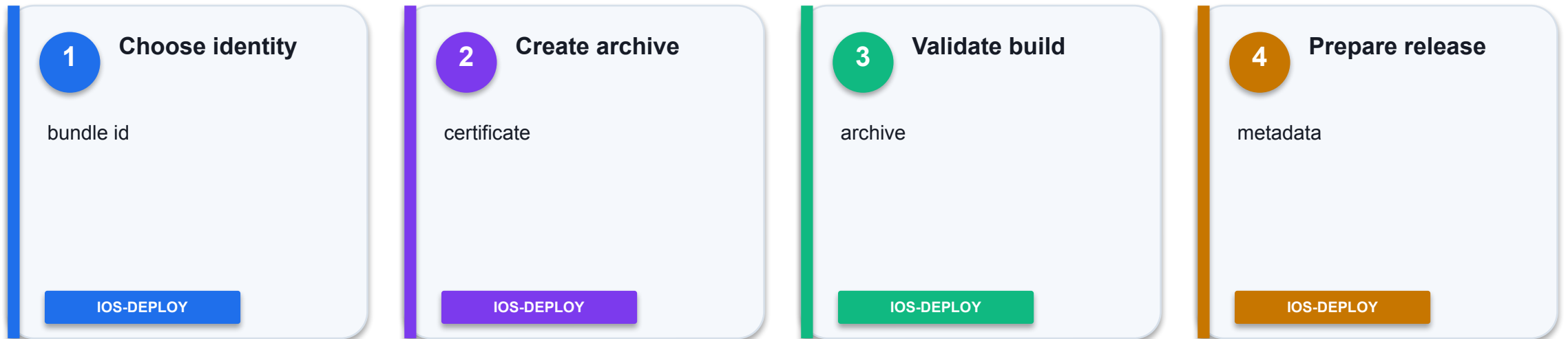
4

roles

EVIDENCE METRIC

New developer reaches a passing test from clean checkout

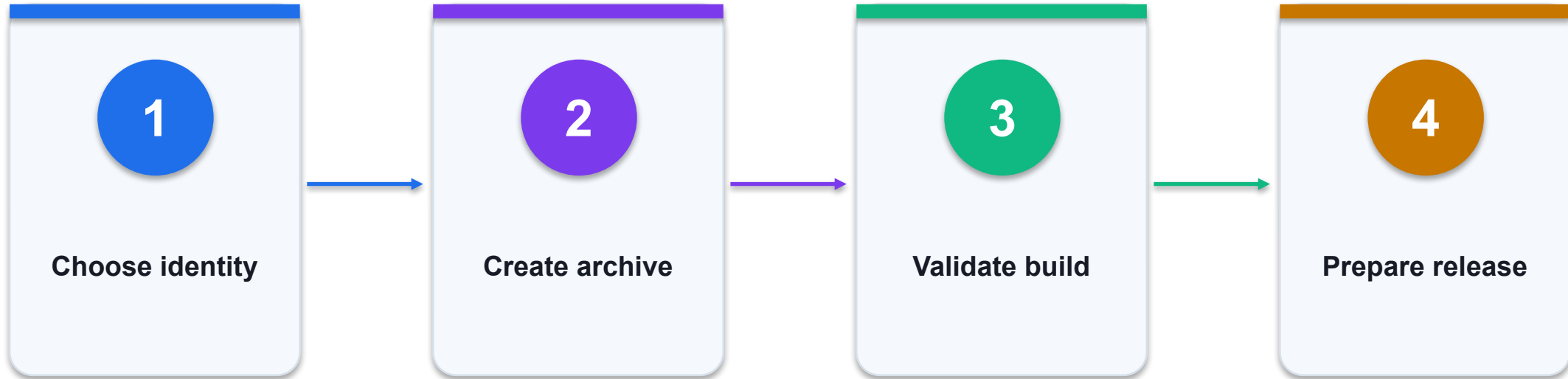
Deployment separates signing from design



DESIGN JUDGEMENT

Treat signing, archive validation and store submission as separate evidence gates.

Deployment separates signing from design · Mechanism



MECHANISM

Failure mode: A local simulator run is described as App Store readiness. **Control:** Treat signing, archive validation and store submission as separate evidence gates.

Deployment separates signing from design · Technical Anchor

```
xcodebuild -scheme BudgetBridgeApp -configuration  
Release archive
```

1

bundle id

2

certificate

3

archive

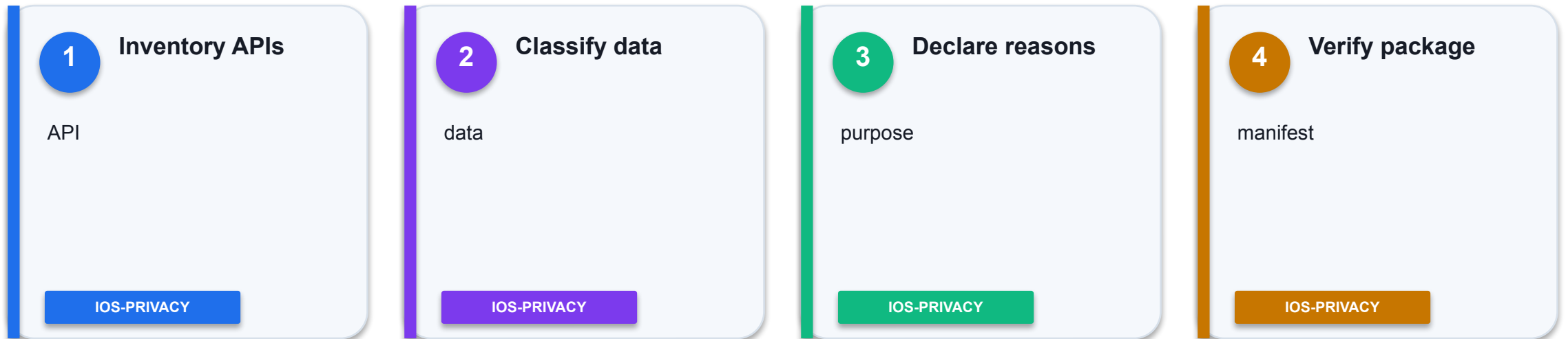
4

metadata

EVIDENCE METRIC

Validated archive with reproducible design evidence

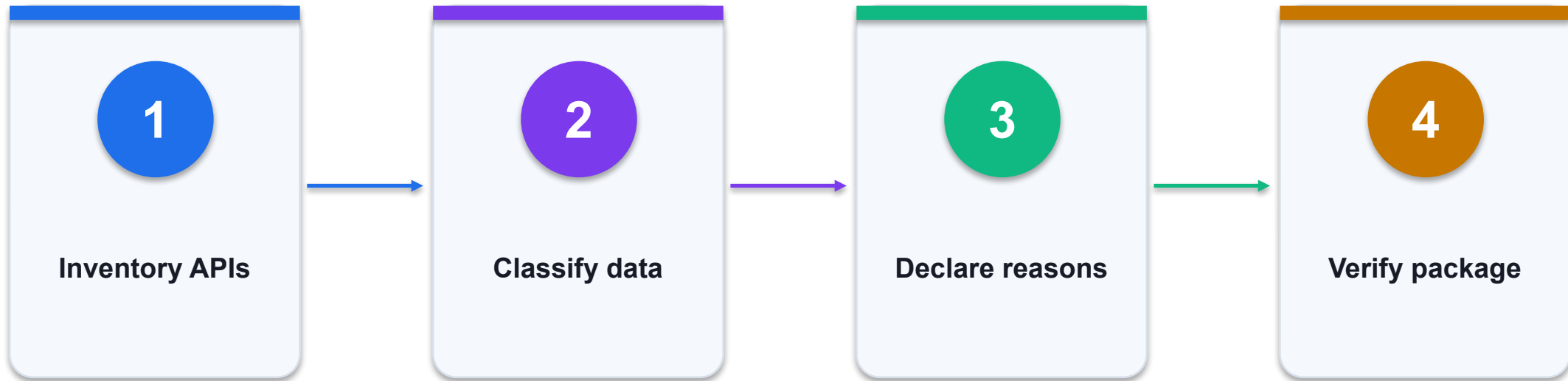
Privacy manifests document data use



DESIGN JUDGEMENT

Re-run privacy inventory whenever dependencies or telemetry change.

Privacy manifests document data use · Mechanism



MECHANISM

Failure mode: An AI-added SDK expands collection without documentation review. **Control:** Re-run privacy inventory whenever dependencies or telemetry change.

Privacy manifests document data use · Technical Anchor

```
PrivacyInfo.xcprivacy // reviewed against linked frameworks
```

1

API

2

data

3

purpose

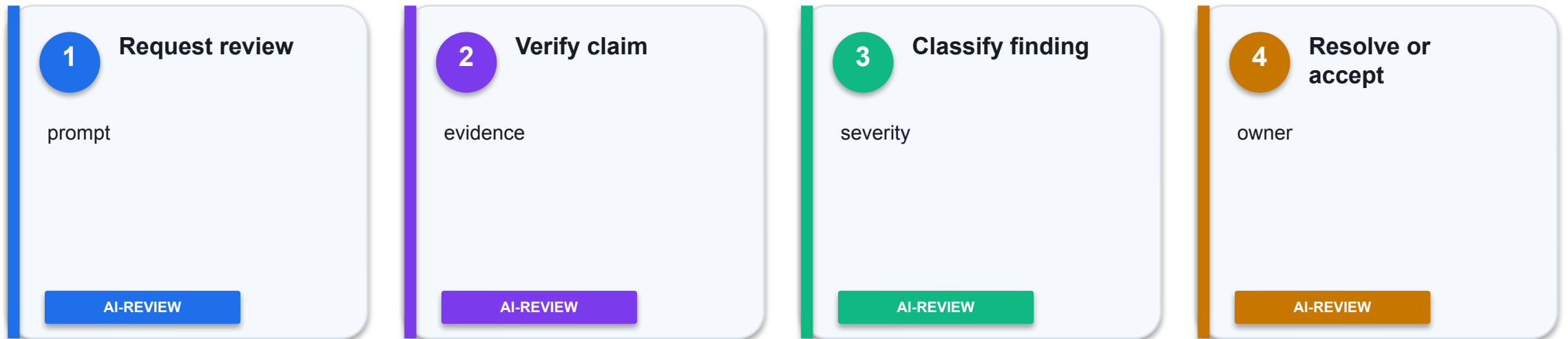
4

manifest

EVIDENCE METRIC

Declared use matches code and third-party packages

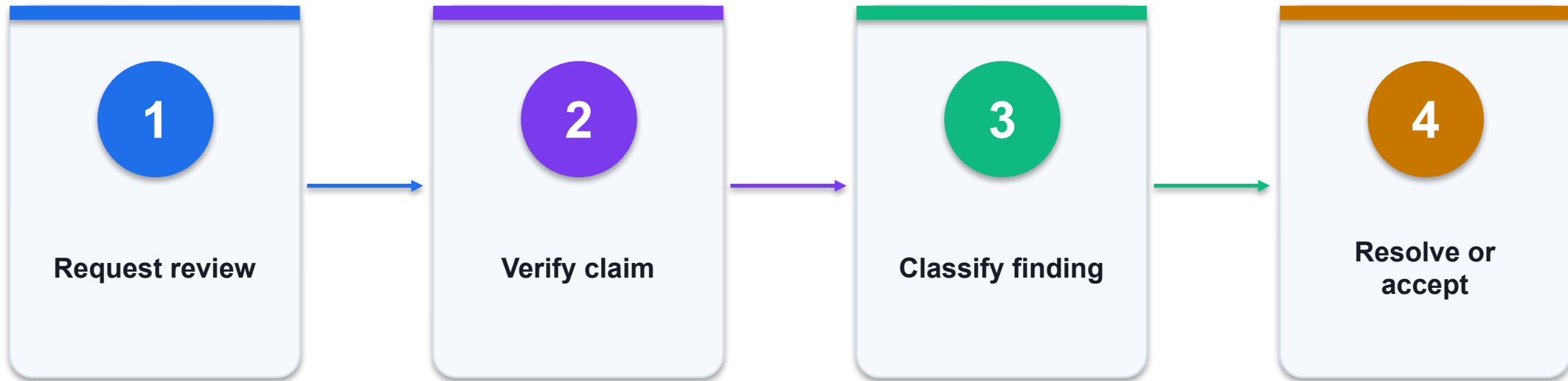
AI review needs accountable evidence



DESIGN JUDGEMENT

Require code locus and a repeatable test for material findings.

AI review needs accountable evidence · Mechanism



MECHANISM

Failure mode: AI review comments are copied into approval without reproduction. **Control:** Require code locus and a repeatable test for material findings.

AI review needs accountable evidence · Technical Anchor

```
Finding: cents overflow. Evidence: boundary test with  
Int.max. Owner: developer.
```

1

prompt

2

evidence

3

severity

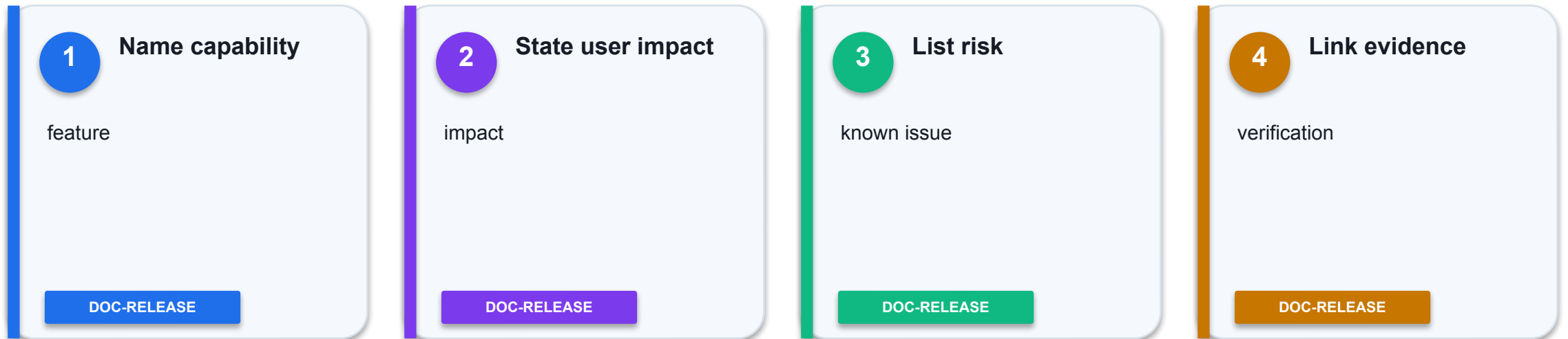
4

owner

EVIDENCE METRIC

Accepted findings have owner and rationale

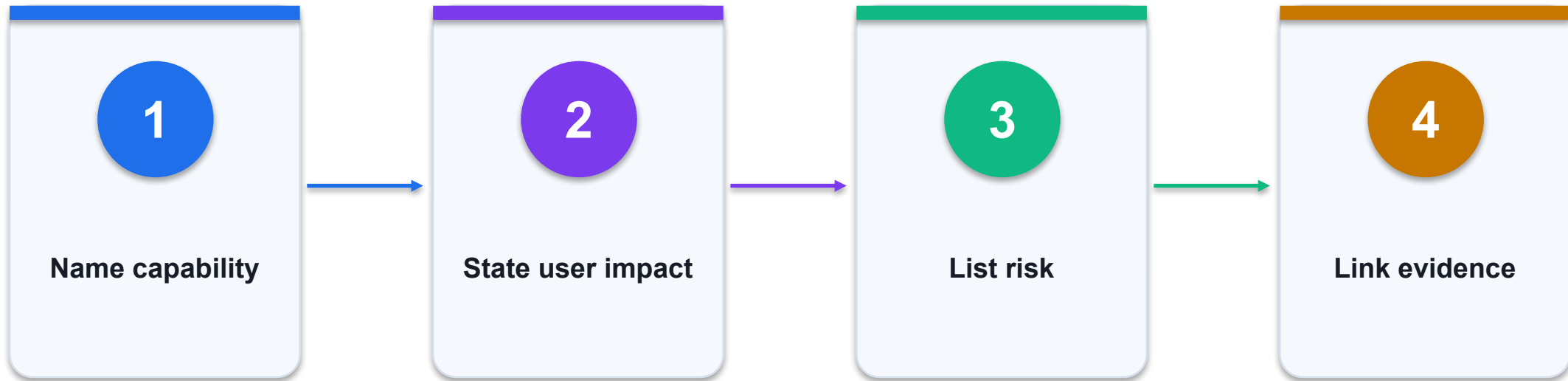
Release notes map value to change



DESIGN JUDGEMENT

Write outcome-oriented notes and disclose known limitations.

Release notes map value to change · Mechanism



MECHANISM

Failure mode: Release notes list commits but not observable user outcomes. **Control:** Write outcome-oriented notes and disclose known limitations.

Release notes map value to change · Technical Anchor

```
Added exact-cent budgeting; validated restart  
persistence and VoiceOver flow.
```

1

feature

2

impact

3

known issue

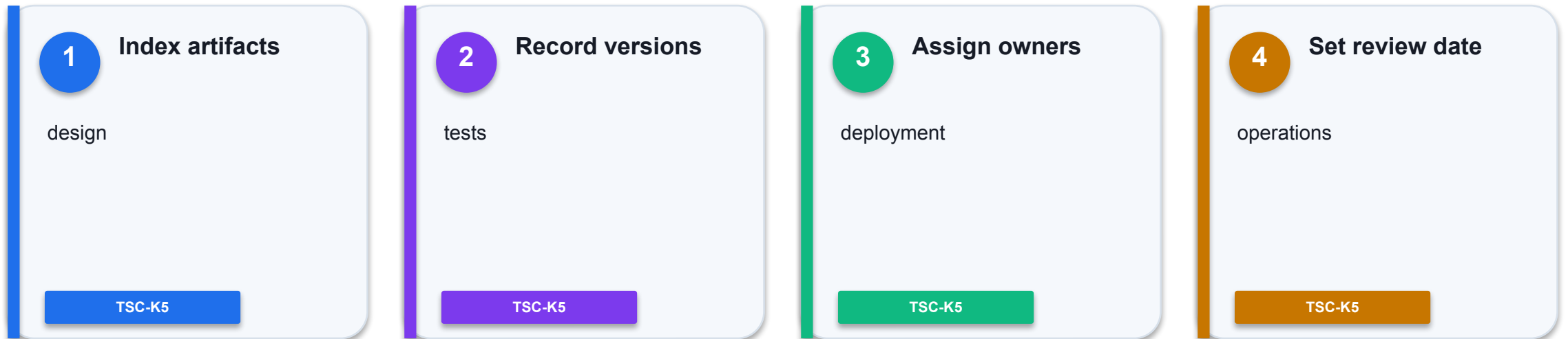
4

verification

EVIDENCE METRIC

Every release claim points to a checked artifact

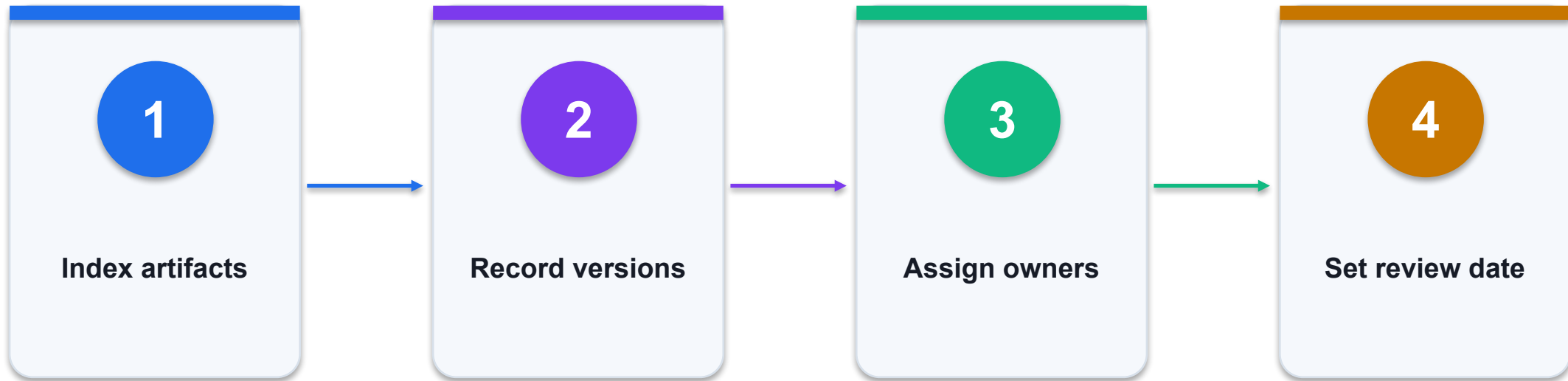
The evidence pack supports handover



DESIGN JUDGEMENT

Package requirements, design, tests, risks and deployment evidence with provenance.

The evidence pack supports handover · Mechanism



MECHANISM

Failure mode: Evidence is scattered across local screenshots and chat history. **Control:** Package requirements, design, tests, risks and deployment evidence with provenance.

The evidence pack supports handover · Technical Anchor

```
EvidencePack v1.2: requirements, ADRs, tests,  
validation, release checklist
```

1

design

2

tests

3

deployment

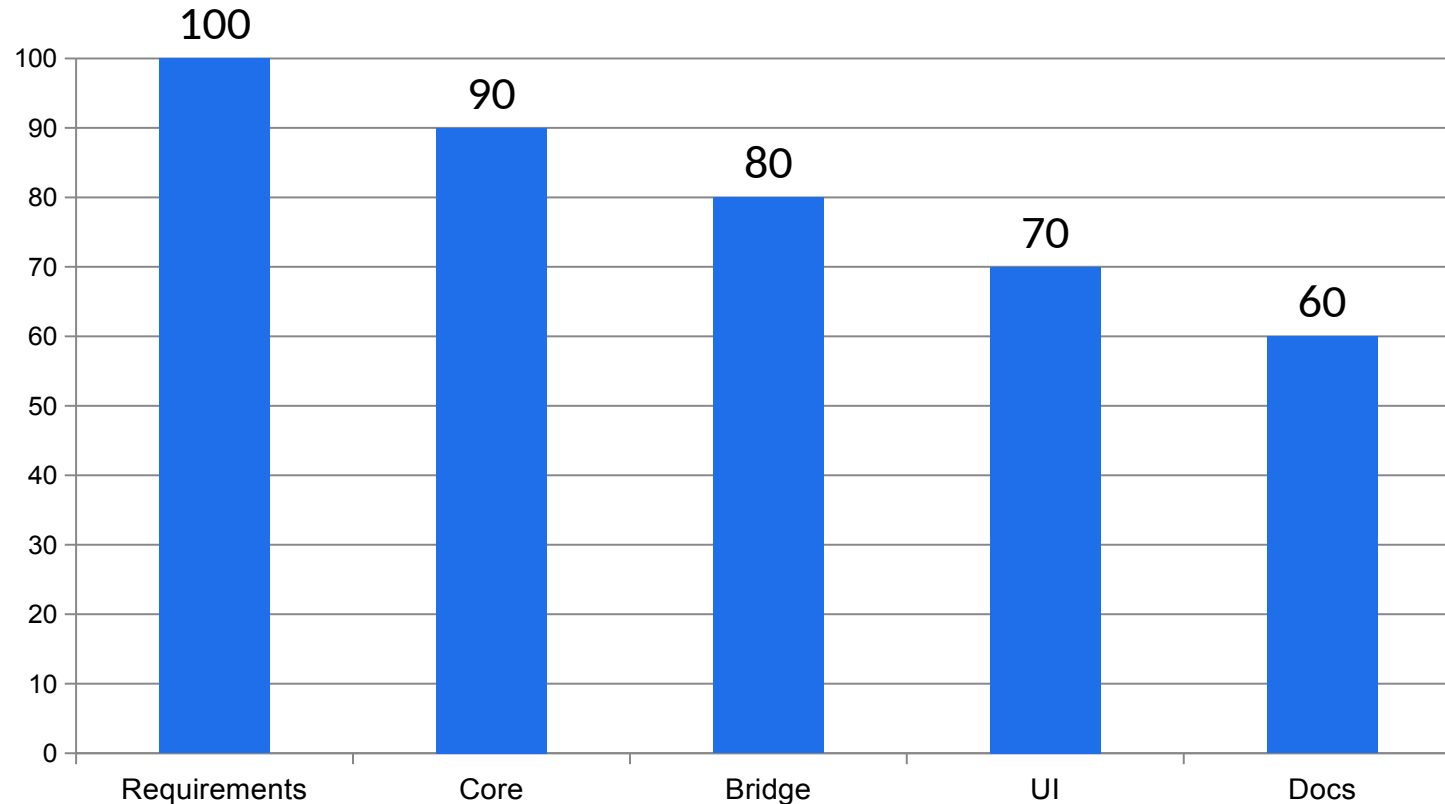
4

operations

EVIDENCE METRIC

A reviewer can reproduce the release decision

Topic 5 · Evidence Coverage by Layer



WHAT THE DATA SHOWS

Coverage should form a connected chain. A polished UI cannot compensate for untested invariants or an undocumented seam.

Editable chart · values are instructional thresholds, not production benchmarks

Activity 9: Architecture and Traceability Documentation

CHALLENGE

Create design documentation that maps user specifications to architecture, interfaces and tests.

REGULATED MAPPING

K5, A5

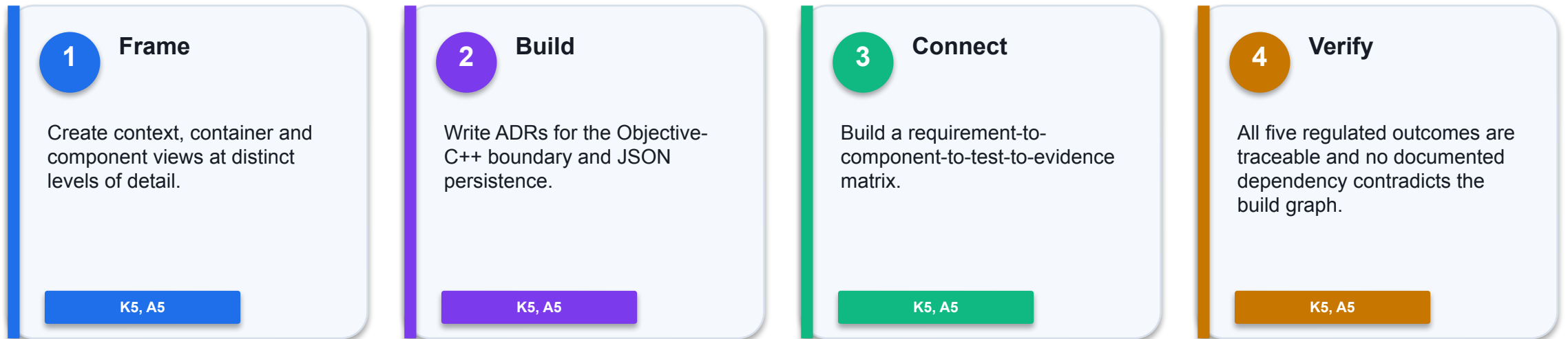
YOU WILL PRODUCE

C4 diagrams, two ADRs, API contract and traceability matrix

Markdown, Mermaid or editable shapes, Doxygen-style comments

Detailed procedures: [Learner Guide and activities/09-architecture-traceability-docs/README.md](#)

Activity 9 · Delivery Workflow



DESIGN JUDGEMENT

The slide communicates the workflow; the Learner Guide carries the detailed procedure and commands.

Activity 9 · Vibe Coding Generation Prompt

GENERATION PROMPT

Generate C4 views, ADR-001 bridge boundary, ADR-002 persistence, bridge API contract and traceability.csv from the verified build graph. Document units, ownership, nullability and typed errors. Do not invent a module, test or evidence path. Output complete documents. Reconcile every dependency arrow and require implementation, test, evidence and status for REQ-001..REQ-005.

PROTECTED FILE SCOPE

architecture.md; ADR-001-objective-cpp-boundary.md; ADR-002-json-persistence.md; bridge-contract.md; traceability.csv; README.md

EXPECTED GREEN

All REQ-001..REQ-005 rows have real implementation, test, evidence, and status; architecture imports match the inward dependency rule.

REVIEW / REPAIR

Use the paired review prompt in the Activity guide. Fix the first causal failure, preserve tests, then rerun focused and full checks.

Activity 9 · Evidence Contract

1 Artifact

C4 diagrams, two ADRs, API contract and traceability matrix

K5, A5

2 Acceptance

All five regulated outcomes are traceable and no documented dependency contradicts the build graph.

K5, A5

3 Failure signal

If the diagram is crowded, split by stakeholder question instead of shrinking labels.

K5, A5

4 Cleanup

Retain source diagrams and Markdown; remove obsolete exported drafts.

K5, A5

DESIGN JUDGEMENT

Evidence is a retrievable artifact or test result, not a statement that the activity was completed.

Activity 9 · Verification Gate

PASS WHEN

All five regulated outcomes are traceable and no documented dependency contradicts the build graph.

IF IT FAILS

If the diagram is crowded, split by stakeholder question instead of shrinking labels.

LEAVE THE WORKSPACE

Retain source diagrams and Markdown; remove obsolete exported drafts.

Activity 10: Capstone: BudgetBuddy Release Evidence

CHALLENGE

Integrate the portable C++ core, Objective-C++ adapter and SwiftUI shell into a verified release evidence pack.

REGULATED MAPPING

**A1, A2, A3, A4,
A5**

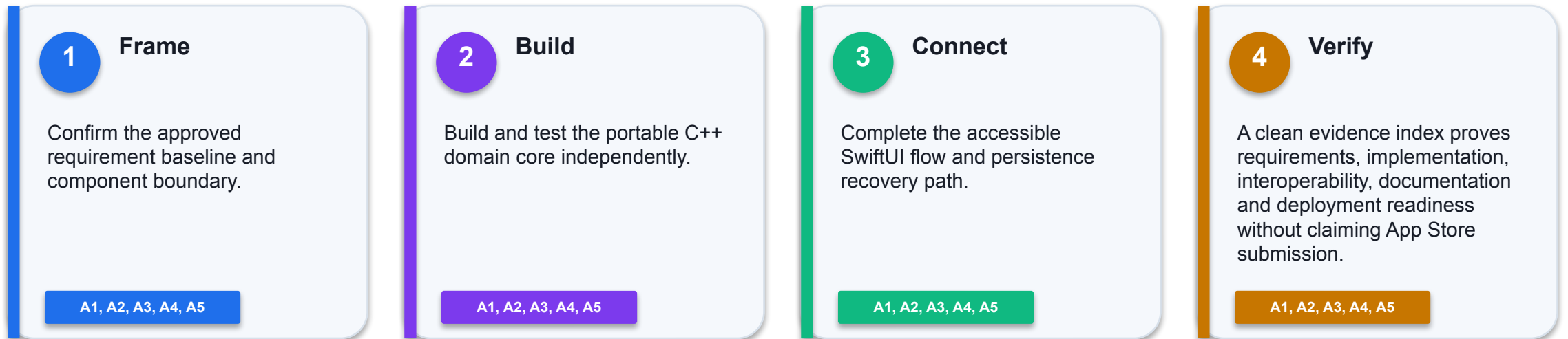
YOU WILL PRODUCE

Working app, complete tests, architecture pack and deployment checklist

C++20, CMake, Xcode, SwiftUI, XCTest,
documentation templates

Detailed procedures: [Learner Guide and activities/10-capstone-release-evidence/README.md](#)

Activity 10 · Delivery Workflow



DESIGN JUDGEMENT

The slide communicates the workflow; the Learner Guide carries the detailed procedure and commands.

Activity 10 · Final Project Audit Prompt

GENERATION PROMPT

Verify BudgetBuddy, FocusCards and HabitPulse from project.yml: build, Swift Testing, XCUIAutomation, launch and inspect each AppIcon. Prove BudgetBuddy exact cents through Objective-C++. Prepare ASC identity, archive/export/upload, metadata, screenshots, privacy, age, availability, attach-build, submit and readback gates. Never expose secrets or claim external execution; leave trainer-only credentialled gates HOLD.

PROTECTED FILE SCOPE

verify-local.sh; CI workflow; privacy-security-review.md; evidence-index.csv; release-scorecard.json; release-notes.md; deployment-position.md; file-inventory.txt; final-audit.md

EXPECTED GREEN

All three generated Xcode projects build, test and launch on a resolved iPhone Simulator; BudgetBuddy proves exact cents through Objective-C++; icons and privacy manifests are present; local evidence is complete; every unexecuted archive/upload/metadata/privacy/age/availability/attach/submit/readback gate remains HOLD.

REVIEW / REPAIR

Use the paired review prompt in the Activity guide. Fix the first causal failure, preserve tests, then rerun focused and full checks.

Activity 10 · Evidence Contract

1 Artifact

Working app, complete tests, architecture pack and deployment checklist

A1, A2, A3, A4, A5

2 Acceptance

A clean evidence index proves requirements, implementation, interoperability, documentation and deployment readiness without claiming App Store submission.

A1, A2, A3, A4, A5

3 Failure signal

When a system test fails, localise the fault by rerunning domain then bridge tests before editing UI code.

A1, A2, A3, A4, A5

4 Cleanup

Remove local build caches and secrets; retain source, fixtures, tests and evidence pack.

A1, A2, A3, A4, A5

DESIGN JUDGEMENT

Evidence is a retrievable artifact or test result, not a statement that the activity was completed.

Activity 10 · Verification Gate

PASS WHEN

A clean evidence index proves requirements, implementation, interoperability, documentation and deployment readiness without claiming App Store submission.

IF IT FAILS

When a system test fails, localise the fault by rerunning domain then bridge tests before editing UI code.

LEAVE THE WORKSPACE

Remove local build caches and secrets; retain source, fixtures, tests and evidence pack.

Topic 5 · Review

1 Explain

LO5: Generate C++ design documentation aligned with user specifications.

LO5 · K5 · A5

2 Demonstrate

Show the Activity evidence that operationalises the outcome.

LO5 · K5 · A5

3 Diagnose

Name the failure signal and the layer where it originates.

LO5 · K5 · A5

4 Trace

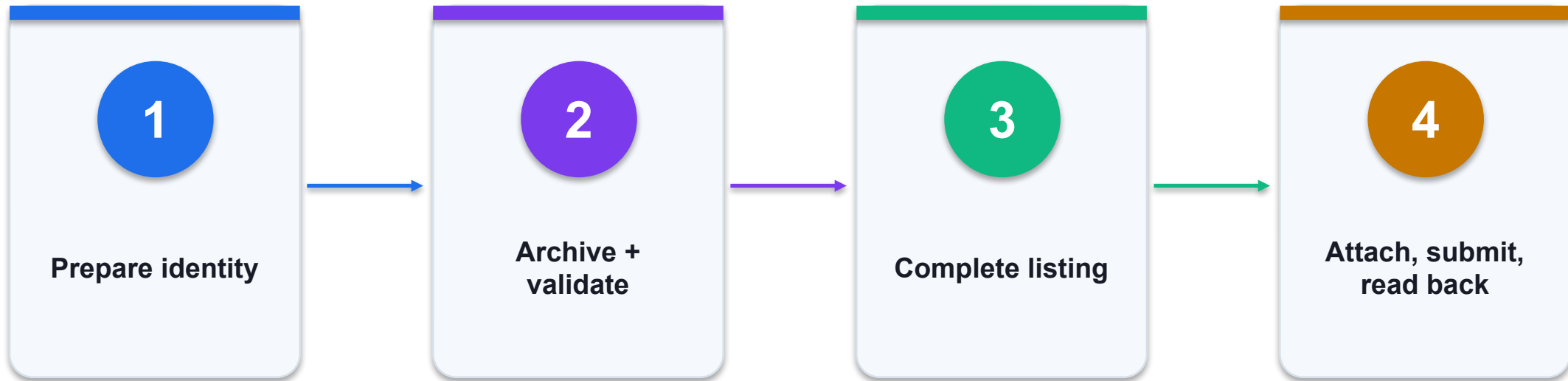
Connect LO5 · K5 · A5 to the assessment and evidence pack.

LO5 · K5 · A5

DESIGN JUDGEMENT

Do not advance on confidence alone; point to the artifact or test result that supports the answer.

App Store Connect · Evidence Gates, Not a Submission Claim



MECHANISM

Local PASS covers XcodeGen, build, tests, Simulator and reviewed assets. Signing, upload, metadata, screenshots, privacy, age rating, availability, build attachment and submission remain HOLD until executed and read back by an authorised trainer.

App Store Connect · Bounded Release Prompt

Audit the selected app identity, version/build, Release settings, opaque 1024px icon, privacy manifest, required-reason APIs and test evidence. Prepare complete archive/export/upload commands and checklists for metadata, real-app screenshots, App Privacy, age rating, rights, price and explicit availability. Then define processed-build attachment, submission and Apple-side readback evidence. Never invent identifiers or credentials; never claim a gated action ran. Mark signing, upload and real ASC mutations
TRAINER DEMO / FUTURE CREDENTIALLED GATE and HOLD until read back.

LOCAL PASS

Generate · build · Swift Testing · XCTest UI · Simulator · icon/privacy inspection

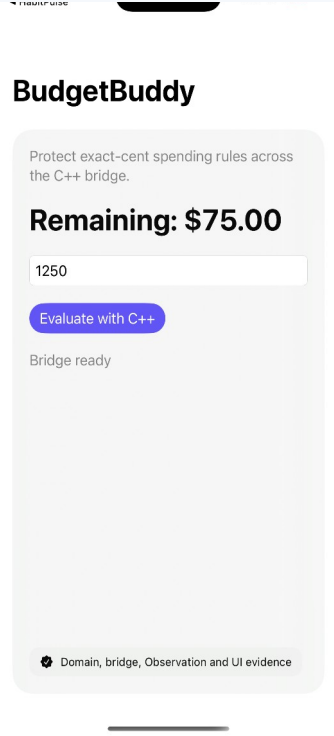
EXTERNAL HOLD

Sign · archive · upload · listing · privacy · age · availability · submit

DONE ONLY WHEN

Identity, build, availability and review state are read back from Apple

Three Apps · Proven in the iPhone Simulator



BudgetBuddy

Protect exact-cent spending rules across the C++ bridge.

Remaining: \$75.00

1250

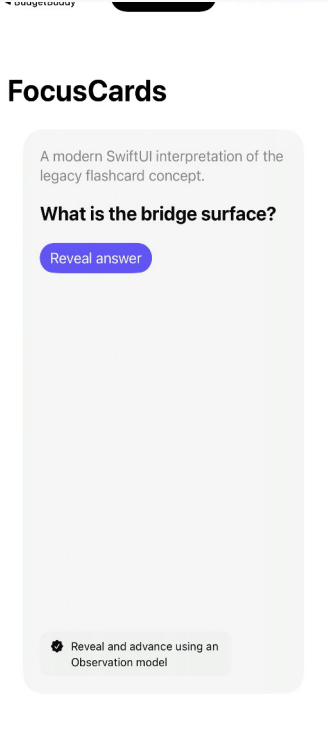
Evaluate with C++

Bridge ready

Domain, bridge, Observation and UI evidence

BUILD + TEST + LAUNCH

Exact-cent save routes through Objective-C++ into the C++ policy.



FocusCards

A modern SwiftUI interpretation of the legacy flashcard concept.

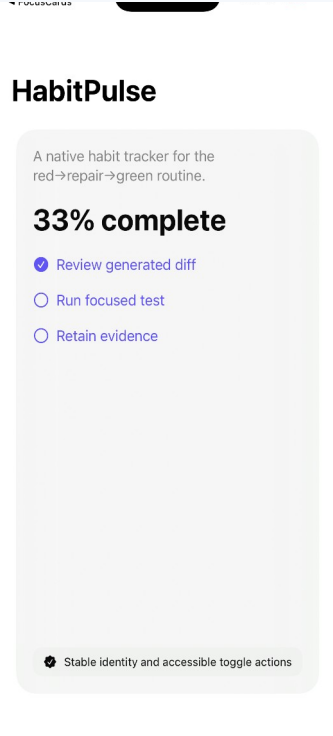
What is the bridge surface?

Reveal answer

Reveal and advance using an Observation model

BUILD + TEST + LAUNCH

Reveal/advance state is generated with Observation and verified by tests.



HabitPulse

A native habit tracker for the red→repair→green routine.

33% complete

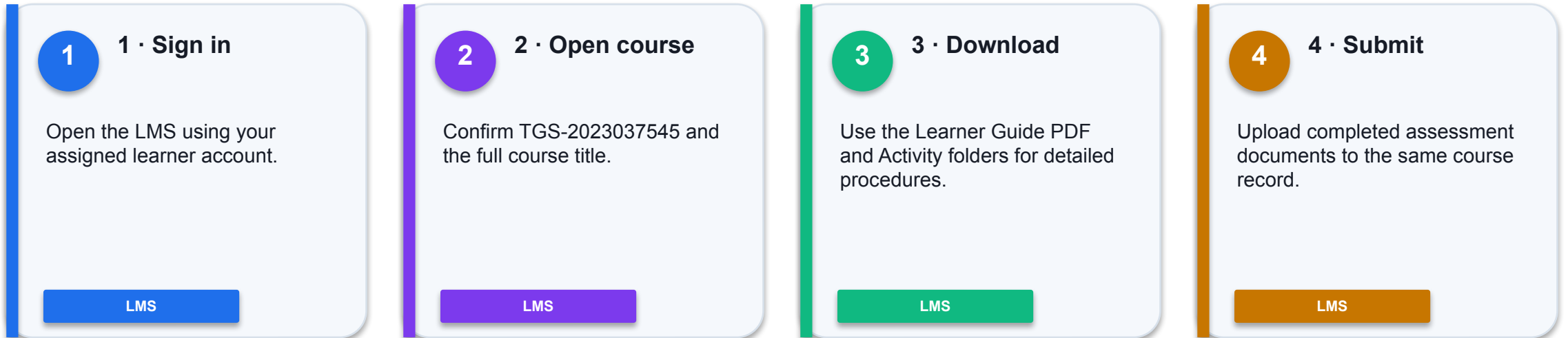
- Review generated diff
- Run focused test
- Retain evidence

Stable identity and accessible toggle actions

BUILD + TEST + LAUNCH

Stable-identity habit actions expose the red → repair → green routine.

Download Course Material

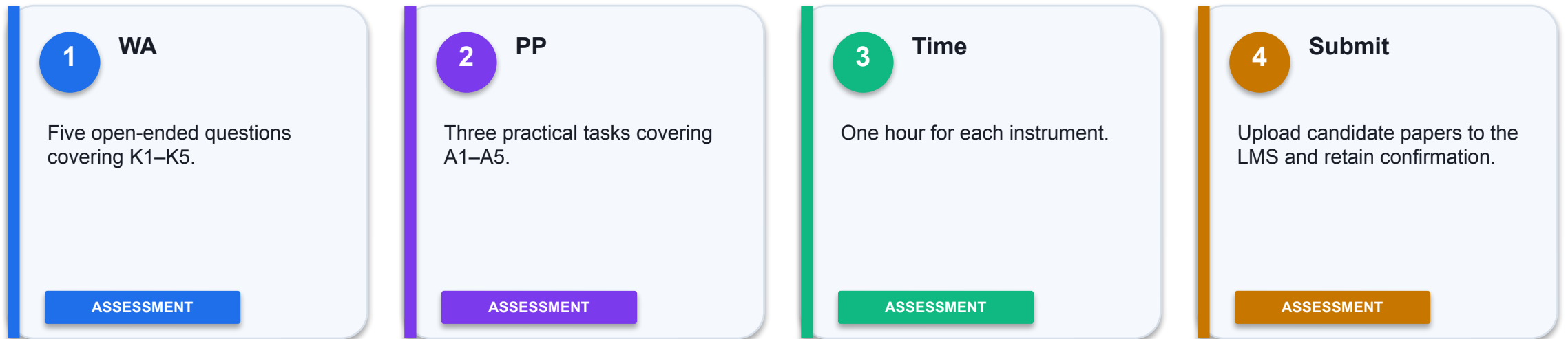


DESIGN JUDGEMENT

Always confirm course identity before downloading or submitting.

<https://lms-tms.tertiaryinfotech.com/>

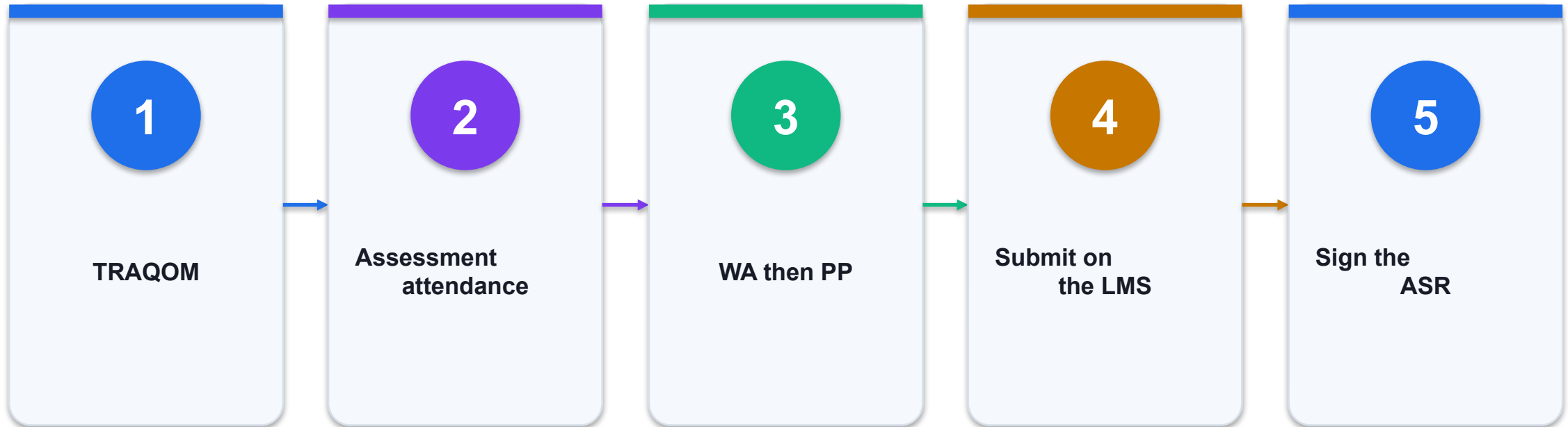
Assessment



DESIGN JUDGEMENT

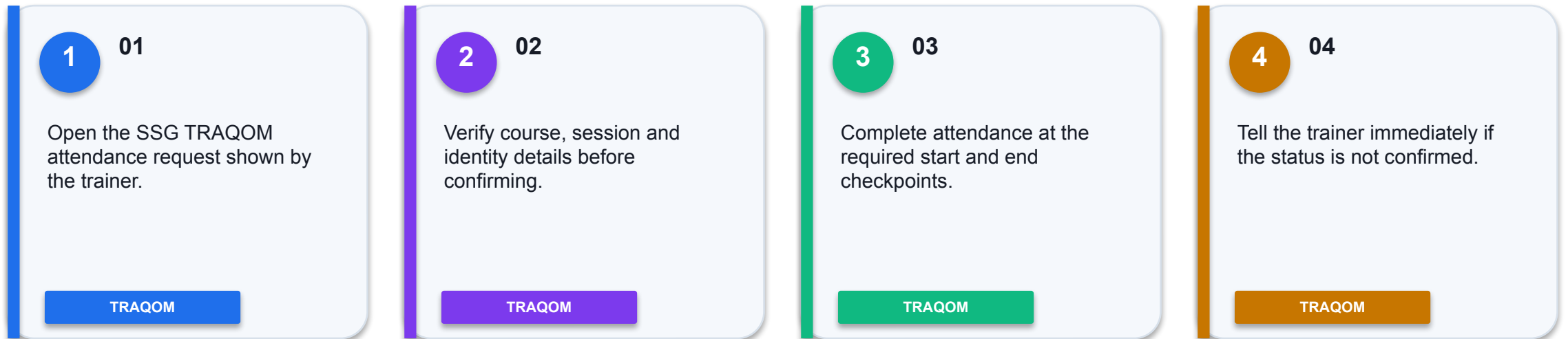
Answer keys remain trainer-only.

Assessment Flow



ASR = Assessment Summary Record · complete the sign-off only after both instruments are submitted.

Digital Attendance · Mandatory Close



DESIGN JUDGEMENT

TRAQOM records SSG digital attendance. It is not a course survey.

THANK YOU

**Build the boundary.
Prove the behaviour.
Document the decision.**

Tertiary Infotech Academy Pte Ltd · UEN 201200696W
enquiry@tertiaryinfotech.com · +65 6100 0613